

# 复杂场景下多模态情感预测的数学建模与算法设计

## 摘 要

多模态情感预测是情感计算与多模态学习交叉的重要问题，在自然人机交互、智能客服与心理健康监测等场景中有广泛的应用需求。真实环境中，设备异常、画面遮挡与转写失败都可能造成某一模态的连续时段失效，模型需要在信息不完整时仍给出稳定可靠的情感极性与强度估计，鲁棒预测与可信解释因此成为该问题落地的两个核心要求。针对这一问题，本文提出一套覆盖特征对齐、鲁棒建模与可解释分析的完整研究方案，以赛题给定的 CMU-MOSEI 派生数据为唯一数据来源，先以词级时间区间为公共锚点完成三模态特征提取与时序对齐，再建立掩码感知的多模态鲁棒融合模型，借助课程式缺失注入、动态专家门控与教师-学生蒸馏应对不同类型的局部缺失，进而在冻结的预测骨干上搭起模态级、片段级与证据级三层解释通路，对预测依据逐项做量化核验。整套方案在缺失场景下同时保住了预测稳定性与解释可复核性。

针对问题一，以词级时间区间作为三模态共同的对齐基准，将文本词序列、语音帧和视觉帧映射到统一的 50 个时间步，并根据零值结构推断每个样本的有效长度，用有效性掩码把尾部填充与有效区间内的真实缺失分开标注。该方案为全部样本建立了可追溯、可核验的输入规范，输出的特征文件与掩码可直接为后两问复用；示意样本的掩码时序对照显示，填充位与缺失位在掩码空间中完全可分离。

针对问题二，设计了掩码感知的多模态鲁棒融合模型 MRF-Net。编码器把缺失指示与可见指示并入每个时间步的输入，融合层先做共享-特有分解保留模态间的共性与个性，再由动态专家门控按样本的缺失结构分配专家权重；由于训练集中的真实缺失极少，训练阶段采用课程式缺失注入与教师-学生蒸馏，让模型从轻微的单模态缺失逐步适应到较深的多模态混合缺失。验证集上，模型取得  $MAE=0.5866$ 、 $ACC=0.6223$ 、 $Macro-F1=0.6158$ 、 $Pearson=0.6543$ ；测试划分上  $ACC=0.6534$ 、 $Macro-F1=0.6338$ ，二分类口径准确率为 0.8514。对 120 个缺失场景的三因素方差分析表明，性能退化主要由模态类型与缺失率的交互项主导，MRF-Net 将该交互的效应量由 0.9517 降至 0.8415，不同缺失位置下的整体退化幅度比朴素基线低 1.5 至 3.3 倍。训练损失曲线、六类缺失的退化曲线、消融对比与错误归因等可视化结果一致支持上述结论；在训练规模约为公开划分五分之一的条件下，二分类口径 0.8514 已达到公开干净设定的量级。

针对问题三，沿用冻结的预测骨干，完成模态级、片段级和证据级三层解释，其中模态级用精确枚举的 Shapley 值衡量各模态贡献，片段级结合积分梯度与遮挡分析定位关键区段，证据级把重要片段映射回原始文本、语音时段和视频关键帧。验证集全局 Shapley 值为文本 0.1842、语音 0.0004、视觉 0.0103；仅保留文本时平均绝对误差为 0.5936，已接近全模态模型的 0.5866；解释排序与问题二删模态退化结果的秩相关系数为 1.000，与附件三实测缺失率的免标注检验相关系数为 -0.4011。典型样本解释卡、模

态作用度分布、一致性散点与片段重要性热力图等可视化结果与统计结论相符；三源融合解释在忠实性与稀疏性之间取得折中，综合表现优于任何单一来源。

综合各项实验，本文模型在局部模态缺失条件下保持了稳定的预测性能。消融实验表明性能来源于掩码感知、缺失注入与门控融合的整体配合；缺失场景扫描、错误归因与拒识分析从不同角度验证了模型的可靠性；解释结论通过了删模态退化与附件三实测缺失两项独立检验。以上结果说明，该方案在鲁棒预测与可解释性两方面都经受住了独立检验，能够支撑复杂场景下多模态情感预测的实际需求。

**关键字：** 多模态情感预测；局部模态缺失；掩码感知融合；动态专家门控；可解释性评价

# 目录

|                                   |    |
|-----------------------------------|----|
| 一、问题重述 . . . . .                  | 5  |
| 1.1 问题背景 . . . . .                | 5  |
| 1.2 本文拟解决的问题 . . . . .            | 5  |
| 二、模型假设与符号说明 . . . . .             | 6  |
| 2.1 模型的基本假设 . . . . .             | 6  |
| 2.2 模型符号说明 . . . . .              | 6  |
| 三、技术路线图 . . . . .                 | 8  |
| 四、问题一模型的建立与求解 . . . . .           | 8  |
| 4.1 问题分析 . . . . .                | 8  |
| 4.2 模型建立与求解 . . . . .             | 8  |
| 4.3 结果分析与可视化 . . . . .            | 9  |
| 4.4 本章小结 . . . . .                | 9  |
| 五、问题二模型的建立与求解 . . . . .           | 9  |
| 5.1 问题分析 . . . . .                | 9  |
| 5.2 数据处理与分析 . . . . .             | 10 |
| 5.2.1 数据说明与预处理 . . . . .          | 10 |
| 5.2.2 模态缺失的数学刻画与模拟设计 . . . . .    | 11 |
| 5.2.3 缺失分布标定与输入条件 . . . . .       | 13 |
| 5.3 模型建立 . . . . .                | 13 |
| 5.3.1 掩码感知编码器 . . . . .           | 13 |
| 5.3.2 动态专家门控融合 . . . . .          | 15 |
| 5.3.3 多任务输出与目标函数 . . . . .        | 17 |
| 5.3.4 缺失鲁棒训练策略 . . . . .          | 18 |
| 5.3.5 决策阈值选择与校准 . . . . .         | 20 |
| 5.4 结果分析与可视化 . . . . .            | 22 |
| 5.4.1 评价指标与评估口径 . . . . .         | 22 |
| 5.4.2 基础性能评价 . . . . .            | 23 |
| 5.4.3 缺失模态类型、位置与时长的影响规律 . . . . . | 25 |

|                                      |           |
|--------------------------------------|-----------|
| 5.4.4 消融实验与超参数搜索 . . . . .           | 29        |
| 5.4.5 错误归因与不确定性分析 . . . . .          | 31        |
| 5.4.6 附件三测试集预测结果汇总与展示 . . . . .      | 34        |
| 5.5 本章小结 . . . . .                   | 37        |
| <b>六、问题三模型的建立与求解 . . . . .</b>       | <b>37</b> |
| 6.1 问题分析 . . . . .                   | 37        |
| 6.2 模型建立 . . . . .                   | 38        |
| 6.2.1 可解释性的形式化定义与方案选择 . . . . .      | 38        |
| 6.2.2 可解释性证据头的结构与目标函数 . . . . .      | 40        |
| 6.2.3 模态级与片段级解释的计算 . . . . .         | 40        |
| 6.2.4 证据定位规则 . . . . .               | 42        |
| 6.3 结果分析与可视化 . . . . .               | 43        |
| 6.3.1 评价指标与解释质量度量 . . . . .          | 43        |
| 6.3.2 典型样本解释卡 . . . . .              | 43        |
| 6.3.3 三模态作用差异与一致性检验 . . . . .        | 46        |
| 6.3.4 片段级证据定位与可读性核查 . . . . .        | 49        |
| 6.3.5 附件四预测与解释结果汇总 . . . . .         | 51        |
| 6.4 本章小结 . . . . .                   | 52        |
| <b>七、模型评价与推广 . . . . .</b>           | <b>52</b> |
| 7.1 模型的优点 . . . . .                  | 52        |
| 7.2 模型的缺点 . . . . .                  | 53        |
| 7.3 模型改进与推广 . . . . .                | 53        |
| <b>参考文献 . . . . .</b>                | <b>53</b> |
| <b>A 支撑材料文件清单 . . . . .</b>          | <b>56</b> |
| <b>B 不确定性与拒识能力分析（补充实验） . . . . .</b> | <b>57</b> |
| <b>C 核心代码 . . . . .</b>              | <b>57</b> |



# 一、问题重述

## 1.1 问题背景

情感分析是具身智能与自然人机交互等领域的关键支撑技术之一。真实交互中，人类情绪通过文本语义、语音韵律、面部表情与肢体动作等多通道信号协同表达，多模态情感预测因此成为情感计算与多模态学习交叉的重要研究方向，在理论上需要刻画不同模态在情感表达中的互补与冗余关系，在应用上则支撑智能客服、心理健康监测与舆情分析等场景在复杂环境中给出可靠的情感极性与强度估计 [2]。

现有研究普遍采用深度网络进行多模态融合，借助注意力机制、图网络与预训练模型挖掘模态间的关联，在信息完整的基准数据集上取得了较高的识别精度。然而在实际工作中，这类方法面临明显的困难。真实采集环境里，设备异常、画面遮挡与转写失败都会造成某一模态的连续时段失效，而主流融合模型默认各模态完整可用，缺失值被置零后仍参与编码与聚合，性能随之下降；已有面向缺失的方法多依赖生成式补全或统一表示，补全质量难以保证，训练成本也较高。同时，深度模型大多是黑箱，难以说明判断主要依赖哪一种模态、哪些时间片段推动了判断，结果缺少可核验性。

针对上述困难，本文提出一套显式建模缺失并兼顾可解释性的完整方案，以词级时间区间为公共锚点完成三模态特征提取与时序对齐，将缺失状态编码为掩码并让掩码全程参与编码、池化与融合，配合动态专家门控与课程式缺失注入，使模型直接学习如何在不完整信息下做判断，并搭建起模态级、片段级与证据级三层解释通路，把预测依据落到可复核的原文词句、语音时段与关键帧。与依赖补全的方法相比，该方案不需要生成任何缺失数据，缺失处理过程透明可查；与纯预测模型相比，它在给出预测的同时附带经过独立检验的解释，鲁棒性与可信度都更经得起检验。赛题给出的三项任务与这一方案对应，先完成三模态特征提取与时序对齐，再研究局部模态缺失条件下的鲁棒情感预测，进而给出可量化、可复核的解释。

## 1.2 本文拟解决的问题

本文需要完成以下三项任务。

**问题一（多模态特征提取与时序对齐）：**基于附件一原始视频，分别提取文本、语音和视觉特征，并以词级时间区间作为公共锚点，使不同采样频率的特征对应到统一的 50 个时间步；对每个时间步同步记录有效性信息，以区分真实数据、尾部填充和有效区间内的缺失。

**问题二（模态缺失条件下的鲁棒情感预测）：**本文把缺失状态显式编码为掩码，使编码器、池化层和融合层能够识别不可用时间步；针对训练数据中真实缺失样本极少的问题，在训练阶段按照预设比例和位置注入连续缺失，并用附件三的实测缺失分布标定注入范围。随后从缺失模态类型、位置和时长三个维度进行系统扫描，分析性能退化规

律，并完成附件三的预测。

**问题三（可解释性情感预测）：**在不改变问题二预测骨干的前提下，分别从模态、时间片段和原始证据三个层次展开解释，先用 Shapley 值衡量模态贡献，再利用积分梯度和遮挡法定位关键时间步，把重要性回映到文本字符区间、语音时段和视觉关键帧。解释与问题二预测来自同一模型，两部分结果可以直接对照。

模型结构、超参数和决策阈值均只在附件二验证集上确定；附件三、附件四不参与任何模型选择，仅用于最终专项推理。

## 二、模型假设与符号说明

### 2.1 模型的基本假设

结合题意和数据结构，本文作如下假设。

**假设一（样本独立性）：**同一数据划分中的不同样本相互独立，不同样本的缺失模式不建立额外相关关系。该假设用于后续验证集指标统计和方差分析。

**假设二（填充非缺失）：**序列尾部连续全零部分视为填充，而非信息缺失。原因在于附件二缺少显式长度字段，且全零区间集中出现在序列尾部；若将其计入缺失，会系统性高估缺失率。

**假设三（连续零判缺）：**有效区间内连续不少于两帧的全零片段视为局部缺失，孤立单帧全零不直接判缺，以避免将特征归一化后自然出现的零值误判为缺失。该规则同时用于实测缺失统计和训练期缺失注入。

**假设四（编码同源）：**附件三、附件四的文本模态与附件二文本特征同源。编码一致性检验的余弦相似度为 1.0000，可支持后续统一建模。

**假设五（标注噪声）：**情感强度标签存在主观标注差异，因此连续回归结果可能向条件均值收缩。本文在误差分析中单独考察这一现象。

**假设六（缺失可迁移）：**训练阶段模拟注入的缺失类型、位置和强度与附件三实测分布处于同一量级，从而使缺失鲁棒性能够迁移到专项测试场景。

### 2.2 模型符号说明

全文所用主要符号、含义与单位汇总于表 1。符号按在正文中首次出现的顺序排列，后续各节的公式与产物均沿用同一套符号。

表 1 全文主要符号、含义与单位对照

| 符号                               | 含义                                                          | 单位 |
|----------------------------------|-------------------------------------------------------------|----|
| $m$                              | 模态编号, $m \in \{\text{t}, \text{a}, \text{v}\}$ 分别表示文本、语音、视觉 | —  |
| $T$                              | 对齐后的时间步数 (统一取 $T = 50$ )                                    | 帧  |
| $\mathbf{X}^m$                   | 第 $m$ 模态的特征矩阵, $\mathbf{X}^m \in \mathbb{R}^{T \times d_m}$ | —  |
| $d_m$                            | 第 $m$ 模态的特征维度                                               | —  |
| $\mathbf{P}^m$                   | 填充掩码, $P_k^m = 1$ 表示第 $k$ 段为尾部填充                            | —  |
| $\mathbf{M}^m$                   | 缺失掩码, $M_k^m = 1$ 表示第 $k$ 段在有效区间内不可用                        | —  |
| $L_m$                            | 第 $m$ 模态的有效长度                                               | 帧  |
| $\rho_m$                         | 第 $m$ 模态的缺失率                                                | —  |
| $\mathbf{e}_m(t)$                | 第 $m$ 模态第 $t$ 帧的编码表示                                        | —  |
| $\mathbf{s}(t), \mathbf{d}_m(t)$ | 跨模态共享分量与第 $m$ 模态的特有分量                                       | —  |
| $g_m$                            | 模态门控权重                                                      | —  |
| $\mathbf{z}$                     | 融合后的样本级表示                                                   | —  |
| $\mu, \sigma^2$                  | 情感强度的预测均值与预测方差                                              | —  |
| $\pi$                            | 极性分类头的三类概率                                                  | —  |
| $\theta$                         | 极性决策阈值                                                      | —  |
| $\alpha, \beta$                  | 强度仿射校准的斜率与截距                                                | —  |
| $\varphi_m$                      | 模态 $m$ 的 Shapley 作用度                                        | —  |
| $\mathbf{w}^m$                   | 模态 $m$ 内各时间步的重要性向量                                          | —  |
| $y, c$                           | 情感强度真值与极性类别真值                                               | —  |
| KL                               | Kullback–Leibler 散度                                         | —  |

### 三、技术路线图

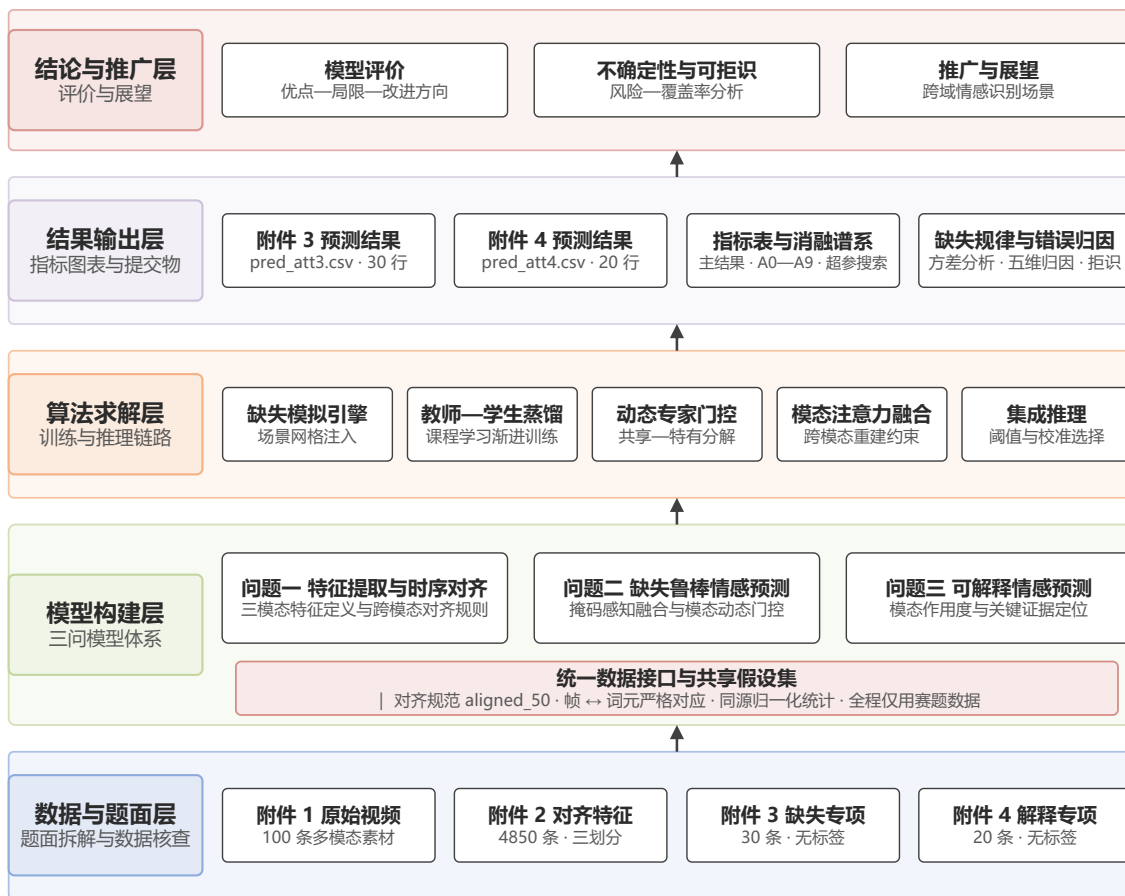


图 1 本文总体建模架构（五层技术路线）

### 四、问题一模型的建立与求解

本章为骨架占位：问题一的特征提取与时序对齐流程尚在完善中，待全量特征结果完成后补充（提交前必须补齐本节内容，否则与摘要及后文相关表述不一致）；其输入接口口径已在第 2 章的假设与符号体系中固化。

#### 4.1 问题分析

（待补：明确附件一的输入形式、三模态输出特征、统一时间轴的定义，以及特征提取与对齐过程中最需要解决的时间粒度差异。）

#### 4.2 模型建立与求解

（待补：给出文本、语音、视觉三模态的具体特征定义、提取工具及版本，说明词级时间锚点的构造方式、跨模态聚合规则和最终 50 步序列的生成过程。）

### 4.3 结果分析与可视化

（待补：至少给出对齐后的有效长度统计、典型样本的跨模态时间线，以及能够证明“词级对齐有效”的定量或可视化结果。）

### 4.4 本章小结

（待问题一实验完成后补写。本节概括本章完成了什么、输出了什么，不再重复方法细节。）

## 五、问题二模型的建立与求解

### 5.1 问题分析

实际采集中的局部缺失通常不是整条模态完全消失，而是某一连续时间段失效；缺失值被置零后仍会参与编码和聚合，固定权重融合模型无法识别这些不可用区段，预测因而受到明显干扰，而附件二训练集中的真实缺失样本极少（有效区间内真缺失率分别为文本 0、语音约 0.0004、视觉约 0.01），模型无法仅靠现有数据学会应对缺失。因此，本问一方面需要让模型在训练数据无法覆盖所有真实缺失模式的情况下，仍能对不同模态、不同位置 and 不同长时的局部缺失保持稳定，另一方面需要定量说明缺失类型、位置与时长对性能的影响，其结论也作为问题三解释分析的对照基准。这两项要求的必要性直接体现在提交任务上。附件三的 30 条样本中，语音与视觉分别有 17 条和 18 条包含连续缺失片段且以头部为主，本问的输出质量决定专项测试的提交结果。

本问的求解可以拆分为四个环节。从数据准备与缺失标定开始，针对附件二无长度字段、填充与缺失易混淆的特点，由尾部全零结构推断有效长度，构造填充掩码与缺失掩码把两类零严格分离，并让训练期注入的比例与位置按附件三的实测分布标定；该口径统一、可逐样本复核，为后续环节准备了干净一致的输入。随后建立掩码感知的鲁棒预测模型。编码器把缺失指示与可见指示并入每个时间步的输入，融合层先做共享-特有分解保留模态间的共性与个性，再由动态专家门控按样本的缺失结构分配专家权重；该方案不依赖生成式补全，缺失处理过程透明可查，得到同时输出情感极性与强度的预测骨干。进而开展课程式鲁棒训练。训练针对真实缺失极少的问题，按由易到难的课程注入缺失，借助教师-学生蒸馏拿到接近完整的监督，并用指数滑动平均稳定参数；这一安排避免了训练早期因缺失过强而优化不稳，使模型平稳地从轻度缺失过渡到较深缺失，训练完成后在验证集上选定阈值并冻结交付权重。评测与推广分析紧随其后，在统一协议下扫描 120 个缺失场景并做三因素方差分析，配合模块消融、多维错误归因与不确定性拒识，不仅给出性能均值，还定位主导退化的因素与典型失效模式，并完成附件三的推理与提交文件生成。整体求解流程如图 2 所示。

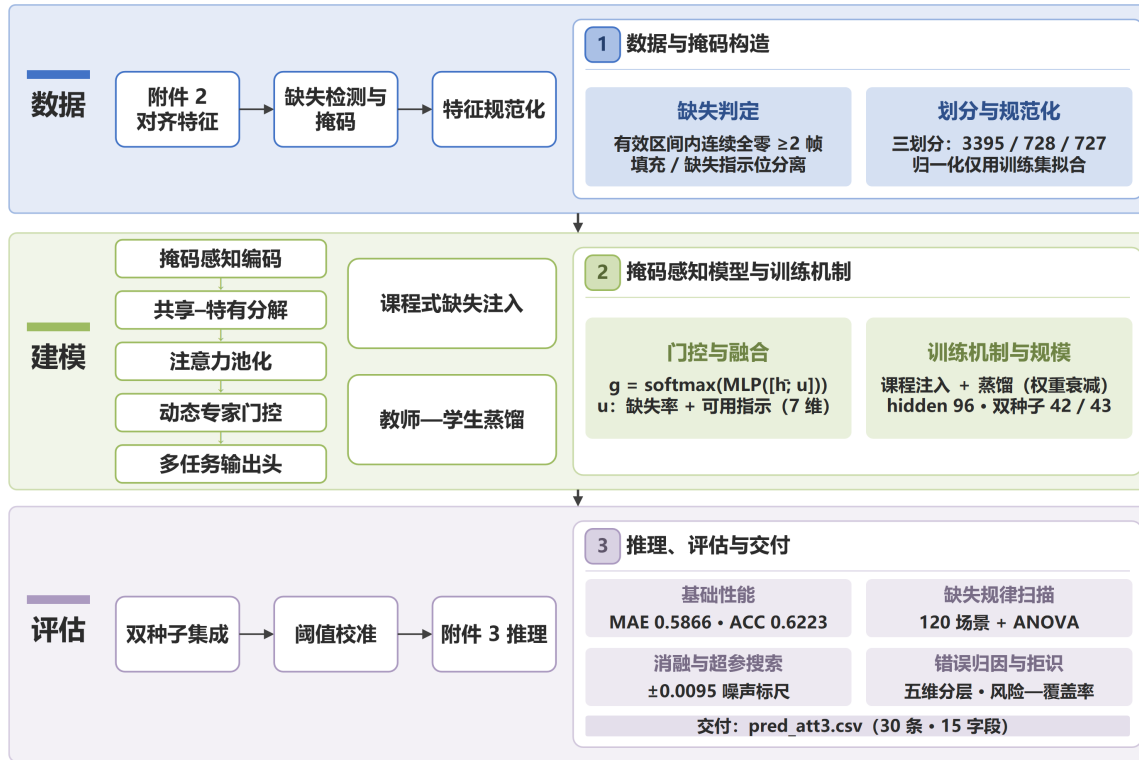


图 2 问题二的求解流程

## 5.2 数据处理与分析

### 5.2.1 数据说明与预处理

附件二的对齐版本以 Python 字典形式组织，包含 train/valid/test 三个划分，每条样本包含原始转写、三模态特征、三分类标签与区间  $[-3, 3]$  内的连续情感强度标签。与赛题说明相比，实际文件缺少 audio\_lengths、vision\_lengths 和 annotations 三个字段，因此需要在读入阶段补充有效长度推断与标准化两步处理。

(1) **有效长度推断**：对每个样本和每个模态，从序列末端向前寻找最后一个非零时间步，并将其及之前的时间步定义为有效区间；有效区间内连续不少于两帧的全零片段判为局部缺失。该规则同时用于训练阶段的缺失注入和附件三的实测缺失统计，保证不同实验使用同一口径。

(2) **标准化**：三模态特征按维度进行 z-score 标准化，均值和标准差只使用训练集估计，避免验证集和测试划分的信息进入训练过程；原始填充位置不参与统计，也不因标准化而改变。

数据概况如表 2 所示。文本模态由预训练语言模型编码得到，有效长度恒为 50 帧；语音与视觉的平均有效长度分别约为 24.6 与 23.8 帧，因此同一缺失率水平下，文本损失的绝对帧数约为另两者的两倍。这一口径差异在 5.4.3 节做了专门的稳健性核验。

**表 2 数据划分与特征概况**

| 项目           | 数值                                                        |
|--------------|-----------------------------------------------------------|
| 样本总数         | 4850 (训练 3395 / 验证 728 / 测试 727)                          |
| 特征形状         | 文本 (N,50,768); 语音 (N,50,74); 视觉 (N,50,35)                 |
| 类别分布 (训练集)   | 负向 967 / 中性 758 / 正向 1670                                 |
| 有效长度 (验证集实测) | 文本 50.0; 语音 24.59 (10 至 90 分位 11 至 44); 视觉 23.82 (9 至 43) |
| 填充率          | 约 0.49 至 0.54                                             |
| 有效区间内真缺失率    | 文本 0; 语音 0.0004; 视觉约 0.01                                 |
| 缺失字段         | 无 audio_lengths / vision_lengths / annotations            |

附件三、附件四逐文件存储（每条样本一个文件），且没有预计算文本特征。本文使用与附件二一致的预训练语言模型重新编码转写文本，并通过同源样本余弦相似度 1.0000 验证编码一致性，特征版本也与附件二保持一致。附件四属于问题三专项数据，在问题二训练与模型选择阶段不使用。

### 5.2.2 模态缺失的数学刻画与模拟设计

设样本第  $m$  个模态 ( $m \in \{t, a, v\}$ ) 的特征矩阵为  $\mathbf{X}^m \in \mathbb{R}^{T \times d_m}$ ，其中  $T = 50$  为序列长度；缺失位置由二值掩码  $\mathbf{M}^m \in \{0, 1\}^T$  刻画， $M_k^m = 1$  表示第  $k$  段不可用。缺失后的特征与缺失率分别定义为

$$\tilde{\mathbf{X}}^m = \mathbf{X}^m \odot (1 - \mathbf{M}^m), \quad \rho^m = \frac{\sum_{k=1}^{L_m} M_k^m}{L_m}, \quad (1)$$

其中  $L_m$  为该样本模态  $m$  的有效区间长度，式 (1) 给出的缺失率即全章统一采用的实测口径。训练阶段只在有效区间内注入连续缺失：将长度为  $L = \text{round}(\rho L_m)$  的随机连续片段置零，并枚举头部、中部、尾部与随机四种位置，同时保留对应掩码，使训练阶段注入的缺失与推理阶段检测到的缺失使用同一数据表示。图 3 以一条真实样本为例展示两类掩码的时序对照，填充位于序列尾部、不携带任何信息；缺失位于有效区间内部、可以出现在任意相对位置；二者在掩码空间中完全可分离（图中灰底为尾部填充位、彩色段为有效区间、深色条为缺失片段）。正是这一可分离性，支持 5.3.1 节把掩码作为显式输入而非隐含信息。

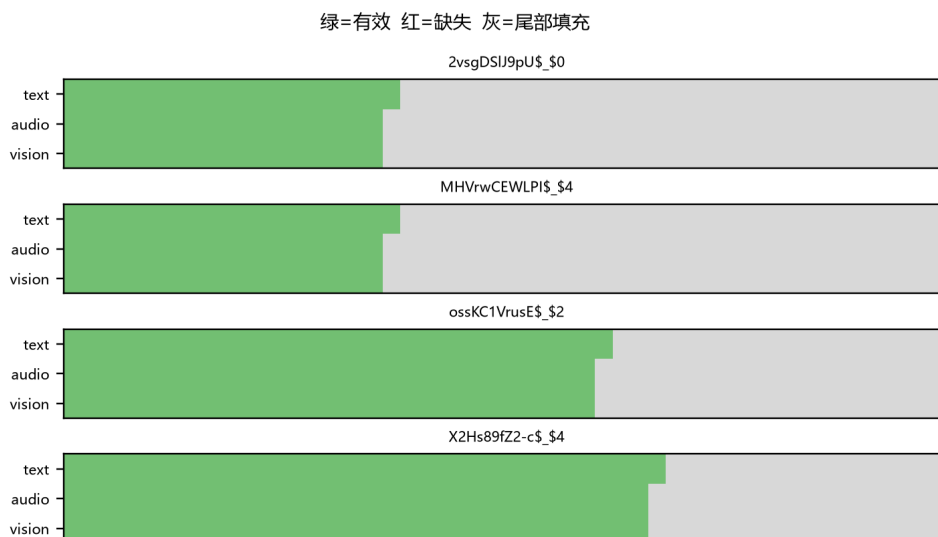


图 3 填充掩码与缺失掩码的时序可视化（示例样本）

为了让模型在训练中见到足够多样的缺失结构，本章设计了一套随机化注入协议。每个训练批次随机抽取缺失模态组合、缺失位置和缺失强度。缺失类型包括仅文本、仅语音、仅视觉及两两组合共 6 类，位置包括头部、中部、尾部和随机 4 类，每个模态允许出现 1 至 3 段碎片化缺失。缺失强度按课程学习逐步增加，训练前期只注入单模态低强度缺失（ $\rho \leq 0.10$ ），中期加入多模态混合缺失（ $\rho$  约 0.04 至 0.20），后期把强度上限提高到 0.25。训练全程保留完整标签，缺失只通过掩码注入特征层，注入与检测共用同一套掩码判定规则，训练阶段的缺失形态与推理阶段保持一致。

需要注意，赛题要求分析缺失时长的影响，而本章的注入实验与场景扫描均以缺失率为控制变量，两者的换算关系需要先厘清（表 3）。文本的有效长度恒为 50 帧，缺失率 0.1、0.2、0.3、0.5 分别对应 5、10、15、25 帧；语音与视觉的平均有效长度仅约 24 帧，同一缺失率对应的绝对帧数大约只有文本的一半。这意味着相同的缺失率并不代表相同的缺失时长，缺失率 0.3 在文本上意味着 15 帧不可用，在视觉上只有约 7 帧。为排除这一口径差异对结论的干扰，5.4.3 节将在匹配绝对缺失帧数的条件下重做关键对比。

表 3 缺失率与平均绝对缺失帧数的换算

| 缺失率 | 文本（帧） | 语音（帧） | 视觉（帧） |
|-----|-------|-------|-------|
| 0.1 | 5.0   | 2.5   | 2.4   |
| 0.2 | 10.0  | 4.9   | 4.8   |
| 0.3 | 15.0  | 7.4   | 7.2   |
| 0.5 | 25.0  | 12.3  | 11.9  |
| 1.0 | 50.0  | 24.6  | 23.8  |



### 5.2.3 缺失分布标定与输入条件

附件三共有 30 条样本，其实测缺失结构如表 4 与图 4 所示（左轴为含缺失样本条数、右轴为平均缺失率）。表中缺失率按 5.2.1 节的统一定义逐样本统计，位置比例按缺失帧落在有效区间前、中、后三段的份额计算。文本模态没有任何实测缺失；语音与视觉分别有 17 条（56.7%）与 18 条（60.0%）含缺失，平均缺失率为 0.1011 与 0.1072，90 分位约 0.224、最大值约 0.40；缺失位置偏有效区间前部（语音头部占比 0.46、视觉 0.43）。总体上，附件三属于中等强度、以视听模态为主、偏向头部的局部缺失场景。

表 4 附件三实测缺失结构（ $n = 30$ ）

| 模态 | 含缺失样本        | 平均缺失率  | 90 分位 | 最大值   | 位置（头/中/尾）          |
|----|--------------|--------|-------|-------|--------------------|
| 文本 | 0/30         | 0.0000 | —     | —     | —                  |
| 语音 | 17/30（56.7%） | 0.1011 | 0.224 | 0.400 | 0.46 / 0.27 / 0.27 |
| 视觉 | 18/30（60.0%） | 0.1072 | 0.224 | 0.400 | 0.43 / 0.29 / 0.29 |

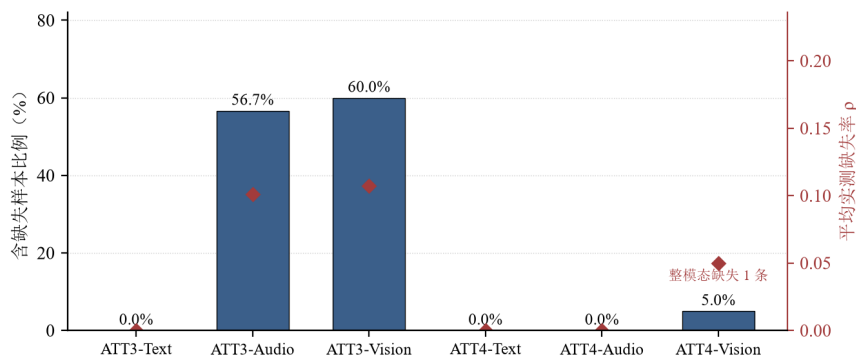


图 4 附件三与附件四的模态缺失分布

有两处结构差异需要交代清楚。一处是位置先验的取向，注入协议默认以中部为主，而附件三实测以头部为主；前期对照实验表明，把位置先验调成头部为主并不会带来稳定收益（视听局部缺失造成的退化不足 1%），因此本章仍采用原注入协议。另一处在于监督的空白，附件三与附件四都没有文本模态实测缺失，文本分支的鲁棒性只能依靠训练期注入检验，本章在 5.5 节小结中一并说明这一局限。

## 5.3 模型建立

### 5.3.1 掩码感知编码器

掩码感知编码器是问题二模型的输入侧基础模块。其基本原理是在特征数值之外显式携带每个时间步的可用性状态，使网络始终能够区分真实观测的零与缺失造成的零，并把这一区分持续传递到池化与融合阶段。编码器处理的是一条带可用性标注的残缺序列，而不是被零填满的伪完整序列。该编码器由输入表示、局部卷积、自注意力编码与

共享-特有分解四个环节串联组成，三种模态结构相同、参数彼此独立；其输入构造与有效位加权池化的示意如图 5 所示。下面先说明这一设计的取舍，再按数据流顺序展开。

在路线选择上，缺失序列的常见建模方式有三条，分别是先补全并按完整序列处理、直接舍弃缺失片段，以及保留原样、仅在损失层面做鲁棒化。第一条途径的补全值并非真实观测，其偏差会沿时间轴向下游传播，甚至把模型引向虚假的情感线索；第二条途径既破坏时间轴的对齐结构，也丢失了缺失位置本身携带的信息，而本赛题中缺失源于静音、黑屏等真实因素，缺失的位置与形态往往与内容相关；第三条途径则始终无法让网络区分真实观测的零与缺失造成的零。我们在第三条途径上再进一步，采用掩码感知的输入设计，不补全也不丢弃，把可用性状态提升为与特征并列的输入信号，使缺失结构在整条数据流中保持可见，也为 5.4 节对缺失规律的分析保留了模型内部的对应依据。

具体而言，局部缺失被置零后，数值零可能同时表示该时间步无信息与该特征本身取零，只输入特征矩阵无法区分这两种情况。为此，我们把缺失指示与可见指示作为附加通道，与原特征共同输入编码器，使每个时间步的输入显式携带自身的可用状态：

$$\mathbf{u}_t^m = [\hat{\mathbf{x}}_t^m \parallel M_t^m \parallel 1 - M_t^m] \in \mathbb{R}^{d_m+2}, \quad \mathbf{h}_m^{(0)}(t) = \text{GELU}(W_{\text{in}} \mathbf{u}_t^m + b_{\text{in}}). \quad (2)$$

这样两类零在输入层即可区分，填充位由有效长度指示，缺失位由掩码指示，模型看到的是标注了可用性状态的残缺序列，而不是被零填满的完整序列。图 5 用一条示意样本概括该机制，灰段为尾部填充、彩色段为有效区间、斜纹段为缺失片段；填充位不进入池化统计，缺失位则始终以掩码标注的形式参与编码与融合，网络处理的是有效位加权表示与可用性指示的组合。

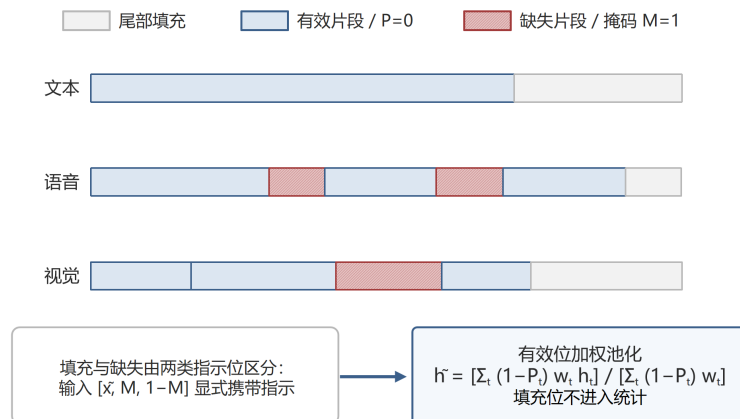


图 5 掩码感知输入与有效位加权池化示意

输入表示确定后，进入序列编码环节。情感线索在时间上呈现两种尺度，语调起伏、表情突变等短时变化决定局部形态，而整段语义的走向需要长程上下文才能判断；单一

结构难以同时覆盖两端，因为纯卷积的感受野受限，而纯注意力对短时突变不够敏感。为此，编码器按局部变化与长程依赖两级结构对序列建模，先投影到统一隐空间，再通过一层轻量的一维卷积提取局部变化（窗口 3、残差连接），并叠加位置编码后送入  $L_e$  层 Transformer 编码器：

$$\check{\mathbf{h}} = \mathbf{h}^{(0)} + \text{GELU}(\text{Conv1d}_3(\mathbf{h}^{(0)})), \quad \mathbf{h}_m = \text{Enc}(\check{\mathbf{h}} + \mathbf{p}), \quad (3)$$

其中位置编码  $\mathbf{p}$  使模型能够区分时间先后；卷积层负责短时变化模式，自注意力层整合长程依赖与跨段信息。编码器以填充掩码作为注意力屏蔽项，尾部填充不参与任何信息聚合；三种模态使用相同的编码结构但参数彼此独立，最终统一映射到维度  $d$  的表示空间，便于后续的共享与融合计算。

编码完成后进入表示分解环节。三模态表示若直接拼接送入融合层，模型需要在有限样本下自行摸索哪些跨模态信息应当对齐、哪些应当保持独立，学习负担较重；更关键的是，某模态缺失时其表示已不可靠，仅靠拼接很难让其余模态及时补位。为此，我们显式地把表示解耦为共享分量与模态特有分量，编码输出经两个投影头解耦，再拼接映射回统一表示，并通过相似性约束和差异性约束限制两类表示之间的关系：

$$\mathbf{s}_m = W_s \mathbf{h}'_m, \quad \mathbf{d}_m = W_d \mathbf{h}'_m, \quad \mathbf{e}_m = W_{\text{mix}}[\mathbf{s}_m \parallel \mathbf{d}_m], \quad (4)$$

其中  $\mathbf{h}'_m$  为经代理令牌修正后的编码表示（见 5.3.2 节）；共享分量承载跨模态共有的情感线索，特有分量保留各模态独有信息。为使两类分量可辨识，训练中加入两项正则，其中相似性项把不同模态的共享分量拉近，差异性项抑制同一模态内共享与特有分量的信息重叠：

$$\mathcal{L}_{\text{sim}} = \frac{1}{3} \sum_{m < m'} \|\bar{\mathbf{s}}_m - \bar{\mathbf{s}}_{m'}\|_2^2, \quad \mathcal{L}_{\text{diff}} = \frac{1}{3} \sum_m \overline{(\cos(\mathbf{s}_m, \mathbf{d}_m))^2}, \quad (5)$$

式中  $\bar{\mathbf{s}}_m$  为共享分量在有效位上的均值池化，第二式的横线表示在有效时间步上取平均。至此，掩码信号已从输入构造贯穿至表示分解，构成模型缺失感知能力的第一处关键机制；分解得到的表示随即进入下一节的专家门控融合。

### 5.3.2 动态专家门控融合

动态专家门控融合是问题二模型的第二个核心模块。其基本原理是让信息路由随样本的缺失结构自适应变化，缺失越严重的模态越依赖可学习的代理表示兜底，缺失组合不同的样本则被路由到不同的专家权重上，从而在部分模态不可用时仍能稳定地合成样本级表示。该模块由代理令牌修正、门控网络与专家合成、掩码感知池化三个环节构成，分别应对单模态退化、模态组合差异与时间步可信度三个层面的问题；其整体数据

流如图 6 所示。下面先说明融合方式的选择依据，再依次展开三个环节。

更直接的做法是固定融合函数，把三模态表示拼接或加权求和后送入统一网络。但它要求同一组参数同时服务差异极大的输入分布，三模态齐全与文本全缺的样本在最优融合策略上几乎没有共性，参数只能在各种模式之间折衷，对每一种模式都难以达到最优。我们因此把路由与融合分开设计，保留一组结构相同、参数互异的专家，由门控网络依据样本的缺失结构分配专家权重，缺失组合不同的样本被分发到不同的子网络，模型无需为每种模式单独训练，却获得了针对性融合的能力；这一条件计算设计还使容量与开销解耦，模型总容量随专家数增长，单个样本的激活量却基本不变。

先处理模态级的退化问题。某一模态缺失严重时，其编码结果本质上是零填充经网络变换的产物，分布与真实观测差异明显，直接送入融合相当于让噪声参与全局表示，缺失越重、污染越大；反过来，硬性剔除该模态又浪费了它尚未缺失的部分。我们的折中方案是为每个模态设置可学习的代理令牌  $\mathbf{p}_m$ （可理解为该模态在训练集上的平均情感表达），并根据缺失率与当前编码状态计算门控系数  $\beta_m$ ，在代理令牌与原表示之间插值：

$$\beta_m = \sigma(w_\beta^\top [\bar{\mathbf{h}}_m; \rho_m] + b_\beta), \quad \mathbf{h}'_m = (1 - \beta_m) \mathbf{h}_m + \beta_m \mathbf{p}_m, \quad (6)$$

其中  $\bar{\mathbf{h}}_m$  为编码表示的有效位均值， $\rho_m$  为该模态缺失率。缺失越严重的模态， $\beta_m$  越大、越依赖代理表示兜底。

代理修正之后进入融合阶段的样本级路由。门控网络为每个样本动态分配专家权重，其输入由融合表示与缺失结构统计量共同构成。缺失结构统计量共 7 维，包括三模态缺失率、三模态可用指示（缺失率低于 1 时置 1）与可用模态数比例：

$$\mathbf{u} = [\rho_t, \rho_a, \rho_v, \mathbb{I}(\rho_t < 1), \mathbb{I}(\rho_a < 1), \mathbb{I}(\rho_v < 1), n_{\text{vis}}/3], \quad (7)$$

将三模态表示拼接为令牌序列并加入模态嵌入后，门控权重与专家融合输出为

$$\mathbf{g} = \text{softmax}\left(W_2 \text{GELU}(W_1[\text{tok}; \mathbf{u}])\right), \quad \mathbf{z}_f(t) = \sum_{e=1}^{n_E} g_e \cdot \text{Expert}_e(\text{tok})(t), \quad (8)$$

式中  $\text{tok}$  为令牌序列的有效位均值表示。各专家为结构相同的跨模态、跨时间自注意力子网络（含多头注意力、前馈与层归一化），门控权重随样本缺失结构自适应变化。为防专家坍塌（权重集中到少数专家），加入熵形式的负载均衡正则：

$$\mathcal{L}_{\text{bal}} = -n_E \sum_{e=1}^{n_E} \bar{g}_e \ln(\bar{g}_e + \epsilon), \quad (9)$$

其中  $\bar{g}_e$  为批内专家  $e$  的平均门控权重；当各专家被均匀使用时该式取最小值。

专家输出再经掩码感知的注意力池化压缩为样本级表示，以可学习打分函数为每个时间步计算重要性，并在 softmax 前对填充位作屏蔽，使其退出有效统计：

$$\alpha_t = \frac{\exp(s(z_f(t)))}{\sum_{t'} \exp(s(z_f(t')))}, \quad z = \sum_{t=1}^T \alpha_t z_f(t), \quad (10)$$

式中求和仅覆盖非填充时间步， $s(\cdot)$  为打分函数。图 6 展示该模块的数据流，三个模态经掩码感知编码与共享-特有分解后拼接为令牌序列，门控网络读取缺失结构统计量与融合表示，为全部专家生成样本级权重；专家输出按权重合成后再经注意力池化得到样本级表示  $z$ 。缺失模式一变、路由权重随之改变，这是模型缺失感知能力的第二处关键机制。

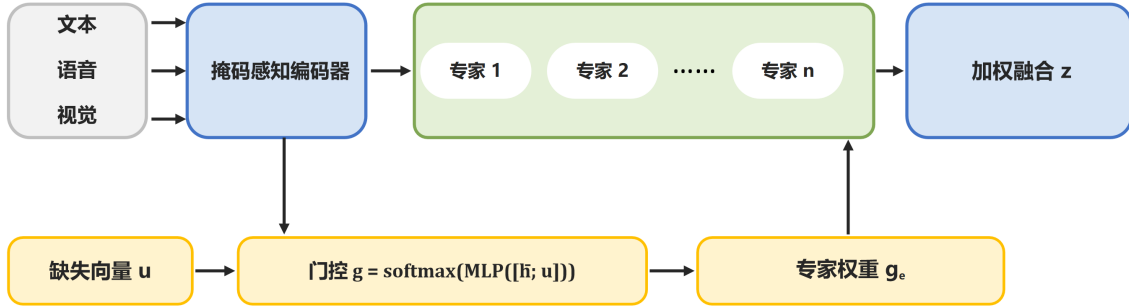


图 6 动态专家门控融合示意

### 5.3.3 多任务输出与目标函数

编码与融合得到样本级表示  $z$  后，模型还需将其映射为具体任务输出并定义优化目标。模型同时优化情感强度与情感极性，强度回归头输出预测均值与对数方差，主损失采用加权平滑 L1（Huber 型， $\beta_s = 0.15$ ），并叠加小权重的高斯负对数似然项，以便在预测强度的同时估计逐样本不确定度：

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{Huber}}(\mu, y) + \lambda_{\text{var}} \cdot \frac{1}{N} \sum_{i=1}^N \left[ \frac{(y_i - \mu_i)^2}{2\sigma_i^2} + \frac{1}{2} \ln \sigma_i^2 \right], \quad \sigma_i^2 = \text{softplus}(s_i) + \sigma_0^2, \quad (11)$$

其中  $\lambda_{\text{var}} = 0.1$ ；平滑 L1 对强情感样本的极端误差更稳健，负对数似然项则保证逐样本方差可用（供 5.4.5 节的不确定性分析使用）。极性分类头采用类平衡交叉熵应对类别不平衡（训练集负/中/正约为 967/758/1670），并叠加标签平滑、弱标注降权与中性类抑制：

$$\mathcal{L}_{\text{cls}} = -\frac{1}{N} \sum_{i=1}^N \omega_{c_i} \sum_{k=0}^2 \mathbb{I}[c_i = k] \ln \pi_{i,k}, \quad \omega_k \propto \frac{1}{n_k}, \quad (12)$$

式中  $\omega_k$  截断于区间  $[0.2, 10]$  以抑制极端类权重；弱标注 ( $|y| \leq 0.5$ ) 样本的分类损失按 0.5 降权，中性类抑制项权重 0.25，另设强度软目标 EMD 项（权重 0.3）为分类头补充幅值侧信息。有序回归辅助头进一步把强度绝对值离散为  $K = 10$  个有序档位，以累积链接模型（CORAL）[4] 施加序数监督，强制模型在零点附近建立分辨率：

$$\mathcal{L}_{\text{CORAL}} = -\frac{1}{N(K-1)} \sum_{i=1}^N \sum_{j=0}^{K-2} \left[ \tilde{k}_i > j \right] \ln \varsigma_{i,j} + \left[ \tilde{k}_i \leq j \right] \ln(1 - \varsigma_{i,j}), \quad (13)$$

其中  $\varsigma_{i,j} = \text{sigmoid}(w^\top z_i + b_j)$  为累积概率，阈值  $\{b_j\}$  可学习、按线性初值排布。总目标函数由上述各项与分解正则、均衡正则共同组成：

$$\mathcal{L} = \mathcal{L}_{\text{reg}} + \lambda_1 \mathcal{L}_{\text{cls}} + \lambda_3 \mathcal{L}_{\text{sim}} + \lambda_4 \mathcal{L}_{\text{diff}} + \lambda_5 \mathcal{L}_{\text{bal}} + \lambda_6 \mathcal{L}_{\text{CORAL}} + \lambda_7 \mathcal{L}_{\text{KD}} + \lambda_2 \mathcal{L}_{\text{rec}}, \quad (14)$$

其中权重分别取  $\lambda_1 = 1.0$ 、 $\lambda_3 = \lambda_4 = 0.5$ 、 $\lambda_5 = 0.01$ 、 $\lambda_6 = 0.5$ 、 $\lambda_7$  随轮数衰减（见 5.3.4 节）。跨模态重建项在最终部署配方中被关闭（ $\lambda_2 = 0$ ）。模型搜索起点包含以缺失位置为目标的重建约束，但消融实验显示关闭重建后误差与宏观 F1 同时改善（见 5.4.4 节），因此最终配方不含该项。

#### 5.3.4 缺失鲁棒训练策略

目标函数之外，训练过程本身也需要针对缺失场景专门设计。训练期的缺失注入按课程学习方式 [6] 由易到难推进，共分三段，第 1 至 8 轮仅注入单模态缺失、强度不超过 0.10；第 9 至 20 轮引入多模态混合缺失、强度区间为  $[0.04, 0.20]$ ；第 21 轮起把强度上限抬高至 0.25：

$$\rho \in \begin{cases} [0, 0.10], & 1 \leq e \leq 8 \quad (\text{单模态}), \\ [0.04, 0.20], & 9 \leq e \leq 20 \quad (\text{混合模态}), \\ [0.04, 0.25], & e \geq 21 \quad (\text{混合模态}). \end{cases} \quad (15)$$

由浅入深的安排让模型先建立完整信息下的基本映射，再逐步适应更复杂的缺失结构，避免训练早期因缺失过强而优化不稳。注入缺失的位置先验为中部 4/9、头部 3/9、尾部与随机各 1/9；每个模态允许 1 至 3 段碎片化缺失。

训练进一步采用教师-学生蒸馏，教师模型在固定中等缺失强度下训练，学生模型接受课程式缺失注入，并通过输出层软标签、强度回归与样本间关系三类蒸馏信号 [5]，

从教师模型获得相对完整的监督信号：

$$\mathcal{L}_{\text{KD}} = \text{KL}(\text{softmax}(\mathbf{o}^T/\tau) \parallel \text{softmax}(\mathbf{o}^S/\tau)) \cdot \tau^2 + \|\mu^T - \mu^S\|^2 + 0.1 \|\mathbf{R}^T - \mathbf{R}^S\|^2, \quad (16)$$

式中  $\tau = 2.0$  为蒸馏温度， $\mathbf{R}$  为批内样本表示的相似度矩阵（关系蒸馏项降权至 0.1，避免量级压过任务损失）。蒸馏权重随训练轮数指数衰减，后期逐渐让位于任务损失本身：

$$\lambda_{\text{KD}}(e) = 0.3 \exp(-e/15). \quad (17)$$

图 7 概括完整流程，左图为缺失注入的课程安排（教师阶段固定中等强度，学生阶段按三段区间递进），右图为蒸馏权重的指数衰减曲线；下方三框对应阶段一、阶段二与最终交付。两步合起来实现先学稳、再学缺的效果，教师端给出干净的监督，学生逐步逼近部署时的缺失场景，交付时只保留学生模型。

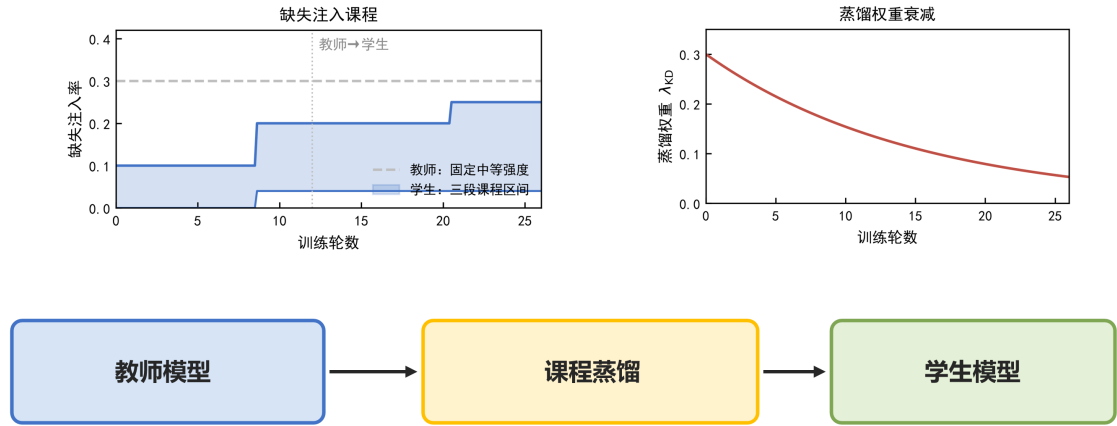


图 7 两阶段课程式缺失训练示意

其余稳定性措施包括参数指数滑动平均（衰减 0.99，用于平滑参数轨迹并作为交付权重）、时间裁剪（至少保留 70% 长度）、特征高斯噪声（ $\sigma = 0.15$ ）与整模态丢弃（概率 0.15）等数据增强，以及配合余弦退火学习率的 AdamW 优化。综合上述设计，两阶段训练的完整流程包含四个步骤。（1）按固定中等强度训练教师模型 12 轮并冻结其参数；（2）以三段课程注入训练学生模型 14 轮，同期叠加蒸馏与多任务损失（早停监控验证集误差，耐心 8 轮）；（3）对种子 42 与 43 的学生模型作集成，并在验证集上确定决策阈值；（4）固定全部参数，对附件三与附件四执行推理并生成提交文件。图 8 给出训练过程的损失与验证指标曲线，教师与学生阶段衔接平稳，验证误差在第 10 轮左右进入平台区，没有明显过拟合。

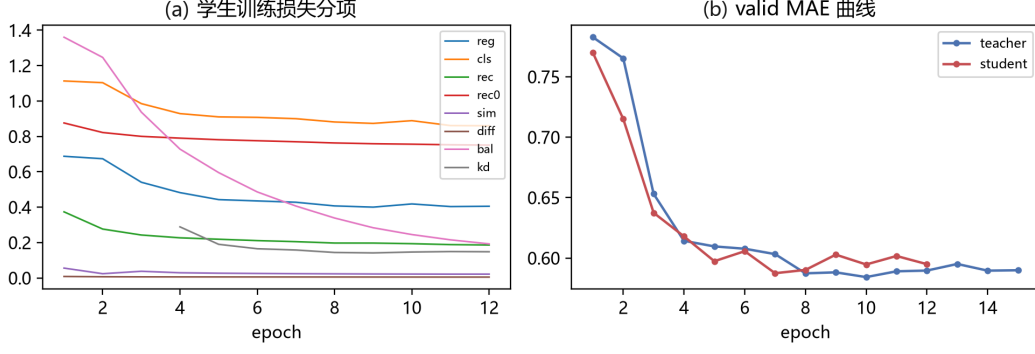


图 8 训练损失与验证指标随轮数的变化

### 5.3.5 决策阈值选择与校准

模型结构确定后，推理阶段还需明确极性的决策规则。情感极性的主输出来自分类头  $\text{argmax}$ ，同时保留按连续强度与对称阈值划分极性的对照口径。题目规定决策阈值必须在验证集上选择；我们采用单一对称阈值，当预测强度  $\hat{y} < -\theta$  时判为负类， $|\hat{y}| \leq \theta$  时判为中性， $\hat{y} > \theta$  时判为正类。阈值只在验证集上搜索，以宏观 F1 最大为目标：

$$\theta^* = \arg \max_{\theta \in \Theta} \text{MacroF1}(\{\text{polar}(\hat{y}_i, \theta)\}, \{c_i\}), \quad \Theta = \{0, 0.02, \dots, 1.50\}. \quad (18)$$

据此选定阈值  $\theta = 0.325$ （仅由验证集拟合）。图 9 的敏感性曲线表明，该值位于准确率和宏观 F1 的稳定平台附近（两者在 0.28 附近同步达到峰值），小范围阈值扰动不会明显改变结果。

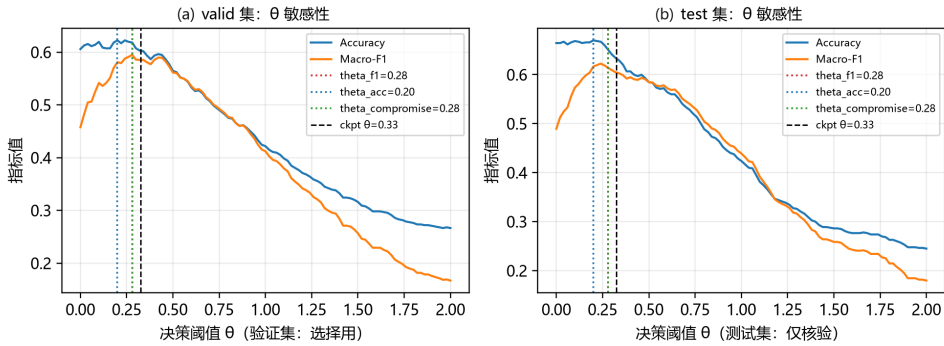


图 9 决策阈值敏感性曲线

考虑到回归输出存在向零收缩，我们同时给出仿射校准作为对照 [8]，其系数同样只在验证集上估计：

$$(\alpha^*, \beta^*) = \arg \min_{\alpha, \beta} \frac{1}{|\mathcal{V}|} \sum_{i \in \mathcal{V}} |\alpha \mu_i + \beta - y_i|. \quad (19)$$



按题目规定，若校准后验证集分类指标不占优，则主口径不采用校准、仅列为敏感性分析。图 10 给出校准前后的强度散点对比，校准把收缩的预测两端幅度拉开，平均绝对误差略有下降，但分类指标未获得稳定收益，因此最终提交仍使用未校准的强度值，避免额外后处理改变主结果，校准结果仅用于说明收缩的来源。

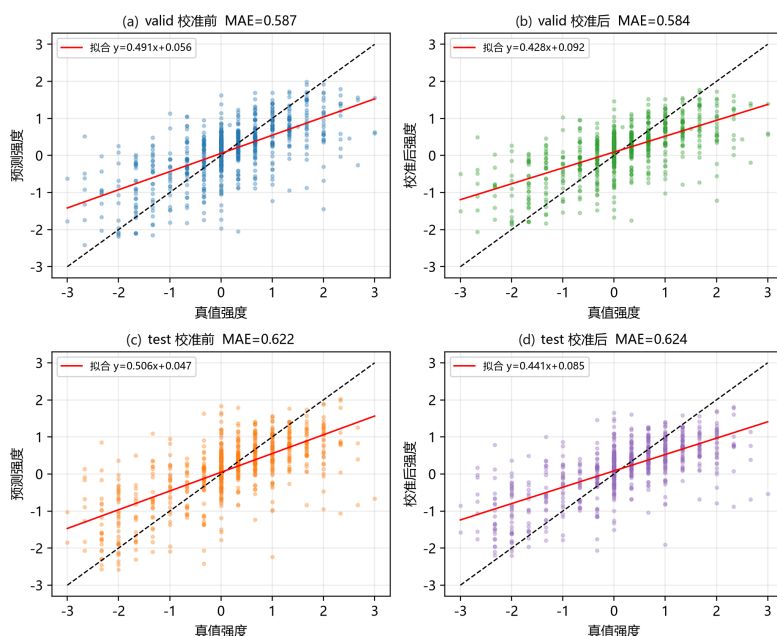


图 10 强度校准前后对比

主模型的关键参数如表 5 所示，全部取值可对照配置文件与检查点元数据逐项复核。

表 5 问题二模型的数据、网络与训练参数

| 参数         | 取值                                           | 参数        | 取值                        |
|------------|----------------------------------------------|-----------|---------------------------|
| 特征版本与序列长度  | aligned, $T = 50$                            | 隐维度       | $d = 96$                  |
| 缺失判定       | 有效区间内连续 $\geq 2$ 帧全零                         | 门控专家数     | $n_E = 2$                 |
| 注意力        | 头数 4, 编码层数 2                                 | 轮数        | 教师 12, 学生 14              |
| 丢弃率与权重衰减   | 0.35, $5 \times 10^{-4}$                     | 批大小与优化器   | 64, AdamW 配余弦退火           |
| 学习率        | 主干 $5 \times 10^{-5}$ , 头 $5 \times 10^{-4}$ | 梯度裁剪与早停   | 1.0, 耐心 8 轮               |
| 输入噪声与整模态丢弃 | 0.15, 0.15                                   | 标签平滑与权重平均 | 0.1, 指数移动平均 0.99          |
| 缺失片段数与裁剪保留 | 每模态 1 至 3 段, 保留 0.7                          | 位置先验      | 中 4/9, 头 3/9, 尾与随机各 1/9   |
| 辅助头        | 有序回归 $K = 10$ , 权重 0.5                       | 决策阈值      | $\theta = 0.325$ (仅验证集拟合) |

本章的设计与题目给定文献存在多处对应，也存在明确差异。动态专家门控与 EMOE 提出的模态专属动态情感专家 [17] 目标一致，不同的是我们的门控在输入中显式包含三模态缺失率与可用性统计；教师-学生蒸馏对应 CMAD 的模态感知蒸馏 [12]，我们额外把缺失类型与缺失率作为课程维度，并加入样本间关系蒸馏。与分解、重建、增强框架 [11] 相比，我们保留了共享-特有分解、但停用了重建项，关闭重建后验证集

误差由 0.5871 降至 0.5825，该负结果已在消融一节一并报告。处理缺失的另一类思路是生成式补全与代理任务，分别见于超图条件扩散 [10] 与 Proxy-Driven 方法 [13]；我们不做跨模态生成，而是以掩码参与注意力、缺失注入与对数方差头来达到同类目的。其余文献支撑本文的选题依据与稳健性动机 [9, 14, 16, 18]，多任务输出头的组织借鉴了多任务学习的设计 [3]。

## 5.4 结果分析与可视化

### 5.4.1 评价指标与评估口径

按赛题要求，情感极性采用准确率与 F1 值评价，情感强度采用平均绝对误差与皮尔逊相关系数评价，并补充 CCC 作为参考。记样本真值为  $y_i$ 、预测值为  $\hat{y}_i$ ，类别真值与预测类别为  $c_i$  与  $\hat{c}_i$ ，四项指标定义为：

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|, \quad r = \frac{\text{cov}(\hat{y}, y)}{\sigma_{\hat{y}} \sigma_y}, \quad (20)$$

$$\text{ACC} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{c}_i = c_i], \quad F1_{\text{macro}} = \frac{1}{3} \sum_{k=1}^3 F1_k, \quad (21)$$

其中宏观 F1（Macro-F1）对三类样本等权计算（三类 F1 的算术平均），能够反映类别不平衡下中性类的识别问题，因此选作分类主指标；各类别 F1 与一致性指数 CCC 一并报告作为参考。各项指标的评估口径汇总于表 6。

表 6 评价指标与评估口径

| 项目     | 规定                                     |
|--------|----------------------------------------|
| 主报集合   | 验证集（728 条）；测试划分（727 条）仅作泛化核验           |
| 极性主口径  | 分类头 $\text{argmax}$ （与分类目标的直接口径）       |
| 极性对照口径 | 强度分数加单一对称阈值 $\theta$ （用于附件三一致性核验）      |
| 强度主指标  | MAE 为主，皮尔逊相关系数为辅，CCC 参考                |
| 决策阈值   | 仅由验证集按宏观 F1 选定（ $\theta = 0.325$ ）     |
| 文献可比口径 | 二分类 Acc-2（分类头概率构造，负向对非负向）：0.8514（测试划分） |
| 合规性    | 结构、超参数、阈值、后处理系数均只在验证集确定；附件三/四仅推理       |

需要先厘清两个易混淆的口径。我们把验证集作为主报集合，附件二测试划分只用于泛化核验；极性主口径采用分类头  $\text{argmax}$ ，阈值口径仅用于一致性比较；模型结构、超参数、阈值与后处理系数均只依据验证集确定。附件二的测试划分含标签、可用于泛化核验，无标签的附件三测试集仅用于最终预测提交，二者性质不同，本章在表述上严格区分。另外，为便于与公开文献参照，本章额外计算了二分类 Acc-2 指标；由于公开设定的训练规模（约 16326 条）约为本章（3395 条）的 4.8 倍，该指标仅作量级参照，

不构成严格优劣比较。

5.4.2 基础性能评价

表 7 汇总提交模型在验证集与测试划分上的完整指标。验证集上（主报口径），MAE 为 0.5866、皮尔逊相关系数 0.6543、CCC 0.6278、准确率 0.6223、宏观 F1 0.6158；测试划分上 MAE 为 0.6216、ACC 为 0.6534、宏观 F1 为 0.6338。测试划分的分类指标与验证集相当或更高，MAE 略有增加，整体未出现明显失稳，说明模型未表现出明显过拟合。

表 7 问题二模型的基础性能

| 划分       | MAE    | Pearson | CCC    | ACC    | Macro-F1 | 中性类 F1 |
|----------|--------|---------|--------|--------|----------|--------|
| 验证集（主报）  | 0.5866 | 0.6543  | 0.6278 | 0.6223 | 0.6158   | 0.5056 |
| 测试划分（核验） | 0.6216 | 0.6807  | 0.6515 | 0.6534 | 0.6338   | 0.4656 |

图 11 的强度散点图（横轴为真值、纵轴为预测，颜色区分真值极性）显示，预测总体方向正确，但高强度的正负样本都存在向零收缩。分组偏差（预测减真值）分别为负类 +0.541、正类 -0.447、中性类 +0.110，说明强情感样本的幅值被系统性压缩，这是逐点回归损失下条件均值解的固有现象，也是本章强度误差的首要来源。

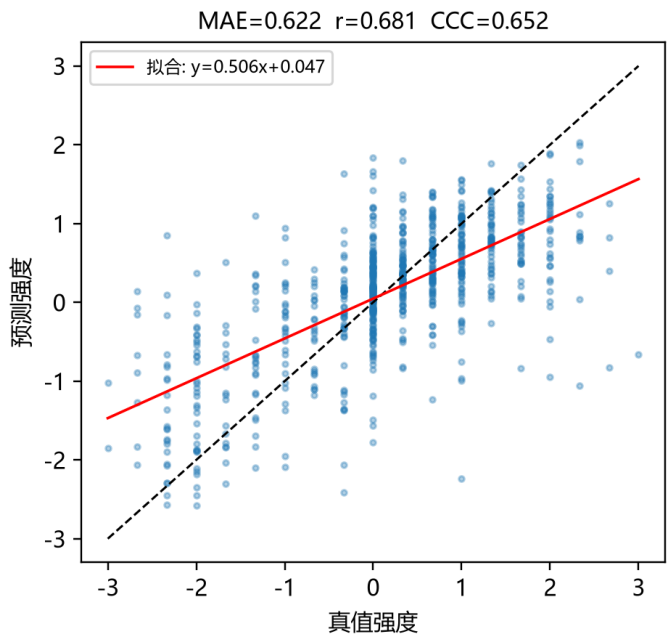


图 11 强度预测值与真值散点图（验证集）

图 12 的三分类混淆矩阵（行、列分别为真值与预测类别）进一步显示，中性类是当前分类性能的主要短板：184 条真中性样本中有 72 条被判为极性样本，同时有 147 条

极性样本被判为中性，双向混淆明显，中性类精确率仅为 0.4324；相比之下，负类和正类的区分更稳定。非对角元还给出了错判的方向与规模，中性边界由此可确认是当前模型的主要误差来源。

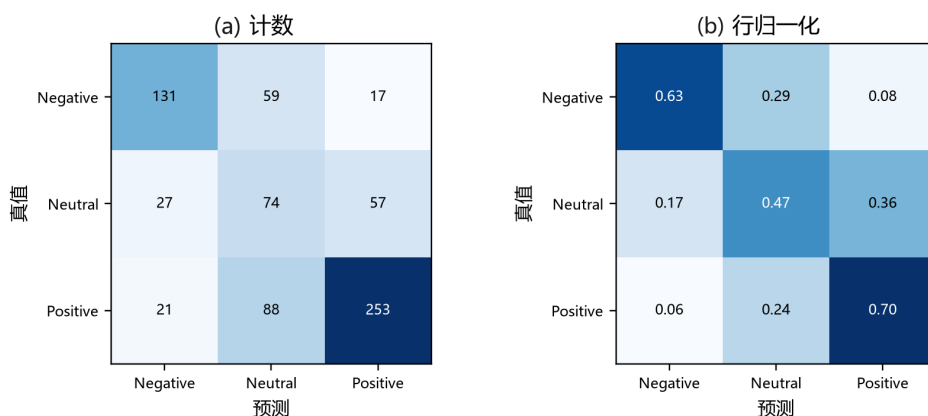


图 12 三分类混淆矩阵（验证集；左为计数、右为行归一化）

图 13 的绝对误差直方图显示误差呈长尾分布，多数样本的误差小于 0.5，少数强情感样本超过 2.0，而平均误差的量级正是由这些长尾样本主导；分位统计与此一致，绝对误差的 90 分位约为中位数的三倍。

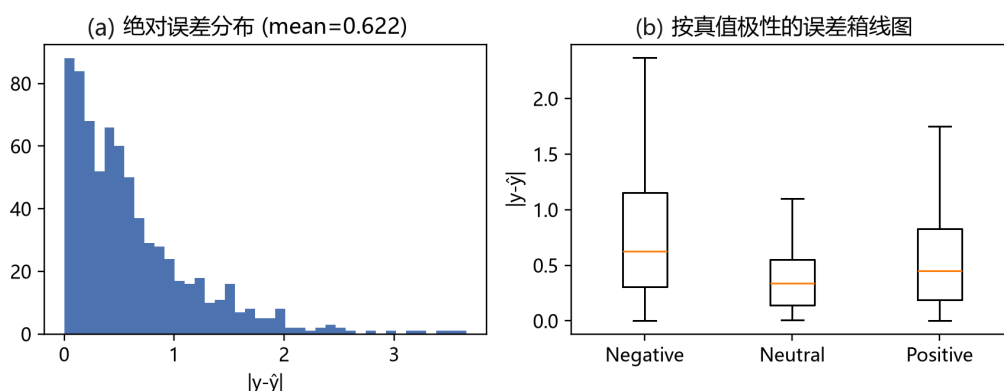


图 13 强度绝对误差的分布（验证集；左为总体分布、右为按真值极性的分层）

为衡量所提模型的水平，表 8 给出与公开结果的量级对照。需要强调两点前提。文献几乎一律主报二分类 Acc-2（负向对非负向），与题目要求的三分类 ACC 不可直接并列；且公开 CMU-MOSEI[1] 划分的训练样本为 16326 条，约为赛题的 4.8 倍。按统一复现口径，公开基线的三分类准确率为 0.661 至 0.676、平均绝对误差为 0.526 至 0.601、皮尔逊相关为 0.683 至 0.778；本文对应为 0.6223、0.5866 与 0.6543。二分类口径 0.8514 落在公开干净设定区间（约 0.84 至 0.86）之内，明显高于随机缺失协议下的公开水平（约 0.71 至 0.79）。在约五分之一训练规模下达到公开量级，说明特征组织与融合设计是有效的；由于训练规模与评价口径的差异，上述对照仅作量级参照，不作严格优劣比较。

表 8 与公开结果的量级对照（口径已对齐说明）

| 方法               | 训练规模  | 模态设定 | 缺失设定  | Acc-2         |
|------------------|-------|------|-------|---------------|
| 公开基线（文献常报区间）     | 16326 | 三模态  | 全模态干净 | 0.84 至 0.86   |
| 公开基线（随机缺失协议）     | 16326 | 三模态  | 随机缺失  | 0.71 至 0.79   |
| 本文 MRF-Net（测试划分） | 3395  | 三模态  | 全模态干净 | <b>0.8514</b> |

注：本表仅作量级参照；公开区间按统一复现口径从相关文献汇总，其训练规模与评价协议与本题并不一致，具体来源见参考文献对应条目。

### 5.4.3 缺失模态类型、位置与时长的影响规律

为系统分析局部缺失对性能的影响，本章把缺失类型、缺失位置与缺失率的组合枚举成一张场景网格，缺失类型 6 种（仅文本、仅语音、仅视觉、文本 + 语音、语音 + 视觉、三模态全缺）、缺失位置 4 种（头部、中部、尾部、随机）、缺失率 5 档（0.1 至 1.0），合计 120 个场景，设计如表 9 所示。缺失率按式 (1) 定义，即缺失帧数占该模态有效长度的比例；不直接以绝对帧数分档，是因为三种模态的有效长度差异很大（文本恒为 50 帧，语音与视觉约 24.6 与 23.8 帧），同一绝对档位在不同模态与样本上对应的缺失比例可相差数倍。扫描在验证集上进行，每个场景对全部 728 条样本注入相同的缺失结构并计算指标；由于头部、中部、尾部三种位置的掩码是确定性的，扫描采用单次确定性评估，统计推断以三因素方差分析中的最高阶交互作为误差项，证据以效应量（ $\eta^2$ ）与实际差值为主、显著性检验为辅。

表 9 缺失规律扫描的实验设计

| 因素   | 水平                                               |
|------|--------------------------------------------------|
| 缺失类型 | 仅文本 / 仅语音 / 仅视觉 / 文本 + 语音 / 语音 + 视觉 / 三模态全缺（6 类） |
| 缺失位置 | 头部 / 中部 / 尾部 / 随机（4 类）                           |
| 缺失率  | 0.1 / 0.2 / 0.3 / 0.5（另含整模态 1.0 的极限情形）           |
| 组合规模 | $6 \times 4 \times 5 = 120$ 种场景                  |

性能退化量定义为相对无缺失基线的差值：

$$\Delta \text{MAE}(\tau, p, \rho) = \text{MAE}(\tau, p, \rho) - \text{MAE}_{\text{clean}}, \quad \Delta \text{ACC} = \text{ACC}(\tau, p, \rho) - \text{ACC}_{\text{clean}}. \quad (22)$$

图 14 的左、右面板分别给出六类缺失下验证集 MAE 与宏观 F1 随缺失率的退化曲线（虚线为无缺失基线）；结果表明，性能退化主要由文本缺失驱动。在 MAE 侧，文本

缺失在缺失率 0.1 至 0.2 时几乎无影响，从 0.3 起明显抬升，到整模态缺失 ( $\rho = 1.0$ ) 时增量达到 +0.1804；语音缺失与视觉缺失即使整模态完全不可用，增量也仅为 +0.0002 与 +0.0068，含文本的混合缺失（文本 + 语音）与文本单缺几乎重合（+0.1810）。一个例外是三模态全缺，其增量升至 +0.1950，比仅文本缺失再高出约 8%，说明保留可见的语音与视觉仍能贡献少量互补信息；该极端情形在训练注入中并不会出现（训练始终保留至少一个可见模态），模型对它承担的属于分布外的下界损失。整模态缺失时，文本造成的误差增量约为视听模态的 25 倍，情感判别由文本主导，鲁棒性设计应以文本分支为重点。

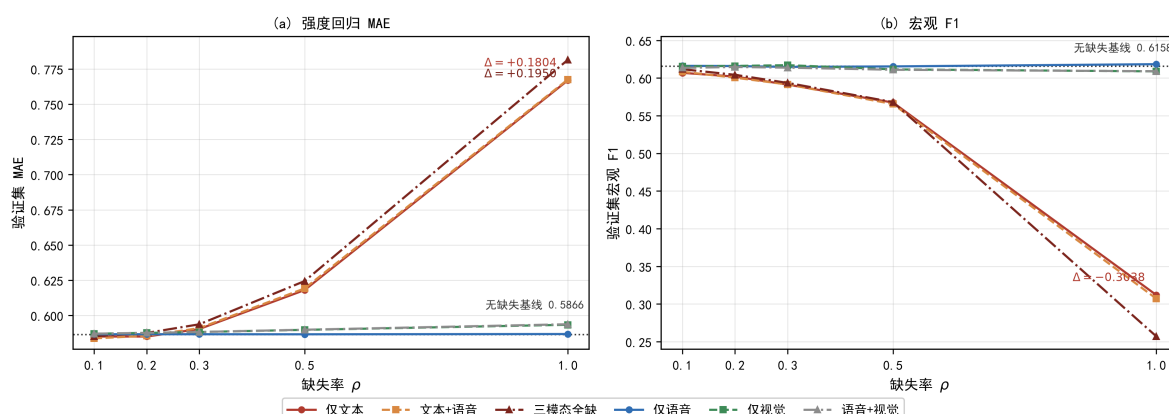


图 14 六类缺失下的退化曲线（左：验证集 MAE；右：宏观 F1；虚线为无缺失基线）

图 14(b) 从分类指标侧给出同一规律的镜像证据，文本缺失使宏观 F1 持续下降且降幅随缺失率放大，整模态缺失时降幅达 0.3038（三模态全缺为 0.3587）；语音与视觉缺失的 F1 曲线仅在噪声范围内波动（降幅不超过 0.007）。两个指标趋势一致，说明文本主导的规律不是单一指标的偶然现象。

图 15 把六类缺失的退化量按缺失率汇总成综合视图（散点为各类退化量、折线为六类均值、阴影为极差范围），随着缺失率上升，退化谱整体右移，文本类始终占据分布的上缘；在缺失率 0.5 处，文本类与视听类的差距已拉开到约 10 倍（+0.031 以上对 +0.003 以下）。综合视图说明退化的主导因素在全部扫描范围内保持稳定。

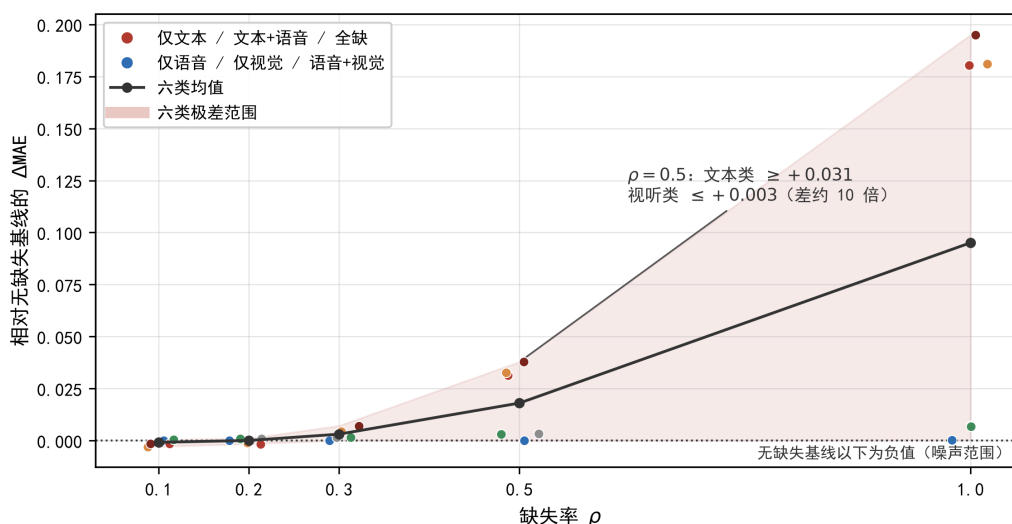


图 15 六类缺失的综合退化视图（散点为各类退化量、折线为六类均值、阴影为极差）

缺失位置同样存在稳定差异。图 16 给出类型与位置的平均退化热力图（对五档缺失率取平均，含整模态档），图上两个结构一目了然，文本相关类型的退化比视听类型高一个量级（约  $+0.04$  至  $+0.05$  对  $+0.003$  以下），且同一类型内头部缺失最重、中部最轻（仅文本一行头部较中部多约  $+0.007$ ）。若剔除整模态档、只在部分缺失范围内统计（表 10），位置次序保持不变，头部缺失的平均退化最大（ $+0.0078$ ），其后依次为随机（ $+0.0052$ ）、尾部（ $+0.0039$ ）与中部（ $+0.0035$ ）；在文本模态上，头部与中部之差达到 2.7 倍（ $+0.0141$  对  $+0.0052$ ）。可能的解释是，文本特征的开头往往承载话题与情感倾向的定调信息，缺失头部意味着丢失上下文起点，模型需要更强的推理能力才能补全；而中部或尾部缺失时，已见片段通常足以恢复大部分语义。两个朴素基线在四个位置上的退化几乎持平（ $0.0102$  至  $0.0137$ ），未表现出明显的位置结构；MRF-Net 的残余退化则呈现头部最重的稳定次序，说明掩码感知设计利用了缺失位置信息。图 16(b) 的宏观 F1 热力图与 MAE 维度的结构同构，位置结论在两个指标上一致。

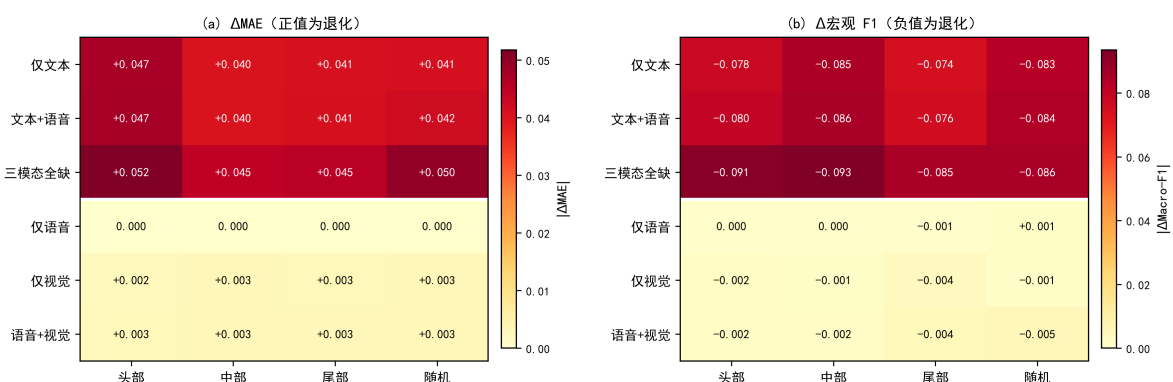


图 16 缺失类型  $\times$  缺失位置的平均退化热力图（左： $\Delta\text{MAE}$ ；右： $\Delta\text{宏观 F1}$ ；对五档缺失率取平均，含整模态档）

**表 10 各模型在四个缺失位置上的平均 MAE 增量**

| 缺失位置 | A0 朴素融合 | A1 加掩码指示 | MRF-Net 单种子 | MRF-Net 集成 |
|------|---------|----------|-------------|------------|
| 头部   | 0.0120  | 0.0133   | 0.0063      | 0.0078     |
| 中部   | 0.0102  | 0.0118   | 0.0022      | 0.0035     |
| 尾部   | 0.0128  | 0.0132   | 0.0031      | 0.0039     |
| 随机   | 0.0121  | 0.0137   | 0.0038      | 0.0052     |

三因素方差分析进一步量化了上述规律。表 11 以退化量为响应做方差分析，报告效应量  $\eta^2$ （误差项取最高阶交互）：

$$\Delta\text{MAE} \sim C(\tau) * C(p) * C(\rho), \quad (23)$$

其中  $C(\cdot)$  表示按类别变量处理。基线模型中缺失类型与缺失率交互的效应量高达 0.9517，意味着退化几乎完全取决于缺失的模式与比例；在 MRF-Net 集成模型上该效应量降至 0.8415，说明缺失感知设计削弱了退化对缺失结构的依赖，退化被整体摊平。与之相对，缺失类型与缺失位置交互的效应量从基线的 0.0053 提升至 0.0614（约 12 倍），即 MRF-Net 对缺失位置变得敏感，这是掩码与位置信息被模型利用的间接证据。按位置统计，MRF-Net 将退化量相对朴素基线降低约 1.5 至 3.3 倍（头部 1.5 倍、中部 2.9 倍、尾部 3.3 倍、随机 2.3 倍），其中头部场景收益最小，与头部缺失本身伤害最大相符。

**表 11 三因素方差分析的效应量（ $\eta^2$ ，响应为 MAE 增量）**

| 效应项                | A0 朴素融合 | A1 加掩码指示 | MRF-Net 单种子 | MRF-Net 集成 |
|--------------------|---------|----------|-------------|------------|
| 缺失类型 $\times$ 缺失率  | 0.9517  | 0.9426   | 0.8772      | 0.8415     |
| 缺失类型 $\times$ 缺失位置 | 0.0053  | 0.0033   | 0.0445      | 0.0614     |
| 缺失位置 $\times$ 缺失率  | 0.0025  | 0.0026   | 0.0273      | 0.0288     |
| 缺失类型（主效应）          | 0.0094  | 0.0132   | 0.0007      | 0.0081     |
| 缺失位置（主效应）          | 0.0002  | 0.0005   | 0.0034      | 0.0041     |
| 缺失率（主效应）           | 0.0004  | 0.0004   | 0.0041      | 0.0053     |

为量化所提方案相对基线的收益，本章把四个模型放在同一退化扫描协议下对比，结果见图 17（左、右图分别为 MAE 与准确率的增量，虚线为零基准）。两个朴素融合基线代表不做缺失处理的常规做法，另两条为本文模型的单种子与集成版本。MRF-Net 的曲线全程位于两个朴素基线下方，且在高缺失率区间差距扩大。



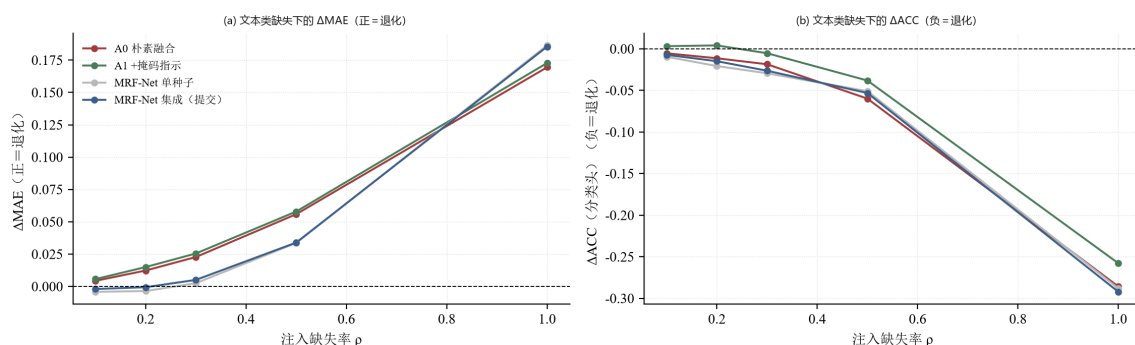


图 17 四个模型在缺失场景下的性能对比

由于三个模态的有效长度不同（文本 50 帧，语音约 24.6 帧、视觉约 23.8 帧），同一缺失率下文本损失的绝对帧数约为另两者的两倍。为排除该口径差异的干扰，本章进一步在匹配绝对缺失帧数的条件下重做对比，在均缺失约 25 帧的匹配组上，文本、语音、视觉的 MAE 增量分别为 +0.0314、+0.0002 与 +0.0068，文本仍是语音的约 170 倍；在约 5 帧的小缺失匹配组上（文本 0.1、语音 0.2、视觉 0.2），三维增量为 -0.0016、+0.0001、+0.0010，均接近零。这说明，文本缺失主导性能退化这一结论不依赖缺失率定义本身。

#### 5.4.4 消融实验与超参数搜索

为保证差值可比，消融实验采用同种子、同基准配方、单开关翻转的配对设计，以种子 42 的基准配方（MAE 0.5871、宏观 F1 0.5857）为参照，每次只关闭一个模块；同时用基准配方在种子 42 与 43 上的表现差（MAE 极差 0.0095、宏观 F1 极差 0.0092）作为种子噪声标尺，若模块的消融幅度小于种子极差，则该模块的单独效应不可辨识。

表 12 同种子配对消融实验结果（验证集）

| 配置               | 种子数 | MAE    | 宏观 F1  | $\Delta\text{MAE}$ | $\Delta$ 宏观 F1 |
|------------------|-----|--------|--------|--------------------|----------------|
| 基准配方（种子 42，配对基准） | 1   | 0.5871 | 0.5857 | —                  | —              |
| 基准配方（种子 43）      | 1   | 0.5967 | 0.5765 | —                  | —              |
| 基准配方两种子极差        | 2   | —      | —      | 0.0095             | 0.0092         |
| 去除掩码指示           | 3   | 0.5866 | 0.5913 | -0.0005            | +0.0056        |
| 去除跨模态重建          | 1   | 0.5825 | 0.5868 | -0.0047            | +0.0011        |
| 单专家（去动态门控）       | 1   | 0.5942 | 0.5733 | +0.0070            | -0.0124        |
| 去除教师蒸馏           | 1   | 0.5913 | 0.5787 | +0.0042            | -0.0070        |
| 去除缺失课程学习         | 1   | 0.5885 | 0.5898 | +0.0013            | +0.0041        |
| 朴素融合（无缺失处理）      | 1   | 0.6010 | 0.5932 | —                  | —              |
| 朴素融合加掩码指示与缺失注入   | 1   | 0.5990 | 0.5782 | —                  | —              |
| 主模型（2 种子集成，最终配方） | 2   | 0.5866 | 0.6158 | —                  | —              |

表 12 给出四条关键证据。（1）所有模块级消融的 MAE 幅度（最大 0.0070）都小于基准配方自身的种子极差 0.0095，即单模块效应比种子抖动还小，说明任何单一模块

都不足以单独解释性能来源；（2）宏观 F1 上仅动态专家门控退化为单专家一项略超种子极差（0.0124 对 0.0092，约 1.35 倍）且双指标同向劣化，是全部模块中最接近可判定的一项；（3）去除跨模态重建使双指标同时改善，与最终配方关闭重建的设计决定一致；（4）最稳健的证据来自朴素融合基线，其 MAE 比基准配方高出 0.0139 与 0.0119，双双超过种子极差，而所有模块级消融均未达到这一幅度。上述证据指向同一结论，即性能来源是缺失检测、掩码参与注意力、缺失注入训练与门控融合的整体配合，无法归因到某个可单独摘除的模块。作为并列参照，主模型的宏观 F1 比基准配方高 0.0301，但其相对基准同时改动了多项（关闭重建、引入有序回归辅助头与弱标注加权、调整轮数并启用权重平均），因此不把该提升归因于任何单项。图 18 以双面板发散条形图呈现各模块效应量与种子噪声带的相对关系，左、右面板分别为  $\Delta\text{MAE}$  与  $\Delta\text{Macro-F1}$ ，灰色竖带为基准配方两种子的极差范围，灰色斜纹条为并列参照的朴素融合基线。

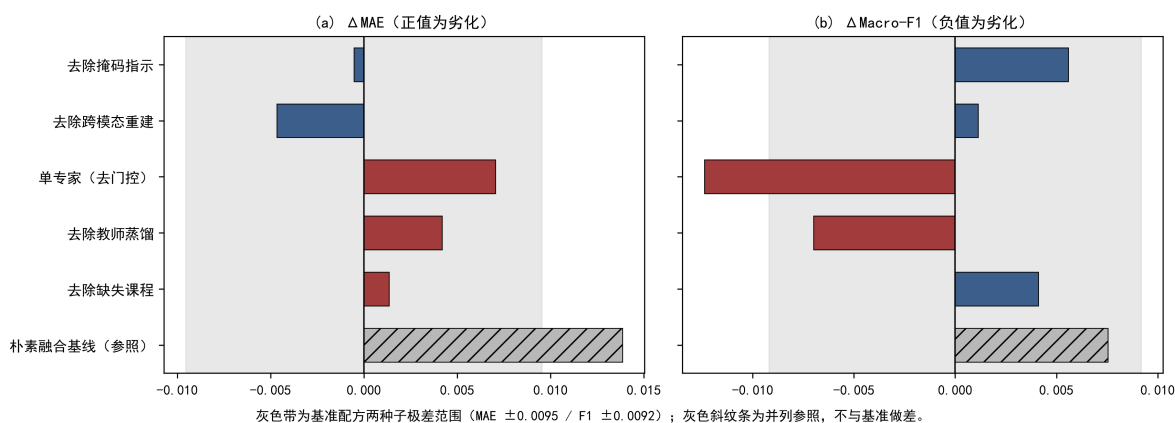


图 18 消融实验对比（左： $\Delta\text{MAE}$ ；右： $\Delta\text{Macro-F1}$ ）

超参数经两阶段搜索确定，先在 1024 条训练子集上快速筛选 4 组候选，再用全量训练集复跑入选配置。表 13 与图 19（哑铃图每行一个候选、左右端点为两阶段指标）汇总了两阶段的结果，宏观 F1 的排名完全一致（秩相关 +1.00），而 MAE 的排名出现反转（秩相关 -0.40），筛选阶段误差最低的隐维度 128 配置在全量训练后成为误差最高者，说明小样本快速筛选对回归误差的排序稳定性有限。以单种子噪声（MAE 标准差 0.0060、宏观 F1 标准差 0.0083）为基准核验后发现，隐维度由 96 增至 128 使 MAE 恶化 +0.0201 ( $z = 2.37$ )、宏观 F1 下降 -0.0362 ( $z = -3.07$ )，是唯一显著的差异，且为显著劣化；蒸馏权重 0.3 到 0.5 与丢弃率 0.35 到 0.50 的差异均落在噪声内 ( $|z| < 0.7$ )，不构成有效改进。由此，本章的结论是当前容量已经足够、不应继续加宽网络；除隐维度不加大外，不宣称任何调参收益。

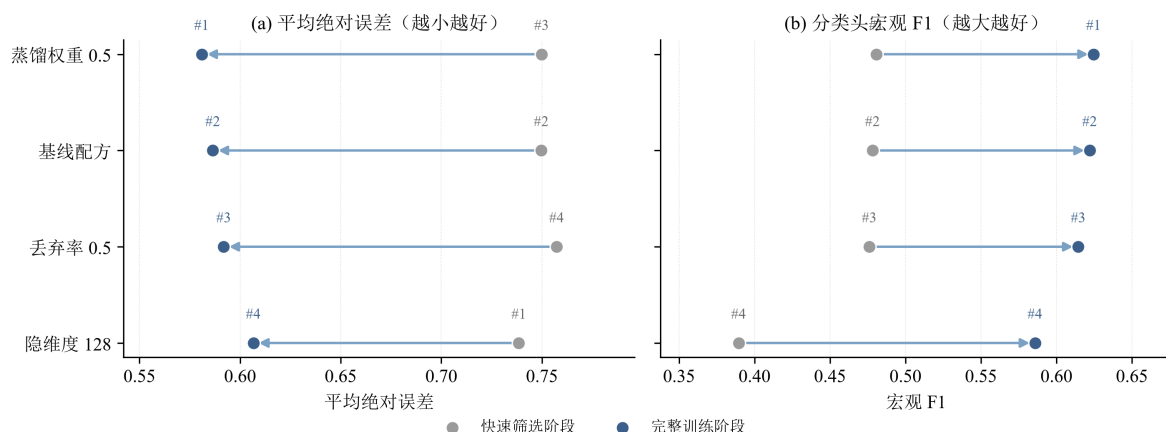


图 19 超参数搜索两阶段对比（哑铃图）

表 13 超参数搜索两阶段的显著性检验

| 候选              | $\Delta$ MAE | $z$   | $\Delta$ 宏观 F1 | $z$   | 判定   |
|-----------------|--------------|-------|----------------|-------|------|
| 蒸馏权重 0.3 到 0.5  | -0.0055      | -0.65 | +0.0026        | +0.22 | 不显著  |
| 丢弃率 0.35 到 0.50 | +0.0054      | +0.64 | -0.0075        | -0.64 | 不显著  |
| 隐维度 96 到 128    | +0.0201      | +2.37 | -0.0362        | -3.07 | 显著劣化 |
| 单种子噪声（参考）       | 0.0060       | —     | 0.0083         | —     | 噪声基准 |

#### 5.4.5 错误归因与不确定性分析

分层误差分析表明，分类与回归的困难来自不同样本。表 14 与图 20 从强度幅值、文本长度、视觉长度、真值极性、是否含缺失五个维度对验证集误差分层（图中柱高为 MAE、折线为分类准确率）。

强度绝对值不超过 0.5 的弱情感样本占验证集 45.5%（331/728），其强度预测较准（MAE 0.402，全桶最低），但分类准确率只有 0.5045，主要困难集中在零点附近的极性边界，这与中性类 F1 偏低的结论一致；相反， $|y| > 1.5$  的强情感样本分类较容易，MAE 却达到 1.080（为弱强度桶的 2.7 倍），说明回归误差主要来自强度幅值。含缺失的 48 条验证样本分类准确率为 0.5208，低于无缺失样本的 0.6294，但由于样本量较小，该差异只能作为趋势性证据。文本长度与视觉长度的影响方向不同，中等长度文本的分类准确率最低（0.583），而视觉有效长度从短到长时 MAE 单调上升（0.492 至 0.669），说明视觉时长主要影响强度回归。

表 14 五维分层错误归因（验证集， $n = 728$ ）

| 维度    | 分组                   | 样本数             | MAE                   | 分类准确率                    |
|-------|----------------------|-----------------|-----------------------|--------------------------|
| 强度幅值  | $ y  \leq 0.5$       | 331             | 0.402                 | 0.5045                   |
|       | $0.5 <  y  \leq 1.5$ | 267             | 0.575                 | 0.6592                   |
|       | $ y  > 1.5$          | 130             | 1.080                 | 0.8462                   |
| 文本长度  | 短 / 中 / 长            | 243 / 242 / 243 | 0.497 / 0.606 / 0.657 | 0.6173 / 0.5826 / 0.6667 |
| 视觉长度  | 短 / 中 / 长            | 243 / 242 / 243 | 0.492 / 0.599 / 0.669 | 0.6214 / 0.5868 / 0.6584 |
| 真值极性  | 负向 / 中性 / 正向         | 206 / 184 / 338 | 0.748 / 0.383 / 0.599 | 0.6408 / 0.6087 / 0.6183 |
| 是否含缺失 | 含缺失 / 无缺失            | 48 / 680        | 0.518 / 0.591         | 0.5208 / 0.6294          |

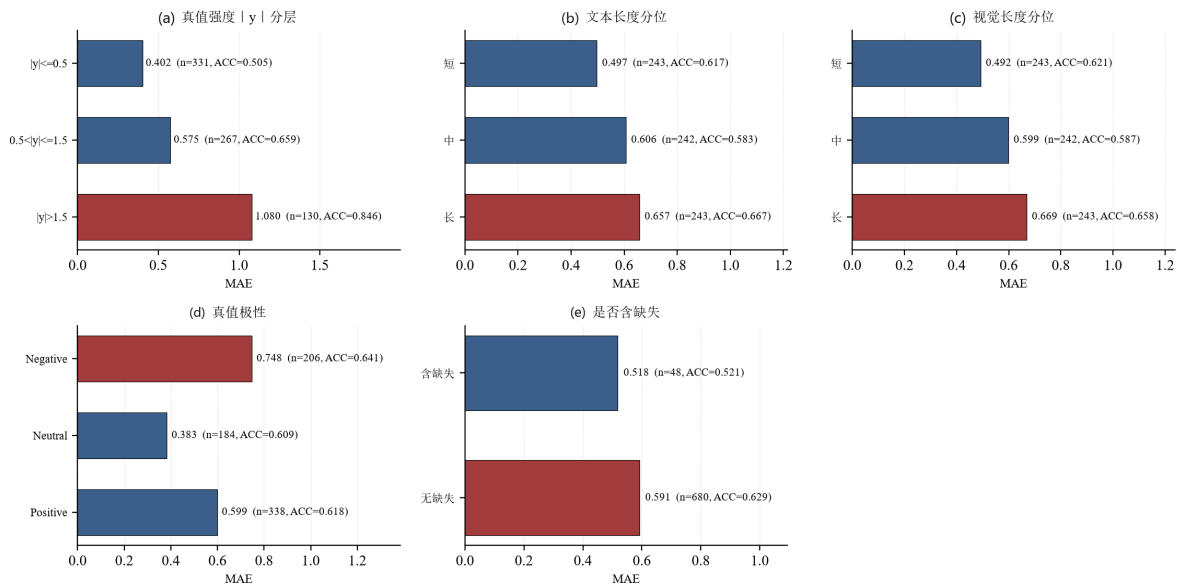


图 20 五维分层错误归因

结合转写文本对误差最大的 20 条样本逐条判读，可将主要失败归纳为五类模式（表 15），其中前三类覆盖了约七成极端误差。（1）强度系统性压缩（数量最多），例如某样本的转写为 “This movie was excellent”，真值强度 +3.0，模型仅给出 +0.63；另一例 “it should be rated junk”，真值 -3.0，模型给出 -0.62，强烈情感词只被赋予约五分之一的幅度，与前述偏差统计一致。（2）转折与否定结构导致极性反转：样本 “But it’s not as bad as I thought it would be”（真值 +1.0）被判为强负（-2.23），“Very long movie but, the time flies by, absolutely fantastic”（真值 +2.67）被压至 -0.83，转写经聚合后从句的极性反转被抹平，否定词的作用域未被显式建模。（3）口语转写噪声干扰：“I’m giving this beast a four out of, (stutter) I mean, (uhh) wait, four, no”（真值 +1.0）被判为 -1.23，填充词与自我纠正使情感信号不稳定。其余两类属于含蓄表达（需语用推理）与语境残缺或标注存疑，可改进空间有限。

表 15 失败模式汇总（基于验证集/测试划分各 20 条极端误差判读）

| 失败模式         | 验证集  | 测试划分 | 机制       | 可改进性   |
|--------------|------|------|----------|--------|
| F1 强度系统性压缩   | 7/20 | 8/20 | 条件均值收缩   | 高      |
| F2 转折/否定反转   | 5/20 | 3/20 | 否定作用域未建模 | 较高     |
| F3 含蓄/隐含评价   | 4/20 | 4/20 | 缺乏语用推理   | 一般     |
| F4 口语转写噪声    | 3/20 | 3/20 | 填充词污染表示  | 高（低成本） |
| F5 语境残缺/标注噪声 | 4/20 | 5/20 | 数据固有     | 低      |

对 20 条最大误差样本的缺失状态统计显示，验证集 20 条均不含缺失，测试划分中也有 17/20 不含缺失。这一现象反过来支持了本章的鲁棒性设计，在现有鲁棒训练下，局部缺失已不再是主要误差来源，语义理解与语用推理才是当前性能的瓶颈；这也解释了消融一节得到的结论。稳健性来自整体配合，而提高上限的方向在表征层，不在缺失处理层。

在基础预测之外，模型还能给出逐样本的不确定度。利用强度回归头的方差输出（偶然不确定度）与两个种子成员之间的分歧（认知不确定度）可以构造总不确定度；分类头概率的熵则刻画极性判别的不确定度。图 21 给出强度回归的风险—覆盖率曲线（黑线为按不确定度剔除、灰虚线为随机剔除），按总不确定度排序剔除样本时，平均绝对误差由 0.5866 降至 0.5465（剔除最不确定的 10% 样本），剔除 50% 后降至 0.4495，曲线下面积（AURC）为 0.4365。图 22 给出极性判别的拒识效果，按分类熵剔除最不确定的 50% 样本后，极性准确率由 0.6223 提升至 0.7637（测试划分 0.8214）。图 23 进一步给出总不确定度与绝对误差的散点关系（斯皮尔曼相关 0.296 与 0.230，验证与测试划分均为正）。两个不确定度并不通用，总不确定度对极性拒识无效（其 AURC 劣于随机），分类熵对强度回归无信息，强度与极性必须分别使用各自的不确定度做拒识，模型因此能够对自身不可靠的判断给出提示。

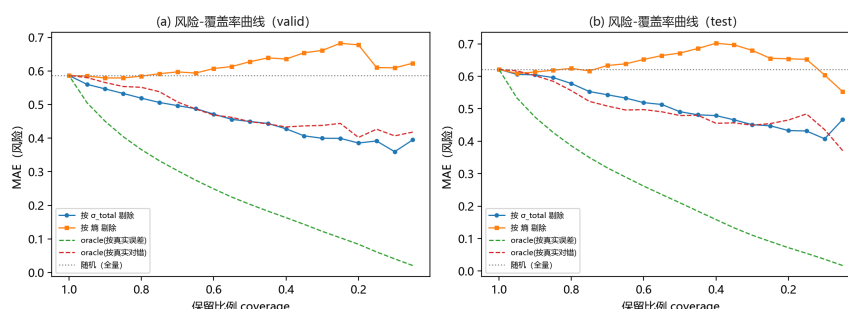


图 21 强度回归的风险—覆盖率曲线

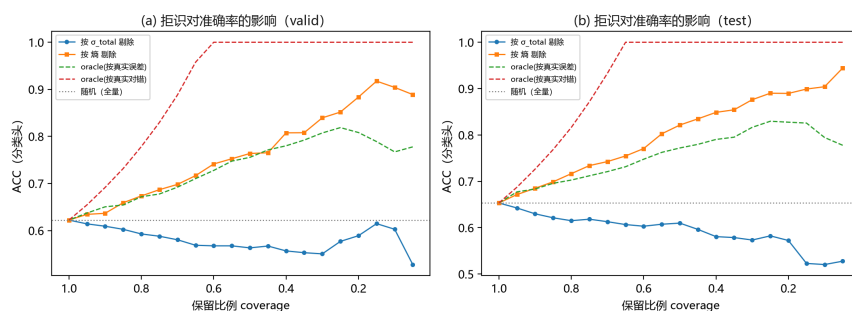


图 22 拒识对极性准确率的影晌

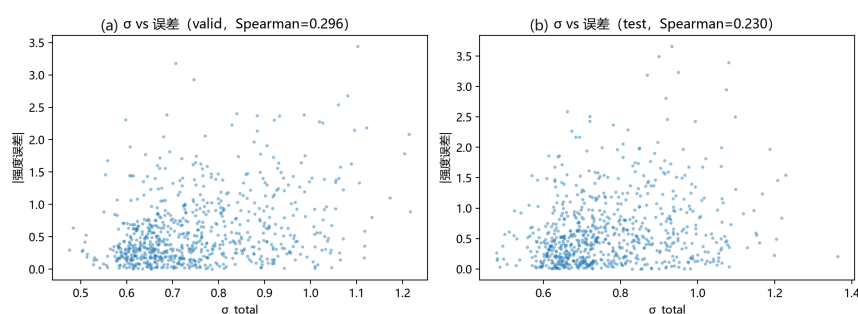


图 23 总不确定度与绝对误差的关系（验证集）

#### 5.4.6 附件三测试集预测结果汇总与展示

附件三共 30 条无标签样本，其缺失为实测结构（表 4），语音 17 条、视觉 18 条含缺失，缺失率均值分别约 0.10 与 0.11，文本无缺失。推理使用 5.3.5 节确定的双种子集成（种子 42 与 43）与决策阈值  $\theta = 0.325$ ，输出极性标签、强度分数、三类概率、置信度与三模态缺失统计。

表 16 与图 24、图 25 给出全量预测的构成与分布（构成图左为极性条数、右为强度与置信度；分布图中虚线为判定阈值）。30 条样本的预测结果为负向 6 条、中性 14 条、正向 10 条；全样本平均强度 +0.2451，平均绝对强度 0.4930，平均置信度 0.5840。按实测缺失状态分组，两组的平均强度拉开差距，含缺失组（18 条）为 +0.1949，低于无缺失组（+0.3204）；但两组的平均绝对强度（0.4304 对 0.5869）与平均置信度（0.5738 对 0.5993）差距不大。这表明缺失样本的预测更接近中性，但模型置信度没有出现异常下降。由于附件三没有标签，这里只能描述输出分布，不能据此判断缺失组的预测更准确或更稳定。预测强度集中于 -1.2 至 1.5、置信度分布于 0.44 至 0.91 之间，分布无异常值。

表 16 附件三测试集的全量预测汇总

| 指标     | 全样本     | 含缺失     | 无缺失     |
|--------|---------|---------|---------|
| 样本数    | 30      | 18      | 12      |
| 预测负向   | 6       | 4       | 2       |
| 预测中性   | 14      | 10      | 4       |
| 预测正向   | 10      | 4       | 6       |
| 平均强度   | +0.2451 | +0.1949 | +0.3204 |
| 平均绝对强度 | 0.4930  | 0.4304  | 0.5869  |
| 平均置信度  | 0.5840  | 0.5738  | 0.5993  |
| 两口径一致率 | 0.8333  | —       | —       |

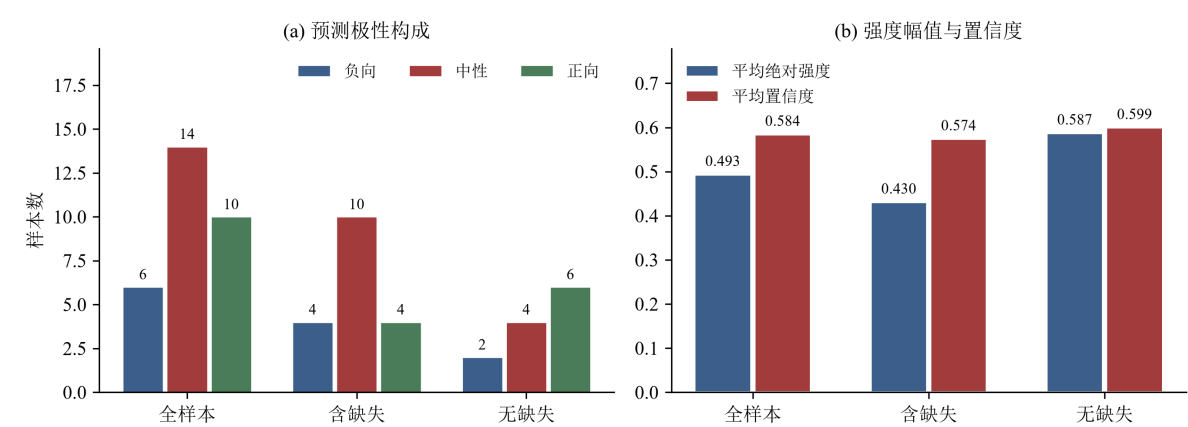


图 24 附件三预测结果构成（全样本 / 含缺失 / 无缺失）

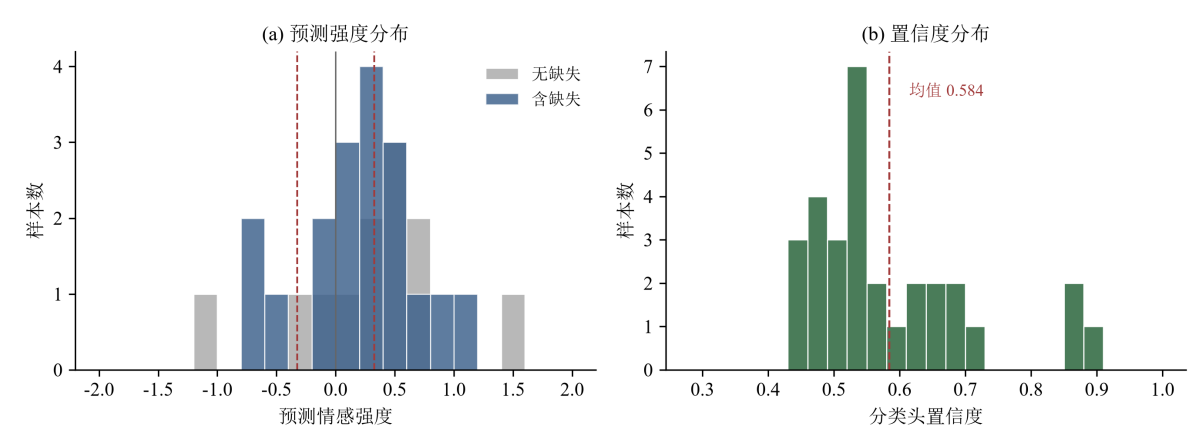


图 25 附件三预测强度与置信度分布

为直观展示缺失结构如何作用于单条样本，图 26 选取两条代表性样本给出逐帧缺失时间线与预测输出。两条样本的语音与视觉缺失区间完全同步（源于同一视频流的分段丢失），文本流完整。附件 3\_26 缺失 4/22 帧（18.2%），集中在头部与中部两段，模



型仍以 0.76 的概率判为正向；附件 3\_30 缺失达 10/25 帧（40.0%）、跨越头、中、尾三段，预测强度降为 -0.11、负向概率仅 0.45，输出明显向中性收缩，缺失越深、判断越保守，与表 16 的组间对比一致。

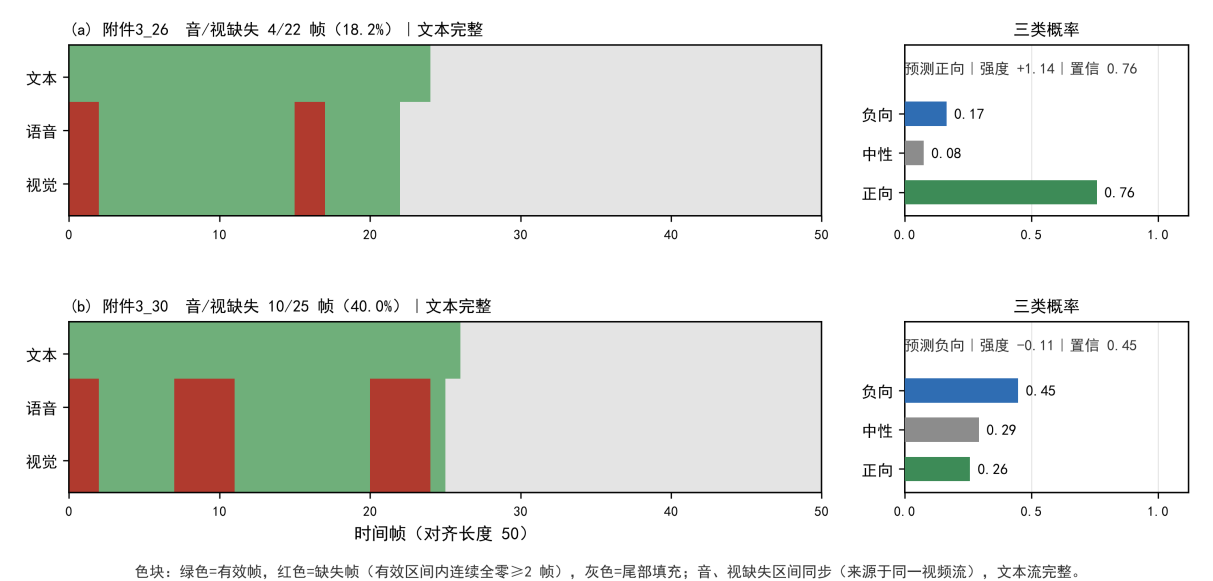


图 26 附件三代表性样本的缺失时间线与预测卡（上：附件 3\_26；下：附件 3\_30）

提交文件 pred\_att3.csv 共 15 个字段，覆盖判定结果、概率、置信度、阈值与缺失统计，便于复核（表 17）。为核验口径稳健性，同时计算了分类头口径与阈值口径两种判别结果，二者的一致率为 0.8333；两种口径的差异集中在强度分数接近阈值的中性边界样本上，与错误归因一节的边界困难结论一致。预测结果之外，还需交代两个专项测试集本身的输入条件，按式 (1) 逐一统计可见，附件三语音与视觉模态的含缺失样本比例分别为 56.7% 与 60.0%，平均实测缺失率约 0.10，且以短碎片形式出现；文本模态在三套数据中均无实测缺失，附件四有 1 条样本视觉模态整条缺失。附件三的文本鲁棒性因此无法直接验证，只能依靠训练期的文本缺失注入兜底，本文在 5.5 节一并说明。

表 17 提交文件主要字段说明

| 字段                                        | 含义                          |
|-------------------------------------------|-----------------------------|
| id                                        | 样本编号（附件 3_01 至附件 3_30）      |
| pred_label / pred_label_theta             | 分类头口径 / 阈值口径的极性判定           |
| pred_score / prob_neg, prob_neu, prob_pos | 情感强度分数 / 三分类概率              |
| confidence / confidence_theta / theta     | 置信度 / 阈值口径置信度 / 所用阈值（0.325） |
| missing_text_ratio 等三列                    | 三模态实测缺失率（与 5.2 节口径同源）       |
| n_missing_intervals                       | 缺失区间段数                      |



## 5.5 本章小结

本章针对局部模态缺失建立了 MRF-Net，并在统一缺失口径下完成了鲁棒性建模、缺失规律扫描、消融验证和附件三推理。数据层面，尾部填充与有效区间内缺失必须分开统计；由于附件二真实缺失极少，训练阶段需要主动注入缺失。模型层面，掩码感知编码、动态专家门控和课程式缺失训练共同构成鲁棒框架，消融结果更支持整体框架的有效性，而非单模块的决定作用。

规律分析显示，性能退化主要由文本缺失驱动，并且头部缺失的影响大于中部和尾部。MRF-Net 将缺失类型与缺失率交互的效应量由 0.9517 降至 0.8415，不同缺失位置下的平均退化较朴素基线降低约 1.5 至 3.3 倍。最终模型在验证集上取得 MAE 0.5866、Pearson 0.6543、ACC 0.6223 和 Macro-F1 0.6158；附件三的 30 条预测已按统一格式提交，预测分布为负/中/正 = 6/14/10。

本章仍有两点限制。附件三和附件四都没有文本真实缺失，文本分支的鲁棒性主要依赖模拟注入验证；验证集中含缺失样本只有 48 条，部分统计结果的不确定性较大，且注入缺失率（模拟口径）与实测缺失率（附件三口径）语义不同，全章已分别标注。验证集同时承担早停、阈值选择和部分归因分析，可能带来轻微乐观偏差，因此测试划分只作为最终泛化核验，不再参与模型选择；文本鲁棒性问题将结合问题三的可解释性框架进一步分析。

# 六、问题三模型的建立与求解

## 6.1 问题分析

问题三不再只关注预测是否正确，而是要求说明预测依据，即三种模态分别起多大作用、模型主要依赖哪一种模态、每个模态中哪些局部片段与最终判断关系最密切；关键证据还需要回到原始文本、语音时段或视频关键帧，保证可以人工复核。解释结论需与预测结果一并提交。解释本身也必须经得起检验，因为解释不可信时，模型的判断依据就无从核验。

综合上述要求，本问的求解可以拆分为四个环节。从骨干冻结与解释头训练开始，直接沿用问题二的骨干并冻结全部参数，在其上并联一个轻量证据头，以忠实性与充分性为训练目标；冻结骨干保证问题三的预测与问题二逐位一致，解释头不接入预测通路，训练解释不会影响预测结果。随后计算三层解释，模态级用精确枚举的 Shapley 值量化各模态作用度，片段级用积分梯度与遮挡分析定位关键区段，证据级用反事实删除把重要性落到具体词片段、秒区间与关键帧；博弈论、梯度与遮挡三类方法互相独立，结论可以交叉核验。进而完成融合与权重标定，三源重要性按验证集上与遮挡基准一致性最大的准则确定融合权重，使解释以模型无关方法为锚。而后进行质量评估与证据回填，用忠实性、充分性、稀疏性与稳定性四项指标评估解释质量，并通过删模态退化一致性

与附件三免标注检验验证机制，把重要片段回映到原文词句、语音时段与视频关键帧，得到解释卡与附件四的提交文件。整体求解流程如图 27 所示。

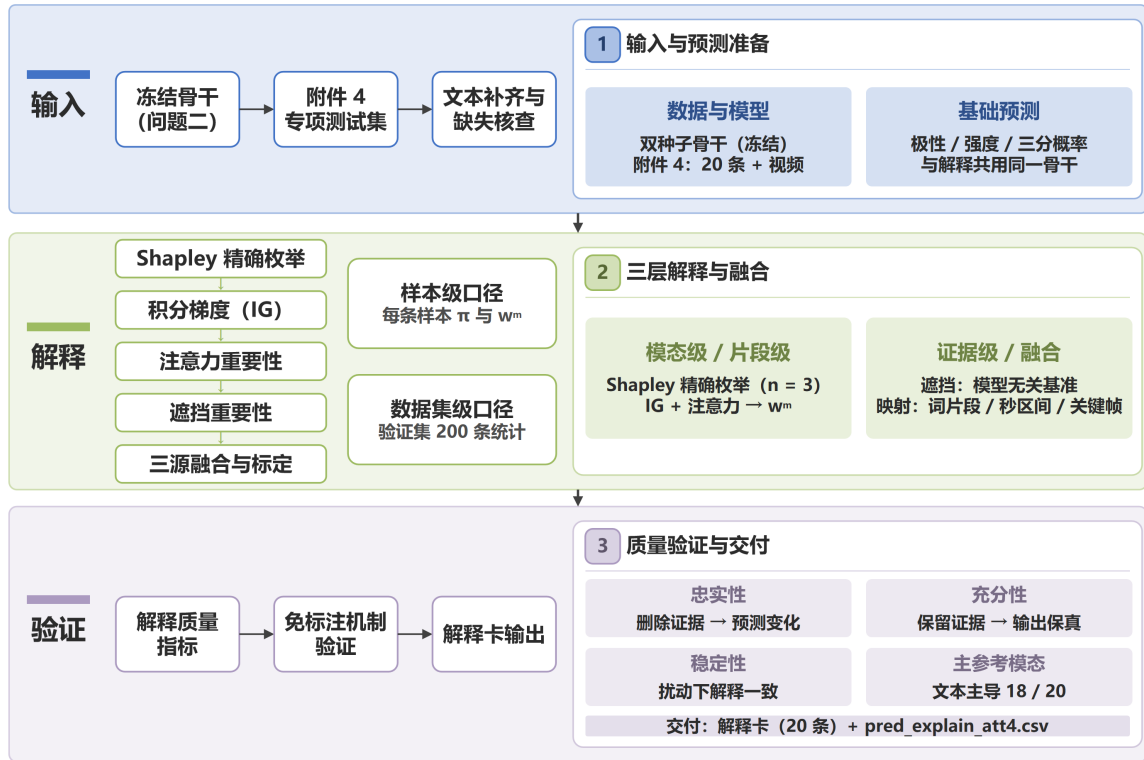


图 27 问题三的求解流程

## 6.2 模型建立

### 6.2.1 可解释性的形式化定义与方案选择

设骨干网络  $f$  将三模态输入  $\mathbf{X} = (\mathbf{X}^t, \mathbf{X}^a, \mathbf{X}^v)$  映射为强度预测  $\hat{y}$  与极性概率  $\hat{p}$ 。对每个样本，解释输出包括三部分，即三模态作用度向量、主要参考模态，以及每个模态 50 个时间步的重要性向量；可解释性要求同时给出三个层次的量化结果：

$$\boldsymbol{\pi} = (\pi_t, \pi_a, \pi_v), \quad \sum_{m \in \mathcal{M}} \pi_m = 1, \quad m^* = \arg \max_m \pi_m, \quad \mathbf{w}^m \in \mathbb{R}_{\geq 0}^T, \quad (24)$$

式中  $\pi_m$  为模态  $m$  对本样本预测的作用程度， $m^*$  为主要参考模态， $\mathbf{w}^m$  为模态  $m$  内各时间步 ( $T = 50$ ) 的重要性向量。题目要求解释结果可量化、可复核，因此我们没有直接把模型内部注意力当作解释结果，而是要求每一项解释都能通过独立实验验证。

图 28 勾勒本问的解释路线，模态级用 Shapley 值计算作用度向量  $\boldsymbol{\pi}$ ，衡量各模态对预测的贡献；片段级用积分梯度与证据头刻画时间重要性  $\mathbf{w}^m$ ，定位关键的区段；证据级用反事实删除与遮挡把重要性落到可回填的词片段、秒区间与关键帧。三路结果汇入

同一张解释卡，再经一致性校验与免标注机制检验。

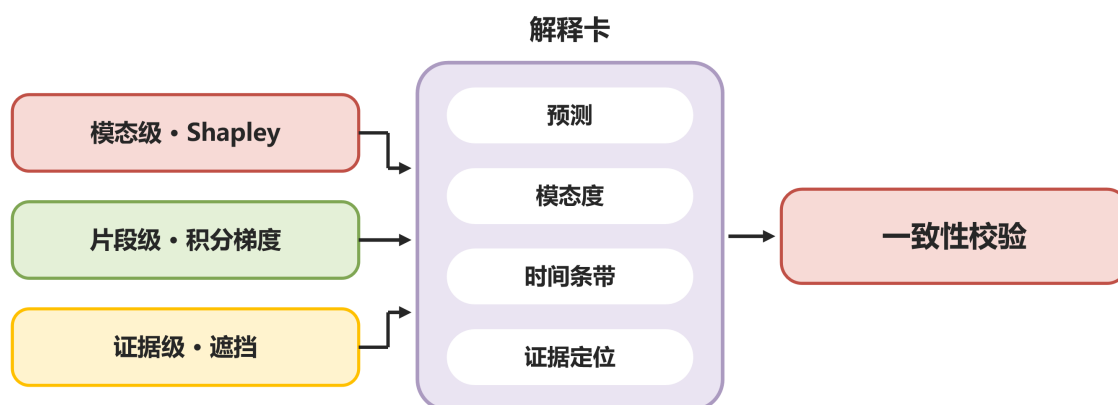


图 28 三层解释通路示意

可选的解释路线有三类，对比如表 18 所示。注意力权重来自模型内部、计算零成本，但其大小受缩放方式、层数与头数影响，已有研究指出注意力与真实重要性不一致的反例，故仅作为对照基线；代理模型需要额外的近似采样，解释方差不可控；精确 Shapley 在模态数仅为三时可对全部  $2^3$  个子集精确枚举，不存在近似误差，再配合积分梯度与遮挡，就构成了公理化、细粒度与模型无关的三重校验。

表 18 可解释性方案候选对比

| 候选                 | 方法族       | 主要不足               | 是否采用 |
|--------------------|-----------|--------------------|------|
| 注意力权重              | 模型内生      | 注意力与重要性的对应关系无公理化保证 | 仅作基线 |
| 代理模型近似             | 模型无关近似    | 需采样近似，结果随随机种子波动    | 否    |
| Shapley+ 积分梯度 + 遮挡 | 博弈论/梯度/遮挡 | 需多次前向（本任务算力可承受）    | 采用   |

选定该组合的量化依据见 6.3.3 小节。实测中，注意力权重与遮挡结果的秩相关仅为  $-0.0268$ ，说明直接以注意力权重作解释在本模型上并不成立；改用三源融合后，与遮挡基准的一致性达到  $0.9561$ ，因此最终采用后者。

本问与题目给定文献存在如下对应与差异。Locate-and-Explain[15] 面向会话级情绪原因抽取与摘要，输出自然语言摘要，我们输出的是可量化、可复核的字符区间与秒区间，粒度更细且可直接回填提交文件；不确定性感知的生成式方法 [10] 关注缺失下的可信预测，我们把不确定度用于解释质量的稳健性核验；因果不变性视角 [14] 与本文关于注意力不等于重要性的实证呼应，说明解释方法需经外部基准校验；知识蒸馏与动态调整 [18] 作用于骨干训练期，与本文解释头不接入预测通路的设计互补。

### 6.2.2 可解释性证据头的结构与目标函数

为了让解释本身可训练，本文在冻结骨干上并联一个轻量证据头。记骨干第  $m$  个模态编码器的输出为  $\mathbf{H}^m \in \mathbb{R}^{T \times d}$  ( $d$  为隐维度)，证据头先给出有效时间步上的证据分布：

$$s_k^m = \frac{\exp(\mathbf{w}_m^\top \tanh(\mathbf{W}_m \mathbf{h}_k^m + \mathbf{b}_m))}{\sum_{k' \in \mathcal{V}^m} \exp(\mathbf{w}_m^\top \tanh(\mathbf{W}_m \mathbf{h}_{k'}^m + \mathbf{b}_m))}, \quad k \in \mathcal{V}^m, \quad (25)$$

式中： $\mathbf{h}_k^m$  为第  $k$  个时间步的表示， $\mathcal{V}^m$  为该样本模态  $m$  的有效时间步集合，分母限制在有效位内求和，使填充位不参与证据分配； $s_k^m$  即片段级重要性的可训练形式。再以证据加权的模态表示汇总为模态作用度：

$$\bar{\mathbf{h}}^m = \sum_{k \in \mathcal{V}^m} s_k^m \mathbf{h}_k^m, \quad \boldsymbol{\pi} = \text{softmax}\left((\mathbf{u}^\top \tanh(\mathbf{V} \bar{\mathbf{h}}^m))_{m \in \mathcal{M}}\right). \quad (26)$$

证据头不接入预测通路，因此式 (26) 的  $\boldsymbol{\pi}$  不影响  $\hat{y}$  与  $\hat{\mathbf{p}}$ ，问题三的预测与问题二逐位一致。训练目标是让  $\mathbf{s}$  与  $\boldsymbol{\pi}$  忠实于骨干的真实决策依据。记  $t(X)$  为骨干的强度输出， $X \setminus^s$  表示把模态  $m$  的缺失指示置为  $s^m$  后得到的输入 ( $s^m$  越大表示该段越应被移除)， $X^{\cap s}$  表示只保留  $s^m$  加权的信息，则损失函数为：

$$\mathcal{L} = \alpha_f \left[ - (t(X) - t(X \setminus^s))^2 + (t(X) - t(X^{\cap s}))^2 \right] + \alpha_1 \sum_m \|\mathbf{s}^m\|_1 + \alpha_2 \sum_m \text{TV}(\mathbf{s}^m), \quad (27)$$

式中第一项为忠实性项，按  $\mathbf{s}$  移除信息后预测变化越大越好；第二项为充分性项，只保留  $\mathbf{s}$  加权信息时应尽量接近全模态预测；第三项为稀疏项，鼓励证据聚焦；第四项为时间全变差项，抑制证据在相邻时间步之间剧烈跳变， $\text{TV}(\mathbf{s}^m) = \sum_k |s_{k+1}^m - s_k^m|$ 。由于缺失指示在编码器内以实数参与运算，式 (27) 对  $\mathbf{s}$  可微，可用反向传播直接优化。关键参数取  $\alpha_f = 1$ 、 $\alpha_1 = 10^{-3}$ 、 $\alpha_2 = 10^{-3}$ ，在验证集上以忠实性减充分性偏差作为选择目标；优化器为 AdamW，学习率  $10^{-3}$ ，批量 64，训练 12 轮并按验证目标早停，骨干权重全程冻结。该目标值由首轮的  $-0.6503$  提升到最优轮（第 11 轮）的  $-0.6103$ ，证据头参数量约 9 万，保存于支撑材料的 runs/q3/q3\_evidence\_head.pt。

### 6.2.3 模态级与片段级解释的计算

把模态集合记为  $\mathcal{M} = \{t, a, v\}$ ，对任一子集  $S \subseteq \mathcal{M}$ ，定义效用  $v(S)$  为仅使用  $S$  中模态时模型的目标标量（以强度  $\hat{y}$  为主口径，极性概率为对照），模态  $m$  的 Shapley 值为：

$$\varphi_m = \sum_{S \subseteq \mathcal{M} \setminus \{m\}} \frac{|S|! (3 - |S| - 1)!}{3!} [v(S \cup \{m\}) - v(S)]. \quad (28)$$

因模态数仅为 3，式 (28) 只需枚举  $2^3 = 8$  个子集即可精确求得，没有近似误差。逐样本回代即得样本级  $\varphi_m$ ；把  $v(S)$  取为验证集上的负平均绝对误差或宏观 F1，即得数据集级的模态重要性。实现时使用效率公理进行自检：验证  $\sum_m \varphi_m = v(\mathcal{M}) - v(\emptyset)$ ，全部样本的残差不超过  $10^{-15}$ 。

图 29 以格图示意枚举过程，从空集出发沿加入一个模态的边逐层到达全集，每条边的增量是一次边际增益  $F(S \cup \{i\}) - F(S)$ ；同一模态在全部路径上的边际增益按组合权重取平均，即式 (28)。 $n = 3$  时全部子集与边际增益都可精确计算，无需任何采样近似。

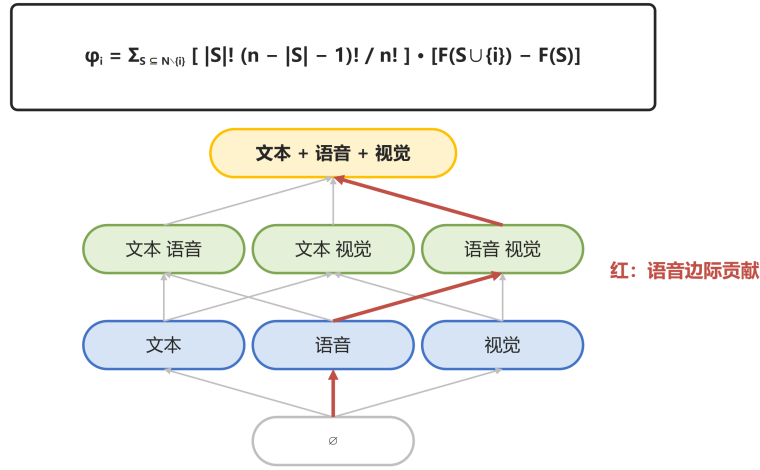


图 29 三模态 Shapley 值枚举示意

屏蔽某模态时，本文把该模态所有有效位置的缺失指示置 1，即使用模型在训练期见过的整模态不可用语义；作为对照，另算把该模态整段置为填充（完全不可见）的口径。由于效用函数并非恒正， $\varphi$  可能出现全负（表示该模态相对无信息先验起抑制作用），因此采用两种归一化，两种口径都会报告，并以与真实删模态退化的秩相关择优（见 6.3.3 小节）：

$$\pi_m^{\text{relu}} = \frac{\max(\varphi_m, 0)}{\sum_{m'} \max(\varphi_{m'}, 0) + \epsilon}, \quad \pi_m^{\text{soft}} = \frac{\exp|\varphi_m|}{\sum_{m'} \exp|\varphi_{m'}|}. \quad (29)$$

屏蔽语义与效用目标是否影响结论，也做了对照检验，把屏蔽由缺失指示置 1 改为整段置为填充后，两组  $\varphi$  的秩相关为 0.5782；把效用由强度目标换为极性概率目标后，与强度口径的秩相关为 0.9758。可见，模态级排序对效用定义高度稳健，对屏蔽语义仅中等稳健；两种语义分别对应模态不可用与模态不可见两类现实场景，本文明确以不可用为主口径。

片段级重要性来自三个互补的来源：模型内生注意力  $\mathbf{w}^{\text{att}}$ 、积分梯度  $\mathbf{w}^{\text{ig}}$  与遮挡  $\mathbf{w}^{\text{occ}}$ 。其中积分梯度以训练集均值特征为基线（归一化空间中即零向量），逐点累计梯度得到每个时间步的重要性：

$$\mathbf{w}_k^{\text{ig}} = \left\| (\mathbf{X}_k^m - \mathbf{X}_k^{\prime m}) \odot \frac{1}{L} \sum_{l=1}^L \nabla_{\mathbf{X}_k^m} t\left(\mathbf{X}' + \frac{l}{L}(\mathbf{X} - \mathbf{X}')\right) \right\|_2, \quad \mathbf{X}' = \mathbb{E}_{\text{train}}[\mathbf{X}^m], \quad (30)$$

它对输入满足完备性公理，且可下钻到词元维度。遮挡源把窗口  $[k, k + \omega)$  用相邻段替换（保持序列长度，避免删段变短的伪影），以目标标量的变化量作为该段重要性：

$$w_k^{\text{occ}} = |t(\mathbf{X}) - t(\mathbf{X}_{\setminus [k, k+\omega)})|, \quad \omega = 3. \quad (31)$$

三种来源先在各模态有效区间内做极值归一化，再加权融合：

$$\mathbf{w}^m = \text{norm}\left(\gamma_1 \tilde{\mathbf{w}}^{\text{att}} + \gamma_2 \tilde{\mathbf{w}}^{\text{ig}} + \gamma_3 \tilde{\mathbf{w}}^{\text{occ}}\right), \quad (32)$$

权重  $\gamma$  在验证集上以与遮挡源（模型无关基准）的秩相关最大为准则，在约束  $\gamma_i \geq 0.1$  的单纯形上枚举，得  $\gamma = (0.1, 0.1, 0.8)$ 。之所以给权重设下界，是因为一旦允许取零，最优解会退化为只保留遮挡源，一致性检验也就失去了意义。

#### 6.2.4 证据定位规则

题面要求关键证据可对应至原始文本片段、语音时段或视觉关键帧。附件二不含逐词时间戳，因此需要先确认对齐粒度，在验证集上统计三模态的平均有效长度，文本为 25.59 步、语音 24.59 步、视觉 23.82 步，且文本长度与语音长度的皮尔逊相关系数为 1.000、平均相差 1.00 步（语音少一步对应起始标记），说明对齐版本基本按词级组织。据此为三种模态建立各自的映射规则。

文本证据按词元位置落定，段号即词元位置，用附件给出的文本词元序列（`text_bert`）编号与 BERT 词表（行号即编号）解码 WordPiece 词元、合并续接标记得到词序列，再用序列匹配把词对齐回原始转写文本，得到字符区间；对齐率低于 0.8 的样本单独标记，不做静默降级。

语音证据把高重要度的段号合并为连续区间，按  $t_k \approx \frac{k+0.5}{L_a} T_{\text{video}}$  映射为秒，并声明该映射为等距近似（附件二不含逐词时间戳），同时用视频音频轨的能量曲线佐证。

视觉证据在最高重要度区间内选择视觉活跃度最大的实际帧，导出该时刻的关键帧图像供回看。

## 6.3 结果分析与可视化

### 6.3.1 评价指标与解释质量度量

问题三的预测骨干与问题二相同，因此预测性能继续沿用 5.4.1 节的评价指标与口径，极性采用准确率与宏观 F1、强度采用 MAE 与皮尔逊相关系数（必要时附 CCC），验证集为主报口径、测试划分为泛化核验，预测结果与问题二逐位一致，不重复训练新的预测模型。本问还要回答解释本身好不好这一问题，为此设计了四个指标，另加两项机制检验。

**忠实性 (Faithfulness):** 删除该模态下重要性排名前 5 的片段后，预测标量的变化量；变化越大，说明被选片段越关键。

**充分性偏差 (Sufficiency gap):** 仅保留前 5 个重要片段时与全模态预测的差距；差距越小，说明被选片段越充分。

**稀疏性 (Sparsity):** 前 5 个片段的权重占模态内总权重的比例；过高说明证据过度集中，过低说明解释未聚焦。

**稳定性 (Stability):** 输入加标准差 0.05 的高斯扰动后重算解释的  $L_1$  变化；变化越小说明解释越稳定。

**机制检验一（与删模态退化的一致性）:** 模态作用度排序应与真实删模态造成的退化排序一致，且样本级应呈正相关。**机制检验二（免标注检验）:** 附件三、附件四带有实测缺失位置，若解释机制成立，则缺失率越高的模态其作用度应越低；该检验不依赖人工标注，可作为解释正确性的外部证据。

### 6.3.2 典型样本解释卡

解释卡把单个样本的全部解释结果组织为一页可视报告，包括预测结果（极性、强度、置信度）、三模态作用度构成、主要参考模态、各模态时间重要性条带、文本证据词级着色、视觉关键帧与音频能量时段。图 30 展示附件四中预测为正向的样本 17（解释卡含时间重要性条带、文本词级着色、视觉关键帧与音频能量四个面板），模态作用度以文本最大，关键证据落在情绪极性词上，说明模型的判断路径与人类阅读直觉一致。

解释卡 • 样本 17 预测：正向（强度 +1.553， $\theta=0.325$ ，主模态 文本）

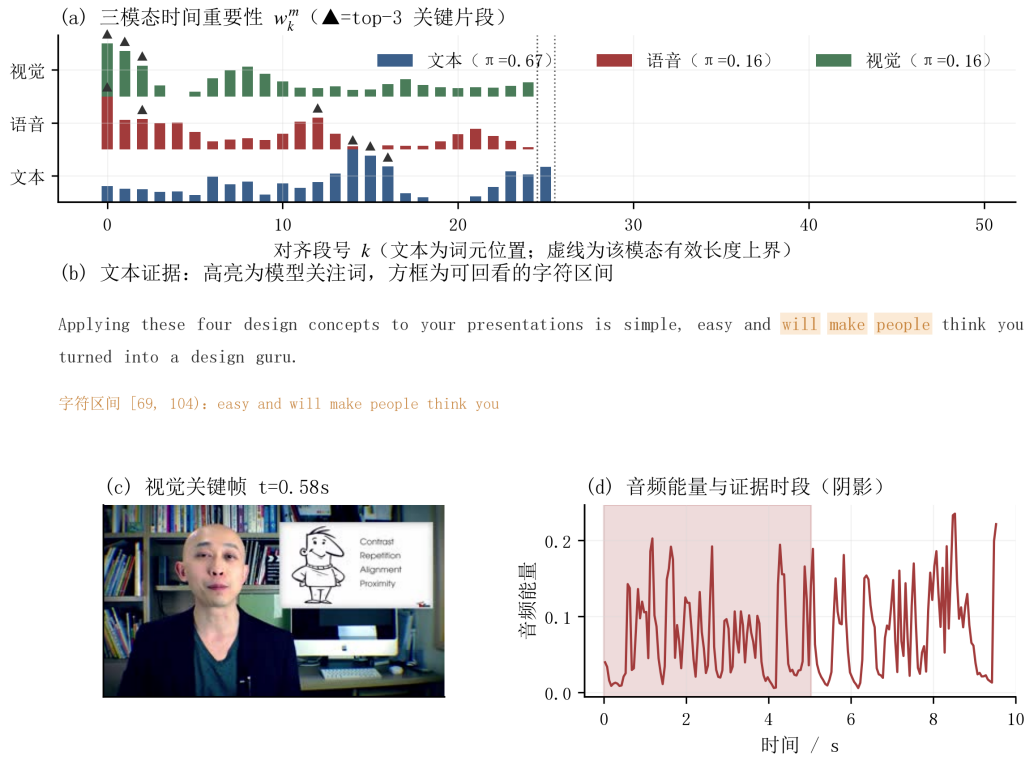


图 30 正向样本 17 的解释卡

图 31 与图 32 分别展示预测为负向与中性的解释卡，覆盖三种极性情形。负向样本 16 的文本证据包含明确的否定与贬义线索，模型的关注词与该线索吻合；中性样本 03 的文本证据为事实性描述，其片段重要性分布更为分散、三模态作用度更趋均匀。三类解释卡共同说明，情感越明确，模型越集中地依赖文本证据，而中性样本更容易出现多模态作用度接近的情况。这也给 6.3.4 小节的错误关联分析留下个体层面的直观例证。



解释卡 · 样本 16 预测：负向（强度 -2.492， $\theta=0.325$ ，主模态 文本）



图 31 负向样本 16 的解释卡

解释卡 · 样本 03 预测：中性（强度 -0.299， $\theta=0.325$ ，主模态 文本）



图 32 中性样本 03 的解释卡

6.3.3 三模态作用差异与一致性检验

图 33 汇总了验证集 728 条样本的模态作用度（箱体为四分位区间），文本模态的平均作用度为 0.4620，远高于语音（0.2627）与视觉（0.2754）；按真值极性分层时，负向样本的文本作用度最高（0.5139）、中性样本最低（0.4068）；按强度幅值分层时，文本作用度从  $|y| \in [0, 0.33)$  的 0.4072 单调升至  $|y| \in [2, 3]$  的 0.5616，作用度熵由 1.0723 降至 0.9452，即情感越明确、文本依据越集中；语音与视觉的分布在三类情感之间高度重叠，其辅助作用不随极性方向变化。

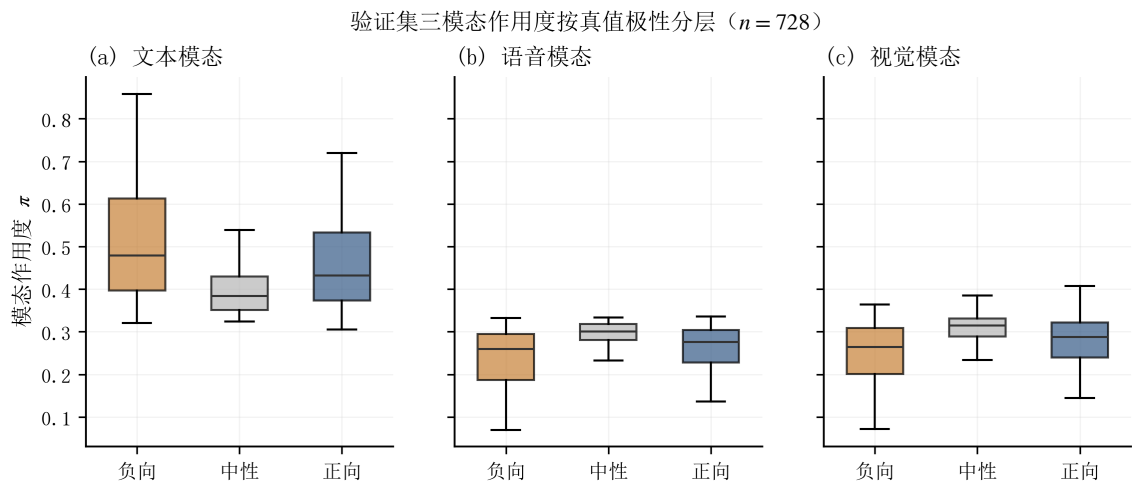


图 33 验证集三模态作用度按真值极性分层的箱线图

全局子集实验与上述结果一致。以数据集级的负平均绝对误差与宏观 F1 为效用，可得到全局模态重要性；八个模态子集的验证集结果列于表 19。文本单独使用即可把平均绝对误差由 0.7816 降到 0.5936，接近全模态的 0.5866，而语音或视觉单独使用几乎无效；全局 Shapley 值呈现同样的排序，文本 0.1842、语音 0.0004、视觉 0.0103（以负平均绝对误差为效用）。

表 19 八个子集的模态组合在验证集上的表现与全局 Shapley 值

| 模态子集    | 强度 MAE | 宏观 F1  | 全局 $\varphi$ ( $-MAE$ 口径) |
|---------|--------|--------|---------------------------|
| 无（全屏蔽）  | 0.7816 | 0.2460 | —                         |
| 文本      | 0.5936 | 0.6091 | +0.1842                   |
| 语音      | 0.7808 | 0.2560 | +0.0004                   |
| 视觉      | 0.7676 | 0.3111 | +0.0103                   |
| 文本 + 语音 | 0.5933 | 0.6089 | —                         |
| 文本 + 视觉 | 0.5867 | 0.6184 | —                         |
| 语音 + 视觉 | 0.7670 | 0.3111 | —                         |
| 全模态     | 0.5866 | 0.6158 | —                         |

这一排序与问题二的缺失实验完全一致。问题二在缺失率为 1.0 时测得文本、语音、视觉的平均绝对误差增量分别为 +0.1804、+0.0002、+0.0068，即退化由文本主导；本问用完全不同的手段（模态删除后的误差增量，而非缺失注入）测得三者分别为 +0.1804、+0.0001、+0.0067，两者排序完全一致，秩相关系数为 1.000。因此，文本主导这一结论不依赖具体实验手段。图 34 呈现样本级的一致性散点（横轴为删模态后的 MAE 增量、纵轴为作用度，颜色区分模态）： $\pi$  的取值以  $\Delta\text{MAE} = 0$  为轴、随  $|\Delta\text{MAE}|$  增大而升高，样本级秩相关为 0.156，散点整体呈 V 形包络，即作用度越高的模态对应更大的预测退化，两种度量给出一致的结论。

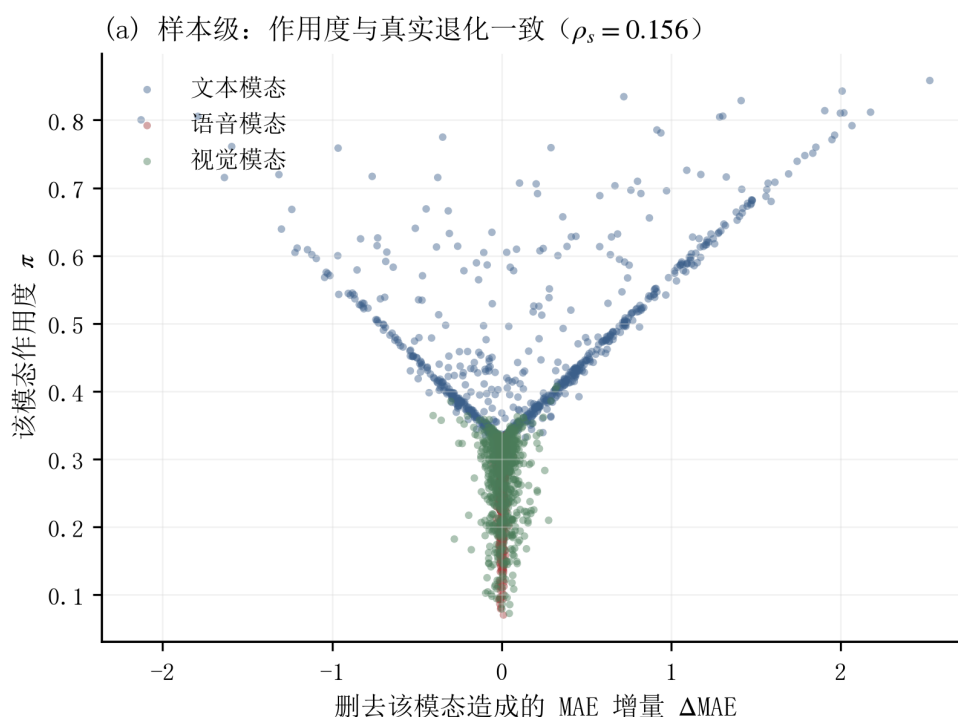


图 34 样本级一致性检验

进一步地，附件三与附件四的样本带有实测缺失位置，可用于无需标注的机制检验，若解释机制成立，则缺失率越高的模态其作用度应越低。附件三的 30 条样本中， $\pi$  与实测缺失率的秩相关系数为  $-0.4011$ ，且含缺失模态的平均作用度（0.2898）低于无缺失模态（0.3610）；附件四中仅 1 条样本含缺失（秩相关 +0.0413，样本量不足不作结论）；把两套数据合并（150 组模态-样本观测）后秩相关为  $-0.202$ ，整体呈缺失越多、作用度越低的趋势（图 35，圆点为附件三、三角为附件四）。

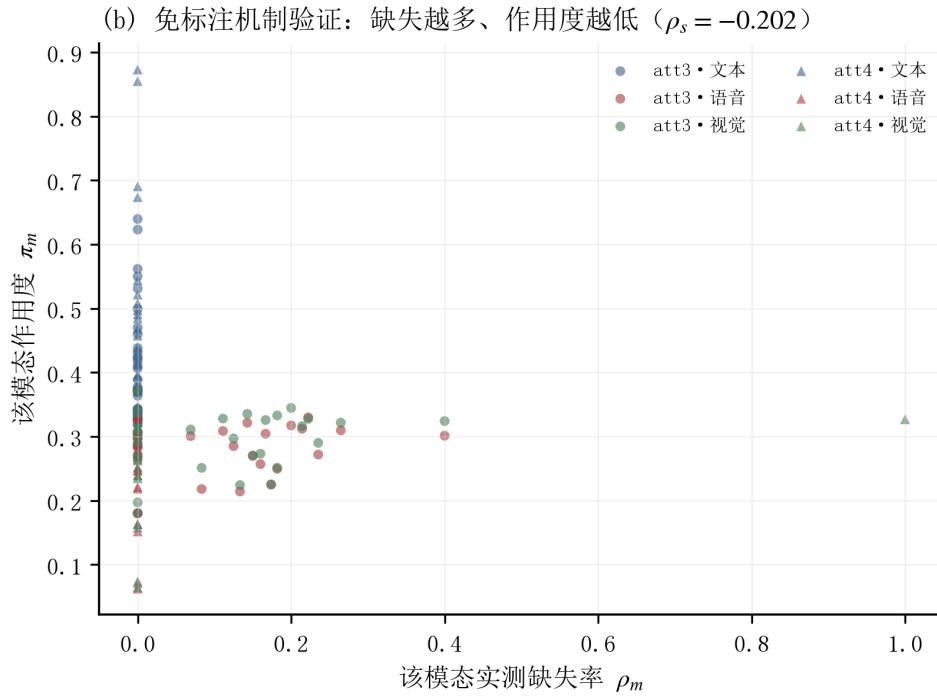


图 35 免标注机制验证：模态缺失率与作用度的关系

四种解释来源的定量对比如图 36 与表 20 所示（三类指标分别为忠实性、充分性偏差与稀疏性）。注意力源在充分性上明显最差（充分性偏差 0.2596），忠实性与稀疏性也均非最优；积分梯度与遮挡源的忠实性更高；三源融合结果在忠实性（0.0146）与稀疏性（0.4392）之间取得折中，其权重  $\gamma = (0.1, 0.1, 0.8)$  由验证集按与遮挡源一致性最大的准则标定，对应一致性为 0.9561（等权方案更低）。因此本文以融合结果作为最终解释输出。

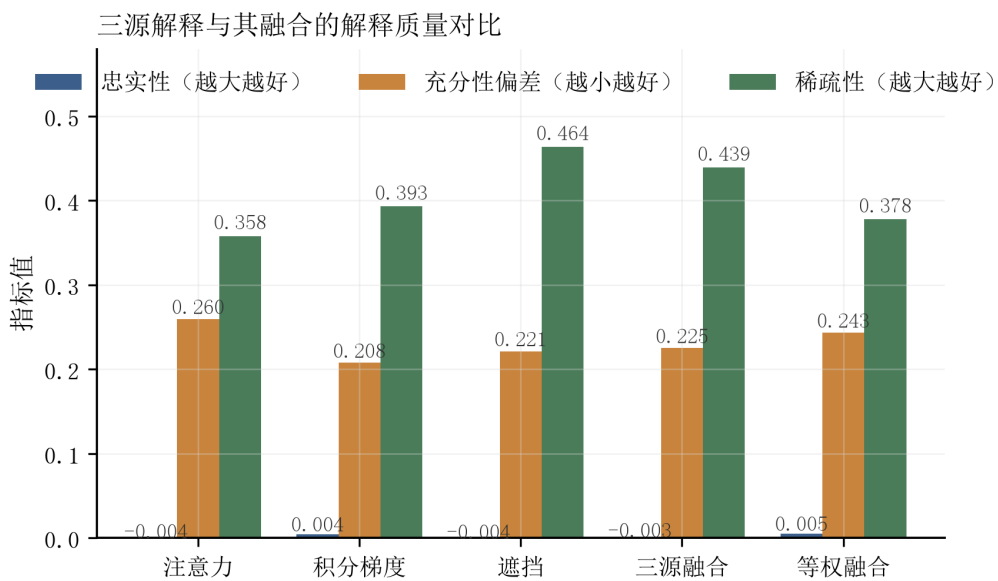


图 36 四种解释来源在附件四上的解释质量对比

表 20 解释质量对比（附件四 20 条与验证集 200 条）

| 解释源      | 忠实性（附件四） | 充分性偏差（附件四） | 稀疏性（附件四） | 充分性偏差（验证集） | 稀疏性（验证集） |
|----------|----------|------------|----------|------------|----------|
| 注意力（基线）  | 0.0143   | 0.2596     | 0.3580   | 0.2332     | 0.4551   |
| 积分梯度     | 0.0171   | 0.2076     | 0.3930   | 0.1972     | 0.4779   |
| 遮挡       | 0.0163   | 0.2213     | 0.4637   | 0.1938     | 0.5611   |
| 三源融合（本文） | 0.0146   | 0.2251     | 0.4392   | 0.1951     | 0.5310   |
| 三源等权（对照） | 0.0124   | 0.2434     | 0.3778   | 0.2066     | 0.4645   |

6.3.4 片段级证据定位与可读性核查

片段级重要性的分布形态见图 37（三行依次为文本、语音与视觉，样本按预测强度升序排列，颜色越深越关键，红色竖线为有效长度上界）。融合解释的 top-5 片段平均承载 0.439（附件四）的模态内重要度（验证集 200 条为 0.531），最高重要度片段的位置中位数为有效长度的 0.32，即证据集中在各模态有效区间的前中部；附件四 60 个模态-样本中有 59 个的 top-5 片段全部落在有效区间内，未出现把填充位当作证据的情形。积分梯度与遮挡两源在片段级的 top-5 平均仅重叠 1.5/5 个片段，说明时间定位本身存在来源依赖，这也印证了证据在时间上较分散的判断。

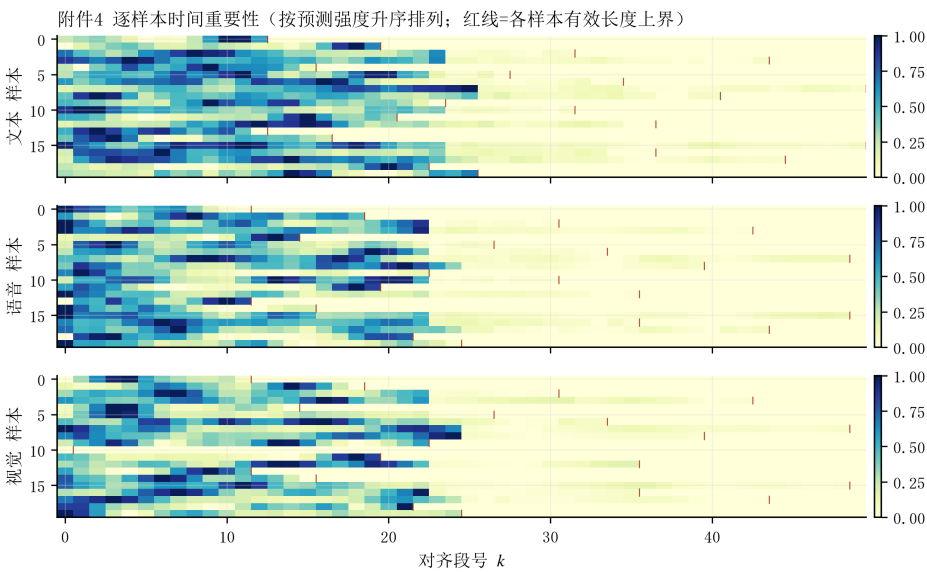


图 37 附件四逐段重要性热力图

为进一步检验证据定位的可读性，本文对附件四 20 条样本逐条判读（判读表见支撑材料 Q3\_ 证据人工判读 \*.csv），文本证据 15/20 承载该样本的情感线索（5 条中性样本为事实性描述，极性样本中 10 条含与预测方向一致的情感、否定或程度线索）；视觉关键帧 19/20 含可辨面部，1 条为房间全景（样本 13）；语音因无听音条件改以能量代理核查，9 条落在高能量或全片峰值段、9 条居中、2 条偏低。图 38 给出关键帧核对的拼图（每格为一条样本的关键帧，上方标注编号与预测强度），将模型导出的关键帧与视频画面逐条对照；未命中集中在寒暄、开场与收尾类语句及远景画面，说明片段级定

位在低信息段会退化为话题匹配，这既支持了模态级结论为主、片段级为辅的取舍，也指出了后续的改进方向。

附件四 关键帧核对表 (19/20 张可用; 标注为 样本号 预测标签 强度 关键帧时刻)

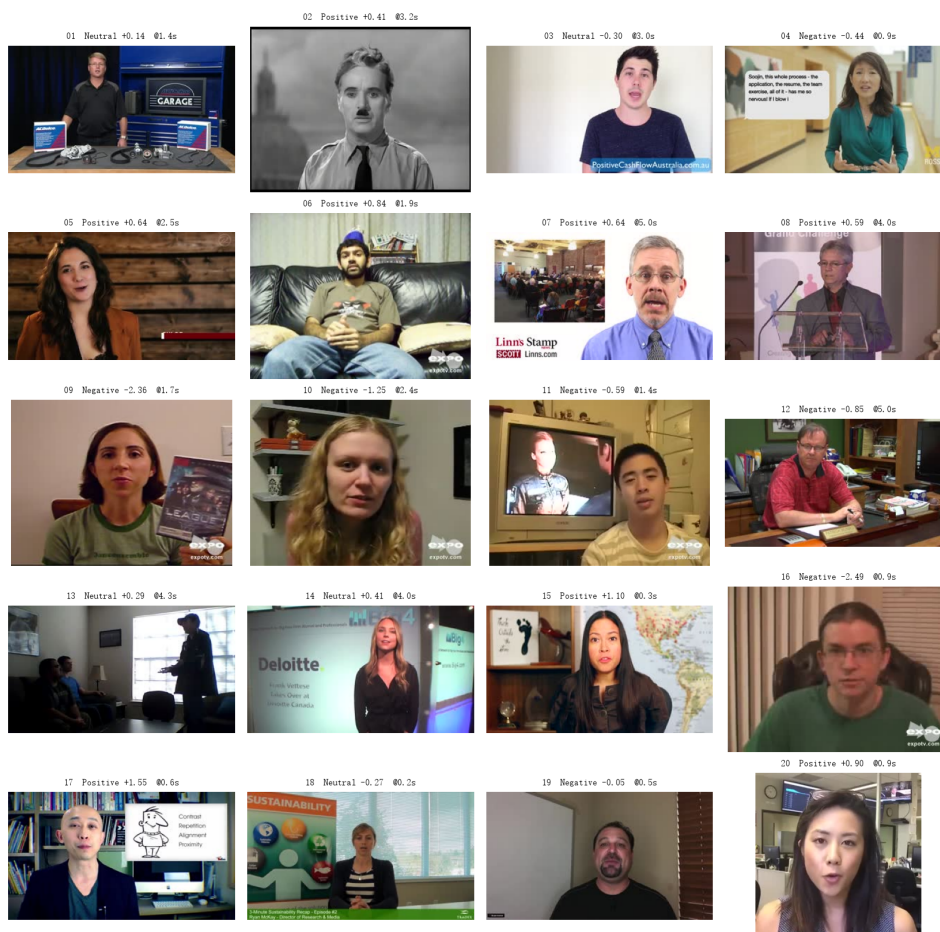


图 38 附件四视觉关键帧核对表 (拼图)

还需指出，上述时间尺度的忠实性指标数值都很小（绝对变化量约 0.0146，而预测强度的标准差为 1.0627），说明本模型的决策证据在时间维度上较为分散、单个片段的替换难以显著改变预测，这与问题二中帧注意力熵 3.86 接近上限 3.91 的实测现象一致。因此本文把模态级解释作为主要的可追溯依据，片段级解释作为辅助定位手段，并在模型改进一节讨论增强时间局部性的方向。解释维度的错误归因如图 39 所示（左、右子图分别为解释熵与置信度），预测正确的 453 条样本其模态作用度的熵均值为 1.0137，预测错误的 275 条为 1.0673，即错误样本的模态依赖更分散、缺少明确的判断依据；两组的平均置信度分别为 0.6771 与 0.5691，说明错误预测通常缺少明显的主导模态；这与问题二中弱强度样本准确率仅 0.5045 的结论一致。



解释维度错误归因（验证集  $n=728$ ，正确 453 / 错误 275）

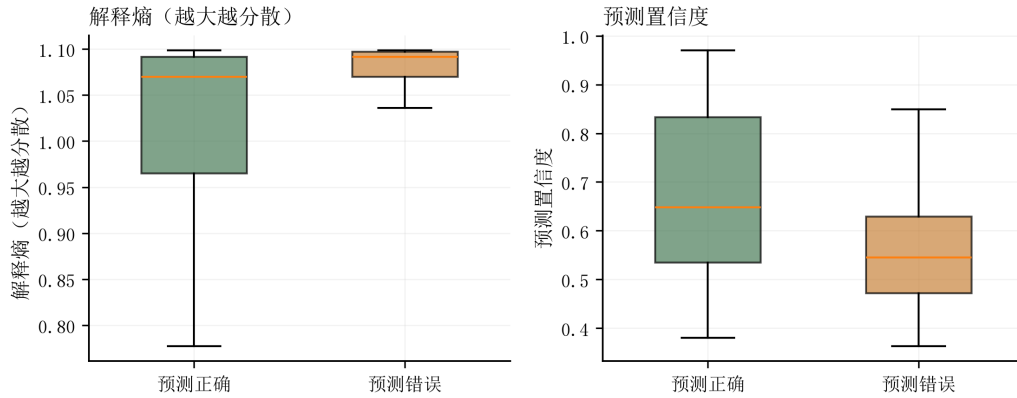


图 39 解释维度的错误归因

### 6.3.5 附件四预测与解释结果汇总

附件四 20 条样本的预测与解释汇总见表 21 与图 40（三个面板依次为极性分布、主参考模态与平均强度/置信度）。极性取分类头主口径，模态作用度取式 (29) 的 softmax 口径。预测分布为负向 7 条、中性 5 条、正向 8 条；文本为主要参考模态 18 条、视觉 2 条、语音 0 条，与验证集上文本主导的作用度结论一致。含缺失样本仅 1 条（样本 13 视觉模态整条缺失），其模态作用度趋于均匀、视觉 Shapley 值近似为零，说明模型在信息缺失时不会依赖该模态。

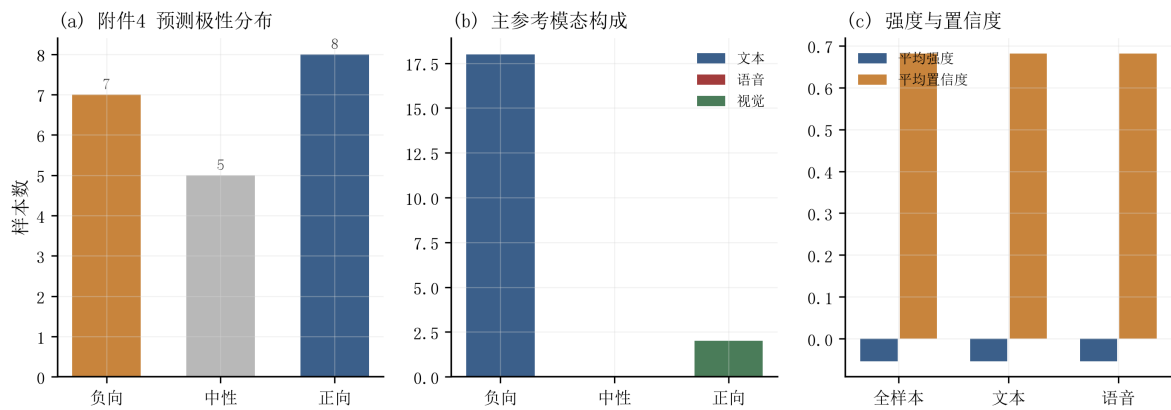


图 40 附件四预测与解释结果汇总

表 21 附件四测试集的预测与解释汇总

| 指标     | 全样本     | 含缺失     | 无缺失     |
|--------|---------|---------|---------|
| 样本数    | 20      | 1       | 19      |
| 平均强度   | -0.0540 | +0.2921 | -0.0722 |
| 平均绝对强度 | +0.8060 | +0.2921 | +0.8331 |
| 平均置信度  | +0.6814 | +0.6379 | +0.6837 |
| 两口径一致率 | 0.9000  | —       | —       |

提交文件 `pred_explain_att4.csv` 逐样本给出五类结果，即情感极性（分类头主口径与阈值口径并列）、情感强度、主要参考模态、三模态作用度（ $\pi_m$  与两套归一化）以及关键证据定位（文本字符区间、语音秒区间、视觉关键帧编号与时刻），并附本样本解释的忠实性、充分性、稳定性与稀疏性指标。与预测文件的一致性在生成时做了断言校验，标签逐位一致、强度分数最大差异小于  $10^{-3}$ 。

## 6.4 本章小结

本章在冻结问题二骨干的基础上搭建了模态级、片段级和证据级三层解释，并完成附件四的全量推理与解释。模态级采用 8 个子集精确枚举 **Shapley** 值（残差小于  $10^{-15}$ ），片段级结合积分梯度和模型无关遮挡，证据级将重要区间映射回文本、语音和视觉；三源融合权重为  $\gamma = (0.1, 0.1, 0.8)$ ，与遮挡基准的一致性达到 0.9561，而注意力源与遮挡源仅为 -0.0268 量级的负相关，说明直接以注意力作解释不成立。

验证集结果显示，文本平均作用度为 0.4620，高于语音 0.2627 和视觉 0.2754，情感越明确文本依据越集中（作用度熵 1.0723 降至 0.9452）。模态级排序与问题二缺失实验的秩相关为 1.000，附件三免标注检验相关系数为 -0.4011。片段级证据大多落在有效区间内（附件四文本证据 15/20 承载情感线索、关键帧 19/20 含可辨面部），但不同解释方法对具体时间位置仍存在一定差异，因此本文将模态级解释作为主要可追溯依据。

本章的限制主要有两点。一是时间片段级的忠实性数值较小（绝对变化量约 0.0146，预测标准差 1.0627），说明证据在时间上较分散；二是附件四只有 1 条含缺失样本，免标注检验的样本量不足。上述局限与改进方向一并列于第 7 章。

# 七、模型评价与推广

## 7.1 模型的优点

我们的模型把缺失当作可观测的结构显式纳入，填充掩码、缺失掩码、共享-特有分解和动态门控都有明确的数学含义，能够对应到具体的数据问题。训练和评价口径也



保持一致，模型结构、超参数、阈值和校准系数只在验证集上选择，专项测试集不参与调参；缺失率沿用全文统一定义并与提交文件逐位对齐，分类主口径与阈值口径并列报告。

问题二与问题三从不同途径得到相同的结论，缺失注入实验表明文本模态主导性能退化，删模态反事实与 Shapley 分析得到相同排序（秩相关 1.000）；解释结果又通过附件三实测缺失位置进行免标注检验，并通过注意力不等于重要性的外部基准校验。相比只报告主指标，我们还给出了缺失场景扫描、消融、方差分析、错误归因与拒识分析，使模型的有效范围与不足都可以量化，并同样报告两条经验证无效的改进路径，避免夸大收益。

## 7.2 模型的缺点

当前模型最明显的问题是中性类边界识别不足，中性类精确率仅约 0.43；连续强度预测也存在向零收缩，负类与正类的平均偏差分别为 +0.541 与 -0.447。部分消融差异小于不同随机种子之间的波动，因此这些模块的独立增益仍需要更多重复实验验证。

另外，真实缺失样本数量有限，验证集中含缺失样本仅 48 条，附件四只有 1 条含缺失样本，且附件三、附件四都没有文本模态实测缺失，这使文本鲁棒性和部分解释结论仍主要依赖模拟实验。片段级解释的忠实性数值较小，说明模型的决策证据在时间上较为分散，片段级定位在低信息段还会退化为话题匹配。

## 7.3 模型改进与推广

针对中性边界问题，我们尝试过两条低成本路径，结果均未奏效。一条路径是用已有有序回归头的期望替代点回归，在验证集与测试划分上均未改善误差；另一条是用不确定度与强度阈值对中性边界作后处理重判定，八个规则族中仅一个通过判据但只改动一条样本，另一个在验证集上改善 0.0054 而在测试划分上反向恶化 0.0139。这表明瓶颈更多出在表征层面，后续应优先增强否定作用域、转折结构与语用信息的建模；阈值后处理的空间有限，不宜继续复杂化。针对强度收缩，可考虑直接建模带符号的序数分布，使模型同时输出强度层级和不确定度；针对片段级解释分散，可在解释头中加入片段对比约束或稀疏时间窗约束，提高关键证据的局部集中性。

从方法结构看，填充与缺失严格分离、掩码显式感知、训练期缺失注入、独立解释校验的方法框架不依赖情感预测本身，可直接迁移至视频检索、行为识别和工业多传感器时序分析等其他多模态序列任务，只要数据具备等长组织、尾部填充、局部不可用的共性，该框架即可复用；其中缺失口径的定义、课程注入比例与解释一致性标定，是需要按域重新校准的三个关键环节，不宜直接照搬本题参数。

## 参考文献

- [1] Zadeh A, Liang P P, Poria S, et al. Multimodal language analysis in the wild: CMU-MOSEI dataset and interpretable dynamic fusion graph[C]. Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, 2018: 2236–2246.
- [2] Tsai Y H H, Bai S, Liang P P, et al. Multimodal transformer for unaligned multimodal language sequences[C]. Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 2019: 6558–6569.
- [3] Cao J, Miranda B, Wang X, et al. A deep multi-task learning model with multiscale feature fusion for multimodal sentiment analysis[J]. Information Fusion, 2023, 91: 232–243.
- [4] Cao W, Mirjalili V, Raschka S. Rank consistent ordinal regression for neural networks with application to age estimation[J]. Pattern Recognition Letters, 2020, 140: 325–331.
- [5] Hinton G, Vinyals O, Dean J. Distilling the knowledge in a neural network[J]. arXiv preprint arXiv:1503.02531, 2015.
- [6] Bengio Y, Louradour J, Collobert R, et al. Curriculum learning[C]. Proceedings of the 26th Annual International Conference on Machine Learning, 2009: 41–48.
- [7] Geifman Y, El-Yaniv R. Selective classification for deep neural networks[C]. Advances in Neural Information Processing Systems, 2017: 4878–4887.
- [8] Kull M, Silva Filho T, Flach P. Beta calibration: a well-founded and easily implemented improvement on logistic calibration for binary classifiers[C]. Artificial Intelligence and Statistics, 2017: 623–631.
- [9] Zhang S, Yang Y, Chen C, et al. Deep learning-based multimodal emotion recognition from audio, visual, and text modalities: A systematic review[J]. Expert Systems with Applications, 2024, 237: 121692.
- [10] Qiu X, Fang Y, Zhou Q, et al. Beyond missing modalities: Hypergraph conditioned diffusion for uncertainty-aware multimodal emotion recognition[C]. CVPR, 2026.
- [11] Yang Z, Li M. Factorize, reconstruct, enhance: A unified framework for multimodal sentiment analysis[C]. CVPR, 2026.
- [12] Zhuang Y, Liu M, Bai W, et al. CMAD: Correlation-aware and modalities-aware distillation for multimodal sentiment analysis with missing modalities[C]. ICCV, 2025.

- [13] Zhu A, Hu M, Wang X, et al. Proxy-driven robust multimodal sentiment analysis with incomplete data[C]. ACL, 2025.
- [14] Mai S, Han S. Learning invariant modality representation for robust multimodal learning from a causal inference perspective[C]. ACL, 2026.
- [15] Wan J, Gong C, Fu G. Locate and explain: Joint multimodal emotion cause extraction and summarization in conversation[C]. ACL, 2026.
- [16] Mai S, Han S. CaReFlow: Cyclic adaptive rectified flow for multimodal fusion[C]. CVPR, 2026.
- [17] Fang Y, Huang W, Wan G, et al. EMOE: Modality-specific enhanced dynamic emotion experts[C]. CVPR, 2025.
- [18] 王楠, 王淇, 欧阳丹彤. 基于知识蒸馏与动态调整机制的多模态情感分析模型 [J]. 计算机学报, 2025, 48(8): 1923–1942.

## A 支撑材料文件清单

本次提交的全部支撑材料按类别汇总于表 22（按提交文件、模型参数、结果表、核心代码与运行说明五类列出），文件命名与论文正文、代码中的口径保持一致。问题一的 100 条样本特征文件、处理日志与复现说明待问题一完成后一并提交。

**表 22 提交材料清单**

| 类别   | 文件                                         | 说明                         |
|------|--------------------------------------------|----------------------------|
| 提交文件 | submission/pred_att3.csv                   | 附件三 30 条预测结果               |
| 提交文件 | submission/pred_att4.csv                   | 附件四 20 条预测结果               |
| 提交文件 | submission/pred_explain_att4.csv           | 附件四预测与解释结果（含模态作用度与证据定位）    |
| 提交文件 | submission/explain_cards/*.json            | 逐样本解释卡（含 50 步三源与融合时间重要性）   |
| 提交文件 | submission/evidence/*.jpg                  | 视觉关键帧（用于回看原始视频）            |
| 模型参数 | runs/ens_top2/student_s42.pt               | 集成成员一（种子 42）               |
| 模型参数 | runs/ens_top2/student_s43.pt               | 集成成员二（种子 43）               |
| 模型参数 | runs/q3/q3_evidence_head.pt                | 可解释性证据头（骨干冻结）              |
| 结果表  | paper_tables/main_results_summary.csv      | 主结果多种子汇总                   |
| 结果表  | paper_tables/ablation_paired.csv           | 同种子配对消融汇总                  |
| 结果表  | paper_tables/missing_low_anova_summary.csv | 方差分析效应量                    |
| 结果表  | paper_tables/error_attribution_groups.csv  | 错误归因分层                     |
| 结果表  | paper_tables/Q3_*.csv                      | 问题三分层作用度、八子集反事实、模态忠实性与解释质量 |
| 核心代码 | code/02_model/model.py                     | 网络结构与门控                    |
| 核心代码 | code/02_model/losses.py                    | 损失函数与指标                    |
| 核心代码 | code/02_model/train.py                     | 训练与推理主流程                   |
| 核心代码 | code/03_xai/*.py                           | 问题三三源解释与证据定位               |
| 运行说明 | code/README.md                             | 环境依赖与运行顺序                  |

## B 不确定性与拒识能力分析（补充实验）

本节为问题二的可选补充实验，不属于题目硬性要求，正文仅保留结论。

模型在式 (11) 中已输出预测方差，据此可定义两类不确定度。偶然不确定度取对数方差头输出的指数：

$$\sigma_{\text{model}} = \exp\left(\frac{\ln \sigma^2}{2}\right), \quad (33)$$

认知不确定度由集成成员的分歧给出：

$$\sigma_{\text{ens}} = \sqrt{\frac{1}{K-1} \sum_{k=1}^K (\mu_k - \bar{\mu})^2}, \quad \sigma_{\text{total}} = \sqrt{\sigma_{\text{model}}^2 + \sigma_{\text{ens}}^2}. \quad (34)$$

另一类与极性判别相关的不确定度取分类头概率熵：

$$H = - \sum_{k=0}^2 \pi_k \ln \pi_k. \quad (35)$$

按不确定度升序保留前  $q$  比例样本，可得风险—覆盖率曲线，其积分即 AURC[7]：

$$\text{AURC} = \int_0^1 \text{MAE}_q \, dq. \quad (36)$$

两类不确定度对分类准确率的影响方向并不一致，按总不确定度剔除会使准确率下降，按分类头概率熵剔除则明显上升。就强度回归而言，剔除最不确定的 10% 样本后平均绝对误差由 0.5866 降至 0.5465，剔除一半后降至 0.4495；风险—覆盖率曲线的曲线下面积（AURC）为 0.4365（随机剔除对应 0.5866、理论下界 0.2258）；总不确定度与绝对误差的斯皮尔曼相关系数为 0.296（验证集）与 0.230（测试划分），均为正。就极性判别而言，按总不确定度剔除的曲线下面积 0.3980 甚至劣于随机的 0.3777，改用分类头概率熵后降至 0.2252。相关曲线见 5.4.5 节的图 21 至图 23。

用单一不确定度同时拒识回归与分类，在本任务上并不成立，原因在于总不确定度与真值强度单调同增（五分位分桶后最确定一桶的真值绝对强度均值 0.411、最不确定一桶 1.233），实质追踪的是强度幅值；分类头概率熵则与强度递减，追踪的是极性模糊度。据此本文把两类不确定度分别用于强度拒识与极性拒识。

## C 核心代码

本节给出问题二与问题三模型的核心代码，与提交的支撑材料中的 `code/` 目录为同一份；问题二的代码对应正文 5.3 节，问题三的代码对应正文第 6 章。代码以 Python 3.12 与 PyTorch 编写，运行顺序见 `code/README.md`。为便于与提交材料核对，每个代码块标注其在支撑材料中的相对路径与总行数。为避免中文字体下缺字形，代码中的少量 Unicode 数学符号（如  $\in$ 、 $\leq$ 、 $\rightarrow$ 、 $\pi$ ）在附录中以 ASCII 等价形式给出（如 `in`、`<=`、`->`、`pi`），仅涉及注释、文档字符串与输出文案，不影响可执行语义；原始文件以提交的

code/ 目录为准。问题一的特征提取与时序对齐脚本及其复现说明随问题一材料一并提交（见表 22 备注）。

### C.1 问题二：MRF-Net 网络结构与动态专家门控

code/02\_model/model.py（共 285 行）

```
1  # -*- coding: utf-8 -*-
2  """
3  MRF-Net: Missing-Robust Fusion Network
4  掩码感知编码 -> 跨模态重建 -> 代理令牌门控 -> 共享/特有分解 -> 动态专家融合 -> 不确定性双头
5  """
6  import math
7
8  import torch
9  import torch.nn as nn
10 import torch.nn.functional as F
11
12 MODALITIES = ("text", "audio", "vision")
13
14
15 class PositionalEncoding(nn.Module):
16     def __init__(self, d, max_len=2048):
17         super().__init__()
18         pe = torch.zeros(max_len, d)
19         pos = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
20         n_even = pe[:, 0::2].shape[1]
21         div = torch.exp(torch.arange(0, n_even, dtype=torch.float) * (-math.log(10000.0)
22 / d))
23         pe[:, 0::2] = torch.sin(pos * div)
24         pe[:, 1::2] = torch.cos(pos * div[:, :pe[:, 1::2].shape[1]])
25         self.register_buffer("pe", pe.unsqueeze(0))
26
27     def forward(self, x):
28         return x + self.pe[:, :x.size(1)]
29
30 class ModalityEncoder(nn.Module):
31     """掩码感知编码器：显式把“这一段是瞎的”喂给网络。"""
32
33     def __init__(self, d_in, hidden, nhead, layers, dropout, down=1):
34         super().__init__()
35         # 逐时间步线性投影（比大核卷积省 5 倍算力）+ 轻量局部混合
36         self.inp = nn.Sequential(nn.Linear(d_in + 2, hidden), nn.GELU())
37         self.local = nn.Conv1d(hidden, hidden, kernel_size=3, padding=1)
38         self.pos = PositionalEncoding(hidden)
39         layer = nn.TransformerEncoderLayer(
```

```

40         hidden, nhead, dim_feedforward=hidden * 4,
41         dropout=dropout, activation="gelu", batch_first=True, norm_first=True)
42     self.enc = nn.TransformerEncoder(layer, num_layers=layers, enable_nested_tensor=
False)
43     self.down = None
44     if down > 1:
45         self.down = nn.Conv1d(hidden, hidden, kernel_size=down, stride=down)
46
47     def forward(self, x, miss, pad):
48         m = miss.float().unsqueeze(-1)          # 1=缺失（显式告知网络该段不可用）
49         v = 1.0 - m
50         h = self.inp(torch.cat([x * v, m, v], dim=-1))
51         h = h + F.gelu(self.local(h.transpose(1, 2))).transpose(1, 2)
52         h = self.enc(self.pos(h), src_key_padding_mask=pad)
53         return h                                # 原生序列长度
54
55     def downsample(self, h, miss, pad):
56         if self.down is None:
57             return h, miss, pad
58         h2 = self.down(h.transpose(1, 2)).transpose(1, 2)
59         k, s = self.down.kernel_size[0], self.down.stride[0]
60         md = F.max_pool1d(miss.unsqueeze(1).float(), k, s).squeeze(1).bool()
61         pd = F.max_pool1d(pad.unsqueeze(1).float(), k, s).squeeze(1).bool()
62         return h2, md, pd
63
64
65     class Expert(nn.Module):
66         """跨模态 + 跨时间自注意力专家。"""
67
68     def __init__(self, hidden, nhead, dropout):
69         super().__init__()
70         self.attn = nn.MultiheadAttention(hidden, nhead, dropout=dropout, batch_first=
True)
71         self.norm1 = nn.LayerNorm(hidden)
72         self.norm2 = nn.LayerNorm(hidden)
73         self.ffn = nn.Sequential(
74             nn.Linear(hidden, hidden * 2), nn.GELU(), nn.Dropout(dropout),
75             nn.Linear(hidden * 2, hidden))
76
77     def forward(self, tok, pad):
78         a, _ = self.attn(tok, tok, tok, key_padding_mask=pad)
79         x = self.norm1(tok + a)
80         x = self.norm2(x + self.ffn(x))
81         return x
82
83

```

```

84 class MRFNet(nn.Module):
85     def __init__(self, dims, hidden=128, nhead=4, enc_layers=2, n_experts=4,
86                 dropout=0.1, downs=None, target_len=50, mag_levels=0,
87                 ms_pool=False, ms_windows=(5, 25, 50), use_pol=False):
88         super().__init__()
89         downs = downs or {"text": 1, "audio": 1, "vision": 1}
90         self.dims = dims
91         self.hidden = hidden
92         self.target_len = target_len
93         self.mag_levels = int(mag_levels)
94         self.ms_pool = bool(ms_pool)
95         self.ms_windows = tuple(int(w) for w in ms_windows)
96         self.use_pol = bool(use_pol)
97         self.mods = list(MODALITIES)
98
99         self.enc = nn.ModuleDict({
100             m: ModalityEncoder(dims[m], hidden, nhead, enc_layers, dropout, downs[m])
101             for m in self.mods})
102
103         # —— 跨模态重建（缺失位置靠“造”） ——
104         self.cross = nn.ModuleDict({m: nn.MultiheadAttention(hidden, nhead, dropout=
105 dropout, batch_first=True) for m in self.mods})
106         self.dec = nn.ModuleDict({m: nn.Linear(hidden, dims[m]) for m in self.mods})
107
108         # —— 代理令牌 ——
109         self.proxy = nn.ParameterDict({m: nn.Parameter(torch.randn(hidden) * 0.02) for m
110 in self.mods})
111         self.beta = nn.ModuleDict({m: nn.Linear(hidden + 1, 1) for m in self.mods})
112
113         # —— 共享 / 特有分解 ——
114         self.share = nn.ModuleDict({m: nn.Linear(hidden, hidden) for m in self.mods})
115         self.spec = nn.ModuleDict({m: nn.Linear(hidden, hidden) for m in self.mods})
116         self.mix = nn.ModuleDict({m: nn.Linear(hidden * 2, hidden) for m in self.mods})
117
118         # —— 动态专家融合 ——
119         self.mod_emb = nn.Embedding(3, hidden)
120         self.experts = nn.ModuleList([Expert(hidden, nhead, dropout) for _ in range(
121 n_experts)])
122         self.gate = nn.Sequential(nn.Linear(hidden + 7, hidden), nn.GELU(), nn.Linear(
123 hidden, n_experts))
124         self.pool_score = nn.Sequential(nn.Linear(hidden, hidden // 2), nn.Tanh(), nn.
125 Linear(hidden // 2, 1))
126
127         # —— 输出头 ——
128         self.head_mu = nn.Sequential(nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(
129 hidden, 1))

```



```

124         self.head_lv = nn.Sequential(nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(
125             hidden, 1))
126
127         self.head_cls = nn.Sequential(nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(
128             hidden, 3))
129
130     # —— 强度分级辅助头（CORAL 有序回归：K-1 个累计 logit） ——
131     # 动机（实测）：61.7% 的样本  $|y| \leq 2/3$ （弱标注）而 ACC 仅 0.486； $|y|=1/3$  层
132     # ACC 更低至 0.361 —— 模型分不清“完全中性”与“轻微极性”。该头把  $|y|$  离散为
133     # K 个等级（步长 1/3）做有序回归，强制模型在零点附近建立分辨率。
134     if self.mag_levels > 1:
135         self.mag_w = nn.Linear(hidden, 1, bias=False)
136         self.mag_b = nn.Parameter(torch.linspace(-1.5, 1.5, self.mag_levels - 1))
137
138     # —— 多尺度时间池化（补足单层注意力池化“过于均匀”的问题） ——
139     # 实测：帧注意力熵 3.86 / 上限 3.91  $\approx$  均匀，说明注意力没聚焦到关键片段。
140     # 改为“多窗口掩码均值池化 + 线性投影”，并与注意力池化残差相加。
141     if self.ms_pool:
142         T = 3 * int(target_len)          # 三模态拼接后的 token 长度
143         nseg = sum(max(1, T // w) for w in self.ms_windows)
144         self._ms_nseg = nseg
145         self._ms_T = T
146         self.ms_proj = nn.Linear(nseg * hidden, hidden)
147
148     # —— 二分类极性辅助头（对齐文献 Acc-2 的做法） ——
149     if self.use_pol:
150         self.head_pol = nn.Linear(hidden, 1)
151
152     # -----
153     def _masked_win_pool(self, h, pad):
154         """多窗口掩码均值池化：把时间轴切成若干段，段内对非填充位置求均值。
155
156         h:(B,T,H) pad:(B,T) bool(True=填充) 返回 (B, nseg*H)
157         """
158         h = h.masked_fill(pad.unsqueeze(-1), 0.0)
159         valid = (~pad).float().unsqueeze(-1)          # (B,T,1)
160         T = h.size(1)
161         outs = []
162         for w in self.ms_windows:
163             nb = max(1, T // w)
164             seg = torch.arange(T, device=h.device) * nb // T    # 把 T 均分到 nb 段
165             for b in range(nb):
166                 m = (seg == b).float().unsqueeze(0).unsqueeze(-1)    # (1,T,1)
167                 s = (h * m * valid).sum(dim=1)
168                 n = (m * valid).sum(dim=1).clamp(min=1.0)
169                 outs.append(s / n)
170         return torch.cat(outs, dim=-1)

```

```

168
169 @staticmethod
170 def _availability(rho):
171     """rho:(B,3) 缺失率 -> (B,7) 可用性特征。"""
172     n_vis = (rho < 0.999).float().sum(dim=1, keepdim=True)
173     return torch.cat([rho, (rho < 0.999).float(), n_vis / 3.0], dim=1)
174
175 def forward(self, x, miss, pad, return_aux=False):
176     """x/miss/pad: dict 模态 -> (B,L,d)/(B,L)/(B,L)"""
177     h, rec, share, spec = {}, {}, {}, {}
178     for m in self.mods:
179         h[m] = self.enc[m](x[m], miss[m], pad[m])
180
181     # --- 1) 跨模态重建 (原生长度上计算, 损失只取缺失位置) ---
182     for m in self.mods:
183         others = [h[o] for o in self.mods if o != m]
184         kv = torch.cat(others, dim=1)
185         kv_pad = torch.cat([pad[o] for o in self.mods if o != m], dim=1)
186         a, _ = self.cross[m](h[m], kv, kv, key_padding_mask=kv_pad)
187         h_hat = h[m] + a * miss[m].float().unsqueeze(-1)
188         rec[m] = self.dec[m](h_hat)
189
190     # --- 2) 降采样到 target_len 统一时间尺度 ---
191     hd, md, pd_ = {}, {}, {}
192     for m in self.mods:
193         hd[m], md[m], pd_[m] = self.enc[m].downsample(h[m], miss[m], pad[m])
194
195     # --- 3) 代理令牌门控 + 分解 ---
196     rho = torch.stack([md[m].float().mean(dim=1) for m in self.mods], dim=1) # (B,3)
197     beta_val = {}
198     for m in self.mods:
199         hm = hd[m]
200         inp = torch.cat([hm.mean(dim=1), rho[:, self.mods.index(m)].unsqueeze(-1)],
201 dim=-1)
202         b = torch.sigmoid(self.beta[m](inp)).unsqueeze(1) # (B,1,1)
203         beta_val[m] = b.detach().view(-1)
204         hm = (1 - b) * hm + b * self.proxy[m].view(1, 1, -1)
205         share[m] = self.share[m](hm)
206         spec[m] = self.spec[m](hm)
207         hd[m] = self.mix[m](torch.cat([share[m], spec[m]], dim=-1))
208
209     # --- 4) 动态专家融合 ---
210     tokens = torch.cat([hd[m] for m in self.mods], dim=1)
211     tok_pad = torch.cat([pd_[m] for m in self.mods], dim=1)
212     mid = torch.arange(3, device=tokens.device).repeat_interleave(tokens.size(1) //
3)

```

```

212     tokens = tokens + self.mod_emb(mid).unsqueeze(0)
213
214     av = self._availability(rho)
215     cls_pool = tokens.mean(dim=1)
216     g = torch.softmax(self.gate(torch.cat([cls_pool, av], dim=-1)), dim=-1)      # (B,E)
217
218     outs = torch.stack([e(tokens, tok_pad) for e in self.experts], dim=1)      # (B,E,
T,H)
219     fused = (g.unsqueeze(-1).unsqueeze(-1) * outs).sum(dim=1)                # (B,T,H
)
220
221     score = self.pool_score(fused).masked_fill(tok_pad.unsqueeze(-1), -1e9)
222     alpha = torch.softmax(score, dim=1)                                       # (B,T
,1)
223     z = (alpha * fused).sum(dim=1)
224     if self.ms_pool:
225         pooled = self._masked_win_pool(fused, tok_pad)
226         z = z + self.ms_proj(pooled)
227
228     mu = self.head_mu(z).squeeze(-1)
229     logvar = self.head_lv(z).squeeze(-1).clamp(-6.0, 4.0)
230     logits = self.head_cls(z)
231
232     if not return_aux:
233         return mu, logvar, logits
234     aux = dict(rec=rec, share=share, spec=spec, gate=g, alpha=alpha,
235               rho=rho, hd=hd, pad_ds=pd_, miss_ds=md, z=z, mu=mu, logits=logits,
236               beta=beta_val)
237     if self.mag_levels > 1:
238         aux["logits_mag"] = self.mag_w(z) + self.mag_b.view(1, -1)
239     if self.use_pol:
240         aux["pol"] = self.head_pol(z).squeeze(-1)
241     return mu, logvar, logits, aux
242
243
244 class PlainFusion(nn.Module):
245     """A0/A1 基线: **朴素拼接 + MLP** 融合（无掩码感知编码/重建/代理令牌/共享-特有分解/动态专家）。
246
247     用于论文消融中的“常规固定权重融合模型”对照：
248     * “use_indicator=False“ -> **A0**（完全不做缺失处理的常规模型）
249     * “use_indicator=True“ -> **A1**（仅额外告知“哪段缺失”）
250
251     接口与 :class:‘MRFNet‘ 完全一致（“forward“ 返回 “mu/logvar/logits[+aux]“），
252     因此可直接复用 train/runtime/infer 的全部代码。
253     """
254

```

```

255 def __init__(self, dims, hidden=128, nhead=4, enc_layers=2, n_experts=4,
256             dropout=0.1, downs=None, target_len=50, use_indicator=True):
257     super().__init__()
258     self.mods = list(MODALITIES)
259     self.use_indicator = bool(use_indicator)
260     d_in = sum(int(dims[m]) for m in self.mods)
261     if self.use_indicator:
262         d_in += 2 * len(self.mods)          # 每模态: 缺失率 + 可见率
263     self.net = nn.Sequential(
264         nn.Linear(d_in, hidden), nn.GELU(), nn.Dropout(dropout),
265         nn.Linear(hidden, hidden), nn.GELU())
266     self.head_mu = nn.Sequential(nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(
267         hidden, 1))
268     self.head_lv = nn.Sequential(nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(
269         hidden, 1))
270     self.head_cls = nn.Sequential(nn.Linear(hidden, hidden), nn.GELU(), nn.Linear(
271         hidden, 3))
272
273 def forward(self, x, miss, pad, return_aux=False):
274     feats = []
275     for m in self.mods:
276         v = (~pad[m]).float().unsqueeze(-1)          # (B,L,1)
277         pooled = (x[m] * v).sum(dim=1) / v.sum(dim=1).clamp(min=1.0)  # 有效位均值池化
278         feats.append(pooled)
279         if self.use_indicator:
280             mval = (miss[m] & ~pad[m]).float().mean(dim=1, keepdim=True)
281             feats += [mval, 1.0 - mval]
282     h = self.net(torch.cat(feats, dim=-1))
283     mu = self.head_mu(h).squeeze(-1)
284     logvar = self.head_lv(h).squeeze(-1).clamp(-6.0, 4.0)
285     logits = self.head_cls(h)
286     if not return_aux:
287         return mu, logvar, logits
288     return mu, logvar, logits, dict(z=h)

```

## C.2 问题二：损失函数（异方差回归、类别加权、一致性、重建与蒸馏）

code/02\_model/losses.py（共 375 行）

```

1  -*- coding: utf-8 -*-
2  """损失函数与评价指标。"""
3  import numpy as np
4  import torch
5  import torch.nn.functional as F
6
7  MODALITIES = ("text", "audio", "vision")

```

```

8
9
10 # -----
11 # 损失
12 # -----
13 def reg_nll(mu, logvar, y):
14     """高斯负对数似然（仅作不确定性辅助项）。"""
15     return 0.5 * (torch.exp(-logvar) * (y - mu) ** 2 + logvar).mean()
16
17
18 def reg_main(mu, y, beta=0.5):
19     """主回归损失：SmoothL1，与官方评价指标 MAE 对齐。"""
20     return F.smooth_l1_loss(mu, y, beta=beta)
21
22
23 def cls_ce(logits, y, weight=None, label_smoothing=0.0):
24     return F.cross_entropy(logits, y, weight=weight, label_smoothing=label_smoothing)
25
26
27 # -----
28 # D 系列：针对“弱标注样本”（|y| 小 / 中性类）的标签分布与有序损失
29 # 实测（附件2）：|y| 以 1/3 为步长、最大 3.0，且 类 = sign(均值强度)。
30 # 因此 |y| 本身就是“标注一致性”的直接度量：|y|=1/3 相当于标注者 2:1 分裂。
31 # 三类混淆中 84% 只是邻类错，说明需要**序信息**而不是更多容量。
32 # -----
33 def intensity_soft_target(y_reg, tau=0.3):
34     """用连续强度 |y| 构造三分类**软目标**（标签分布 / 置信度感知平滑）。
35
36     q_极性 = tanh(|y|/tau), q_中性 = 1 - q_极性, y=0 时 q = 纯中性
37
38     tau 越小越接近硬标签；tau->inf 退化为“只区分中性/非中性”。
39     """
40     y = y_reg
41     conf = torch.tanh(torch.abs(y) / max(float(tau), 1e-6)).clamp(0.0, 1.0)
42     q = torch.zeros(y.shape[0], 3, dtype=y.dtype, device=y.device)
43     pos = y > 0
44     neg = y < 0
45     q[:, 1] = 1.0 - conf
46     q[pos, 2] = conf[pos]
47     q[neg, 0] = conf[neg]
48     q[~(pos | neg), 1] = 1.0
49     return q / q.sum(dim=-1, keepdim=True).clamp(min=1e-8)
50
51
52 def emd2_loss(logits, q):
53     """EMD^2（有序）惩罚：累计分布的平方差，使“跨类错误”代价 > “邻类错误”。"""

```

```

54     p = torch.softmax(logits, dim=-1)
55     return ((torch.cumsum(p, dim=-1) - torch.cumsum(q, dim=-1)) ** 2).sum(-1).mean()
56
57
58 def neu_suppress_loss(logits, y_cls, lam=0.0):
59     """中性抑制正则（代价敏感学习）：惩罚”真值为极性、却把概率压在 neu 上”。
60
61     实测误差结构：valid 上 299 个错误里 **167 个（56%）是把极性样本判成中性**
62     （预测为 neu 的 269 个样本里只有 102 个正确，neu precision 仅 0.36）。
63     该正则直接把 p_neu 从极性样本上挤走，把”中性<->极性”错误转化为”邻类极性”错误。
64     """
65     if not lam or lam <= 0:
66         return None
67     p = torch.softmax(logits, dim=-1)
68     m = y_cls != 1
69     if m.sum() < 1:
70         return None
71     return p[m, 1].mean()
72
73
74 # -----
75 # D4: 强度分级辅助头（CORAL 有序回归）
76 # 动机（实测）：61.7% 的样本 |y|<=2/3，其 ACC 仅 0.486；|y|=1/3 层更低到 0.361。
77 # 误差 84% 是邻类错 —— 模型分不清”完全中性”与”轻微极性”。把 |y| 按 1/3 步长
78 # 离散为 K 个等级做有序回归，等于直接监督“强度分辨率”，而不是让分类头自己猜。
79 # -----
80 def mag_level(y_reg, n_levels=10):
81     """|y| -> 等级 k in [0, n_levels-1]（|y| 的离散档位，步长 1/3 -> n_levels=10）。"""
82     k = torch.round(torch.abs(y_reg) * 3.0).long()
83     return k.clamp(0, int(n_levels) - 1)
84
85
86 def coral_loss(logits_mag, y_reg, n_levels=10):
87     """CORAL：共享投影 + 有序偏置，对 K-1 个累计事件 P(k>j) 做二分类。
88
89     logits_mag: (B, n_levels-1)
90     """
91     k = mag_level(y_reg, n_levels)
92     levels = torch.arange(logits_mag.size(1), device=k.device).unsqueeze(0)
93     target = (k.unsqueeze(1) > levels).float()
94     return F.binary_cross_entropy_with_logits(logits_mag, target)
95
96
97 def mag_consistency(mu, logits_mag, n_levels=10):
98     """辅助一致性：累计概率得到的期望等级应与 |mu| 对得上（弱监督，可选）。"""
99     p = torch.sigmoid(logits_mag) # P(k > j)

```

```

100     exp_k = p.sum(dim=1)                                # E[k] = Sigma_j P(k>j)
101     return F.smooth_l1_loss(exp_k / 3.0, torch.abs(mu).detach(), beta=0.2)
102
103
104 def cls_D_loss(logits, y_cls, y_reg, cls_weight=None, label_smoothing=0.0,
105                 soft_tau=0.0, soft_mix=0.0, lam_emd=0.0, weak_alpha=0.0,
106                 lam_neusup=0.0):
107     """D 系列合成分类损失（针对弱标注样本 / 中性类）。
108
109     D2 soft_mix>0: 硬 CE 与强度软目标 KL 的凸组合
110     D2 lam_emd>0: 有序 EMD^2 项（内建序结构；实测 84% 错误只是邻类错）
111     D3 weak_alpha>0: 按  $w = 1 + \alpha \cdot (1 - |y|/3)$  给**弱标注样本**加权
112     D3' lam_neusup>0: 中性抑制正则（代价敏感，专治“极性样本被判成中性”）
113     """
114     hard = F.cross_entropy(logits, y_cls, weight=cls_weight,
115                             reduction="none", label_smoothing=label_smoothing)
116     q = None
117     if soft_mix and soft_mix > 0:
118         q = intensity_soft_target(y_reg, tau=soft_tau if soft_tau > 0 else 0.3)
119         soft = -(q * F.log_softmax(logits, dim=-1)).sum(-1)
120         per = (1.0 - float(soft_mix)) * hard + float(soft_mix) * soft
121     else:
122         per = hard
123     if weak_alpha and weak_alpha > 0:
124         w = 1.0 + float(weak_alpha) * (1.0 - (torch.abs(y_reg) / 3.0).clamp(0, 1))
125         per = per * w
126     loss = per.mean()
127     if lam_emd and lam_emd > 0:
128         if q is None:
129             q = intensity_soft_target(y_reg, tau=soft_tau if soft_tau > 0 else 0.3)
130             loss = loss + float(lam_emd) * emd2_loss(logits, q)
131     if lam_neusup and lam_neusup > 0:
132         ls = neu_suppress_loss(logits, y_cls, lam_neusup)
133         if ls is not None:
134             loss = loss + float(lam_neusup) * ls
135     return loss
136
137
138
139 def consistency_loss(mu, logits):
140     """极性-强度一致性。
141
142     实验事实：附件2的 sign(强度) 与三分类标签一致率为 **1.0000**（n=4850），
143     即“极性 == 强度符号”是硬约束。因此 tanh(mu) 与分类头的期望极性 E[cls]=P(pos)-P(neg)
144     应当一致；该正则把回归头与分类头对齐。
145     """

```

```

146     p = torch.softmax(logits, dim=-1)
147     e = p[:, 2] - p[:, 0] # in[-1,1]
148     return F.mse_loss(torch.tanh(mu), e)
149
150
151 def weighted_smooth_l1(mu, y, beta=0.5, alpha=0.0, gamma=0.5):
152     """按 |y| 加权的 SmoothL1: 把训练重心移向误差主体 (强情感样本)。
153
154     alpha=0 时退化为普通 SmoothL1 (与 MAE 目标对齐)。
155     """
156     loss = F.smooth_l1_loss(mu, y, beta=beta, reduction="none")
157     if alpha and alpha > 0:
158         w = 1.0 + float(alpha) * torch.abs(y).pow(float(gamma))
159         loss = loss * w
160     return loss.mean()
161
162
163 def masked_l1(pred, target, sel):
164     """sel:(B,L) bool, 只对选中位置求 L1。"""
165     if sel.sum() < 1:
166         return pred.new_zeros(())
167     p = pred[sel]
168     t = target[sel]
169     return (p - t).abs().mean()
170
171
172 def recon_loss(rec, x, miss, pad, use_missing=True):
173     """重建损失: use_missing=True 时只算缺失位置, 否则只算可见位置。"""
174     total, n = rec[list(rec.keys())[0]].new_zeros((), 0)
175     for m in MODALITIES:
176         sel = (miss[m] & ~pad[m]) if use_missing else (~miss[m] & ~pad[m])
177         if sel.sum() < 1:
178             continue
179         total = total + masked_l1(rec[m], x[m], sel)
180         n += 1
181     return total / max(n, 1)
182
183
184 def masked_mean(h, valid):
185     w = valid.float().unsqueeze(-1)
186     return (h * w).sum(dim=1) / w.sum(dim=1).clamp(min=1.0)
187
188
189 def sim_loss(share, pad):
190     """共享表示相似性: 把各模态"模态不变"部分拉到一起。"""
191     pooled = {m: masked_mean(share[m], ~pad[m]) for m in MODALITIES}

```



```

192     loss, n = pooled["text"].new_zeros(()), 0
193     keys = list(pooled.keys())
194     for i in range(len(keys)):
195         for j in range(i + 1, len(keys)):
196             loss = loss + F.mse_loss(pooled[keys[i]], pooled[keys[j]])
197             n += 1
198     return loss / max(n, 1)
199
200
201 def diff_loss(share, spec, pad):
202     """共享/特有正交：用余弦平方，避免尺度影响。"""
203     loss, n = None, 0
204     for m in MODALITIES:
205         a, b = share[m], spec[m]
206         cos = F.cosine_similarity(a, b, dim=-1)
207         w = (~pad[m]).float()
208         v = ((cos ** 2) * w).sum() / w.sum().clamp(min=1.0)
209         loss = v if loss is None else loss + v
210         n += 1
211     return loss / max(n, 1)
212
213
214 def balance_loss(gate):
215     """专家负载均衡（熵形式），防止专家坍塌。"""
216     gbar = gate.mean(dim=0)
217     E = gate.size(1)
218     return -(gbar * torch.log(gbar + 1e-8)).sum() * E
219
220
221 def total_loss(out, batch, cfg, lam_kd=0.0, teacher_out=None):
222     """组装总损失；返回 (loss, 明细 dict)。"""
223     mu, logvar, logits = out["mu"], out["logvar"], out["logits"]
224     x, miss, pad = batch["x"], batch["miss"], batch["pad"]
225
226     l_reg_main = weighted_smooth_l1(mu, batch["y_reg"], beta=getattr(cfg, "reg_beta", 0.5),
227                                     alpha=getattr(cfg, "y_weight_alpha", 0.0),
228                                     gamma=getattr(cfg, "y_weight_gamma", 0.5))
229     l_reg_var = reg_nll(mu, logvar, batch["y_reg"])
230     l_reg = l_reg_main + getattr(cfg, "lam_var", 0.1) * l_reg_var
231     cw = getattr(cfg, "cls_weight", None)
232     if cw is not None:
233         cw = torch.tensor(cw, dtype=logits.dtype, device=logits.device)
234     soft_mix = float(getattr(cfg, "soft_mix", 0.0) or 0.0)
235     lam_emd = float(getattr(cfg, "lam_emd", 0.0) or 0.0)
236     weak_alpha = float(getattr(cfg, "weak_alpha", 0.0) or 0.0)
237     lam_neusup = float(getattr(cfg, "lam_neusup", 0.0) or 0.0)

```

```

238     if soft_mix > 0 or lam_emd > 0 or weak_alpha > 0 or lam_neusup > 0:
239         l_cls = cls_D_loss(logits, batch["y_cls"], batch["y_reg"], cw,
240                             label_smoothing=getattr(cfg, "label_smoothing", 0.0),
241                             soft_tau=float(getattr(cfg, "soft_tau", 0.0) or 0.0),
242                             soft_mix=soft_mix, lam_emd=lam_emd,
243                             weak_alpha=weak_alpha, lam_neusup=lam_neusup)
244     else:
245         l_cls = cls_ce(logits, batch["y_cls"], cw, label_smoothing=getattr(cfg, "
label_smoothing", 0.0))
246     zero = l_reg.new_zeros(())
247     has = out.__contains__
248     # D4: 强度分级辅助头 (CORAL 有序回归)
249     lam_mag = float(getattr(cfg, "lam_mag", 0.0) or 0.0)
250     l_mag = (coral_loss(out["logits_mag"], batch["y_reg"],
251                         n_levels=int(getattr(cfg, "mag_levels", 10) or 10))
252              if (lam_mag > 0 and has("logits_mag")) else zero)
253     # 二分类极性辅助头: 直接监督 "是否负面", 与 Acc-2 的判定目标一致
254     lam_pol = float(getattr(cfg, "lam_pol", 0.0) or 0.0)
255     l_pol = zero
256     if lam_pol > 0 and has("pol"):
257         tneg = (batch["y_reg"] < 0).float()
258         pw = ((1.0 - tneg).sum() / tneg.sum().clamp(min=1.0)).detach()
259         l_pol = F.binary_cross_entropy_with_logits(out["pol"], tneg,
260                                                    pos_weight=pw)
261     # 模块可缺省: A0/A1 朴素基线没有重建/分解/专家, 对应项自动跳过 (置 0)
262     l_rec = recon_loss(out["rec"], x, miss, pad, use_missing=True) if has("rec") else zero
263     l_rec0 = recon_loss(out["rec"], x, miss, pad, use_missing=False) if has("rec") else zero
264     l_sim = (sim_loss(out["share"], out["pad_ds"]))
265         if (has("share") and has("pad_ds")) else zero)
266     l_diff = (diff_loss(out["share"], out["spec"], out["pad_ds"]))
267         if (has("share") and has("spec") and has("pad_ds")) else zero)
268     l_bal = balance_loss(out["gate"]) if has("gate") else zero
269     lam_cons = float(getattr(cfg, "lam_cons", 0.0))
270     l_cons = consistency_loss(mu, logits) if lam_cons > 0 else zero
271
272     loss = (l_reg + cfg.lam_cls * l_cls + cfg.lam_rec * l_rec + cfg.lam_rec0 * l_rec0
273            + cfg.lam_sim * l_sim + cfg.lam_diff * l_diff + cfg.lam_bal * l_bal
274            + lam_cons * l_cons + lam_mag * l_mag + lam_pol * l_pol)
275
276     detail = dict(reg=float(l_reg.detach()), reg_main=float(l_reg_main.detach()),
277                  reg_var=float(l_reg_var.detach()), cls=float(l_cls.detach()),
278                  cons=float(l_cons.detach()), mag=float(l_mag.detach()),
279                  pol=float(l_pol.detach()),
280                  rec=float(l_rec.detach()),
281                  rec0=float(l_rec0.detach()), sim=float(l_sim.detach()),
282                  diff=float(l_diff.detach()), bal=float(l_bal.detach()))

```

```

283
284     if teacher_out is not None and lam_kd > 0:
285         tau = cfg.kd_tau
286         l_kd_logit = F.kl_div(F.log_softmax(logits / tau, dim=-1),
287                               F.softmax(teacher_out["logits"] / tau, dim=-1),
288                               reduction="batchmean") * tau * tau
289         l_kd_reg = F.mse_loss(mu, teacher_out["mu"])
290         # 关系蒸馏: 样本间相似度矩阵 (CMAD 相关性感知的简化实现)
291         st = F.normalize(out["z"], dim=-1)
292         tt = F.normalize(teacher_out["z"], dim=-1)
293         sim_s, sim_t = st @ st.t(), tt @ tt.t()
294         l_kd_rel = F.mse_loss(sim_s, sim_t)
295         l_kd = l_kd_logit + l_kd_reg + 0.1 * l_kd_rel      # 关系蒸馏降权, 避免量级压过任务损
失
296         loss = loss + lam_kd * l_kd
297         detail.update(kd=float(l_kd), kd_logit=float(l_kd_logit),
298                      kd_reg=float(l_kd_reg), kd_rel=float(l_kd_rel))
299     return loss, detail
300
301
302 # -----
303 # 指标
304 # -----
305 def _to_np(t):
306     return t.detach().cpu().numpy() if torch.is_tensor(t) else np.asarray(t)
307
308
309 def polar_from_score(score, theta):
310     """连续强度 -> 三分类 (题目定义: <0 负, =0 中, >0 正; theta 容差在 valid 上标定)。"""
311     out = np.ones_like(score, dtype=np.int64)
312     out[score < -theta] = 0
313     out[score > theta] = 2
314     return out
315
316
317 def tune_threshold(score, label_cls, grid=None):
318     """在验证集上按 Macro-F1 选择决策阈值。"""
319     from sklearn.metrics import f1_score
320     if grid is None:
321         grid = np.arange(0.0, 2.01, 0.05)
322     best, best_theta = -1.0, 0.0
323     for th in grid:
324         pred = polar_from_score(score, th)
325         f1 = f1_score(label_cls, pred, average="macro", zero_division=0)
326         if f1 > best:
327             best, best_theta = f1, float(th)

```

```

328     return best_theta, best
329
330
331 def compute_metrics(y_true_reg, y_pred_reg, y_true_cls, theta=0.0, y_pred_cls=None):
332     from sklearn.metrics import accuracy_score, f1_score
333     from scipy.stats import pearsonr
334     yt = _to_np(y_true_reg).astype(np.float64)
335     yp = _to_np(y_pred_reg).astype(np.float64)
336     tc = _to_np(y_true_cls).astype(np.int64)
337
338     mae = float(np.abs(yt - yp).mean())
339     try:
340         corr = float(pearsonr(yt, yp)[0])
341     except Exception:
342         corr = float("nan")
343     # CCC
344     mt, mp = yt.mean(), yp.mean()
345     vt, vp = yt.var(), yp.var()
346     cov = ((yt - mt) * (yp - mp)).mean()
347     ccc = float(2 * cov / (vt + vp + (mt - mp) ** 2 + 1e-12))
348
349     # 极性判别有两套口径:
350     # (A) theta 阈值口径: |y_hat| <= theta 判为 Neutral (theta 在 valid 上按 Macro-F1 选出)
351     # —— 与提交 CSV 的标签完全一致, 故作为**主指标**
352     # (B) 分类头 argmax 口径: 直接取分类头三类的最大值
353     pc_th = polar_from_score(yp, theta).astype(np.int64)
354     res = dict(mae=mae, pearson=corr, ccc=ccc,
355               acc=float(accuracy_score(tc, pc_th)),
356               f1=float(f1_score(tc, pc_th, average="macro", zero_division=0)),
357               theta=float(theta))
358     f1s = f1_score(tc, pc_th, average=None, zero_division=0, labels=[0, 1, 2])
359     res["f1_neg"], res["f1_neu"], res["f1_pos"] = [float(x) for x in f1s]
360     if y_pred_cls is not None:
361         pc_h = _to_np(y_pred_cls).astype(np.int64)
362         res["acc_head"] = float(accuracy_score(tc, pc_h))
363         res["f1_head"] = float(f1_score(tc, pc_h, average="macro", zero_division=0))
364     res["n"] = int(len(yt))
365     return res
366
367
368 def format_metrics(res, tag=""):
369     s = ("%s MAE=%.4f Pearson=%.4f CCC=%.4f | ACC(theta)=%.4f MacroF1(theta)=%.4f"
370         % (tag, res["mae"], res["pearson"], res["ccc"], res["acc"], res["f1"]))
371     if "acc_head" in res:
372         s += " | ACC(分类头)=%.4f MacroF1(分类头)=%.4f" % (res["acc_head"], res["f1_head"])
373     s += (" | F1 neg/neu/pos=%.3f/%.3f/%.3f theta=%.2f n=%d"

```

```

374         % (res["fl_neg"], res["fl_neu"], res["fl_pos"], res["theta"], res["n"]))
375     return s

```

### C.3 问题二：训练与推理主流程（缺失课程学习、EMA、早停）

code/02\_model/train.py（共 476 行）

```

1  # -*- coding: utf-8 -*-
2  """
3  问题2 训练主程序：教师（全模态）->学生（课程学习+缺失注入+蒸馏）->评价->缺失因素扫描
4
5  用法示例：
6  python train.py --data_dir E:\数学建模\data --version aligned
7  python train.py --version unaligned --epochs_teacher 30 --epochs_student 50 --sweep 1
8  python train.py --limit 64 --epochs_teacher 2 --epochs_student 2 --seed 42 # 冒烟测试
9  """
10 import json
11 import os
12 import sys
13 import time
14
15 import numpy as np
16 import torch
17 from torch.utils.data import DataLoader
18
19 sys.path.append(os.path.dirname(os.path.abspath(__file__)))
20 from config import get_config # noqa: E402
21 from data_utils import load_all, MoseiDataset, collate, describe # noqa: E402
22 from missing_sim import (inject_missing, batch_missing_ratios, # noqa: E402
23                          make_scenario_masks, crop_batch,
24                          inject_modality_dropout)
25 from model import MRFNet, PlainFusion # noqa: E402
26 from losses import (total_loss, compute_metrics, tune_threshold, # noqa: E402
27                    format_metrics)
28
29 MODALITIES = ("text", "audio", "vision")
30
31
32 # -----
33 def set_seed(seed):
34     np.random.seed(seed)
35     torch.manual_seed(seed)
36     torch.cuda.manual_seed_all(seed)
37
38
39 def make_loader(split, cfg, shuffle, augment=False):

```

```

40     ds = MoseiDataset(split, augment=augment)
41     return DataLoader(ds, batch_size=cfg.batch_size, shuffle=shuffle,
42                       num_workers=cfg.num_workers, collate_fn=collate,
43                       drop_last=False)
44
45
46 def prep(batch, device):
47     x = {m: batch[m].to(device) for m in MODALITIES}
48     miss = {m: batch["miss_" + m].to(device).bool() for m in MODALITIES}
49     pad = {m: batch["pad_" + m].to(device) for m in MODALITIES}
50     for m in MODALITIES:
51         pad[m] = ensure_valid(pad[m])          # 防全遮 NaN
52     return dict(x=x, miss=miss, pad=pad,
53               y_reg=batch["y_reg"].to(device),
54               y_cls=batch["y_cls"].to(device),
55               idx=batch["idx"].to(device))
56
57
58 def ensure_valid(pad):
59     """保证每条样本每个模态至少有一个非填充位（否则注意力 key 全被屏蔽 -> NaN）。"""
60     allpad = pad.all(dim=1, keepdim=True)
61     return torch.where(allpad, torch.zeros_like(pad), pad)
62
63
64 def mask_feed(b, cfg, unaware=False):
65     """模型看到的”缺失指示”。
66
67     unaware=True（推理期 A0 基线）或 cfg.ablate_mask_indicator=True（A2 消融，训练期也不给）
68     时，把指示位置零：特征仍在缺失位被置零，但模型不知道”那段是缺失的”。
69     """
70     if unaware or getattr(cfg, "ablate_mask_indicator", False):
71         return {m: torch.zeros_like(b["miss"][m]) for m in MODALITIES}
72     return b["miss"]
73
74
75 def stack_masks(b):
76     M = torch.stack([b["miss"][m] for m in MODALITIES], dim=-1)
77     P = torch.stack([b["pad"][m] for m in MODALITIES], dim=-1)
78     return M, P
79
80
81 def unstack_masks(b, M):
82     """把缺失/填充掩码写回，并把不可用位置的特征置零（屏蔽进模型的有效值）。"""
83     for i, m in enumerate(MODALITIES):
84         b["miss"][m] = M[..., i]
85         bad = (b["miss"][m] | b["pad"][m]).float().unsqueeze(-1)

```

```

86         b["x"][m] = b["x"][m] * (1.0 - bad)
87
88
89 def curriculum(epoch, cfg):
90     """课程学习的缺失注入参数。"""
91     for e0, e1, r0, r1, mode in cfg.curric:
92         if e0 <= epoch <= e1:
93             return (r0, r1, mode)
94     return (0.2, 0.5, "mixed")
95
96
97 # -----
98 def run_epoch(model, loader, cfg, device, optimizer=None, teacher=None,
99               lam_kd=0.0, inject_cfg=None, rng=None, ema=None, ema_decay=0.995,
100               epoch=None):
101     train = optimizer is not None
102     model.train(train)
103     if teacher is not None:
104         teacher.eval()
105     if rng is None:
106         rng = np.random.default_rng()
107     tot, cnt, details = 0.0, 0, {}
108     for batch in loader:
109         b = prep(batch, device)
110         x_full = {m: b["x"][m].clone() for m in MODALITIES}
111         if train and getattr(cfg, "crop_aug", False):
112             b = crop_batch(b, rng, min_keep=cfg.crop_min_keep)
113         if train and getattr(cfg, "noise_sigma", 0.0) > 0:
114             sig = float(cfg.noise_sigma)
115             for m in MODALITIES:
116                 b["x"][m] = b["x"][m] + torch.randn_like(b["x"][m]) * sig
117         M, P = stack_masks(b)
118         if inject_cfg is not None:
119             mix = None
120             if getattr(cfg, "rho_mix", None):
121                 ep = 10 ** 9 if epoch is None else int(epoch)
122                 if ep >= int(getattr(cfg, "rho_mix_epoch", 1) or 1):
123                     mix = cfg.rho_mix
124             M2, _ = inject_missing(M, P, inject_cfg[0], inject_cfg[1],
125                                   inject_cfg[2], tuple(cfg.pos_modes), rng=rng,
126                                   n_intervals=tuple(getattr(cfg, "n_intervals", (1, 3))),
127                                   rho_mix=mix)
128             M = M2
129         if train and getattr(cfg, "mod_dropout", 0.0) > 0:
130             M = inject_modality_dropout(M, P, cfg.mod_dropout, rng=rng, n_mods=1)
131         unstack_masks(b, M)

```

```

132
133     with torch.set_grad_enabled(train):
134         ac = autocast_ctx(cfg)
135         if ac is None:
136             mu, logvar, logits, aux = model(b["x"], mask_feed(b, cfg), b["pad"],
return_aux=True)
137         else:
138             with ac:
139                 mu, logvar, logits, aux = model(b["x"], mask_feed(b, cfg), b["pad"],
return_aux=True)
140                 mu, logvar, logits, aux = _to_fp32(mu), _to_fp32(logvar), _to_fp32(logits
), _to_fp32(aux)
141                 out = dict(aux)
142                 out.update(mu=mu, logvar=logvar, logits=logits)
143
144                 t_out = None
145                 if teacher is not None and lam_kd > 0:
146                     with torch.no_grad():
147                         zero = {m: torch.zeros_like(b["miss"][m]) for m in MODALITIES}
148                         if ac is None:
149                             t_mu, _, t_logits, t_aux = teacher(x_full, zero, b["pad"],
return_aux=True)
150                         else:
151                             with ac:
152                                 t_mu, _, t_logits, t_aux = teacher(x_full, zero, b["pad"],
return_aux=True)
153                                 t_mu, t_logits = t_mu.float(), t_logits.float()
154                                 t_out = dict(mu=t_mu, logits=t_logits, z=t_aux["z"].float())
155
156                 loss, det = total_loss(out, b, cfg, lam_kd=lam_kd, teacher_out=t_out)
157
158     if train:
159         optimizer.zero_grad()
160         loss.backward()
161         torch.nn.utils.clip_grad_norm_(model.parameters(), cfg.grad_clip)
162         optimizer.step()
163         if ema is not None:                                     # 权重指数移动平均
164             with torch.no_grad():
165                 sd = model.state_dict()
166                 for k in ema:
167                     v = sd[k]
168                     if v.dtype.is_floating_point:
169                         ema[k].mul_(ema_decay).add_(v.detach().float(), alpha=1.0 -
ema_decay)
170                     else:
171                         ema[k].copy_(v.detach())

```



```

172
173     tot += float(loss.detach())
174     cnt += 1
175     for k, v in det.items():
176         details[k] = details.get(k, 0.0) + v
177     return tot / max(cnt, 1), {k: v / max(cnt, 1) for k, v in details.items()}
178
179
180 @torch.no_grad()
181 def predict(model, loader, cfg, device, split=None, scenario=None, unaware=False):
182     model.eval()
183     ac = autocast_ctx(cfg)
184     S, CL, YR, YC, ID, RH = [], [], [], [], [], []
185     for batch in loader:
186         b = prep(batch, device)
187         M, P = stack_masks(b)
188         if scenario is not None:
189             mtype, pos, rho = scenario
190             M = make_scenario_masks(M, P, mtype, pos, rho, seed=1234)
191         unstack_masks(b, M)
192         feed = mask_feed(b, cfg, unaware=unaware)
193         if ac is None:
194             mu, _, logits = model(b["x"], feed, b["pad"])
195         else:
196             with ac:
197                 mu, _, logits = model(b["x"], feed, b["pad"])
198             mu, logits = mu.float(), logits.float()
199         S.append(mu.cpu().numpy())
200         CL.append(logits.cpu().numpy())
201         YR.append(b["y_reg"].cpu().numpy())
202         YC.append(b["y_cls"].cpu().numpy())
203         RH.append(batch_missing_ratios(M, P).cpu().numpy())
204         if split is not None:
205             idx = b["idx"].cpu().numpy()
206             ID.extend([split["ids"][i] for i in idx])
207     out = dict(score=np.concatenate(S), logits=np.concatenate(CL),
208               y_reg=np.concatenate(YR), y_cls=np.concatenate(YC),
209               rho=np.concatenate(RH))
210     out["ids"] = np.array(ID) if ID else None
211     return out
212
213
214 def report(pred, theta, tag=""):
215     res = compute_metrics(pred["y_reg"], pred["score"], pred["y_cls"],
216                           theta=theta, y_pred_cls=pred["logits"].argmax(-1))
217     print(format_metrics(res, tag=tag))

```

```

218     return res
219
220
221 # -----
222 def build_model(cfg, dims):
223     kind = getattr(cfg, "model_kind", "mrf")
224     if kind == "plain":
225         # A0/A1 基线：朴素拼接 + MLP；—ablate_mask 时连“缺失指示”这一输入通道也不给
226         return PlainFusion(dims=dims, hidden=cfg.hidden, dropout=cfg.dropout,
227                             target_len=cfg.target_len,
228                             use_indicator=not getattr(cfg, "ablate_mask_indicator", False)).to(
229                                 cfg.device)
230     return MRFNet(dims=dims, hidden=cfg.hidden, nhead=cfg.nhead,
231                   enc_layers=cfg.enc_layers, n_experts=cfg.n_experts,
232                   dropout=cfg.dropout, downs=cfg.down_factor_map,
233                   target_len=cfg.target_len,
234                   mag_levels=int(getattr(cfg, "mag_levels", 0) or 0),
235                   ms_pool=bool(getattr(cfg, "ms_pool", False)),
236                   ms_windows=tuple(getattr(cfg, "ms_windows", (5, 25, 50))),
237                   use_pol=float(getattr(cfg, "lam_pol", 0.0) or 0.0) > 0).to(cfg.device)
238
239 def autocast_ctx(cfg):
240     """CPU bfloat16 autocast 上下文（本机 AMD Zen5 支持 AVX512-BF16）。
241
242     注意：CPU 上 PyTorch 的“半精度加速”是 **bfloat16** 而不是 float16
243     （AVX512-FP16 仅 Intel Sapphire Rapids 有；AMD 无 fp16 原生通路）。
244     bfloat16 与 fp32 动态范围相同（8 位指数），无需 GradScaler。
245     """
246     if not getattr(cfg, "bfloat16", False):
247         return None
248     try:
249         return torch.autocast(device_type="cpu", dtype=torch.bfloat16)
250     except Exception as e:
251         # pragma: no cover
252         print("[WARN] bfloat16 autocast 不可用：%s" % e)
253         return None
254
255 def _to_fp32(obj):
256     """把模型输出递归转回 float32，保证自定义损失在 fp32 下计算（可预测、无精度噪声）。"""
257     if torch.is_tensor(obj):
258         return obj.float() if obj.dtype.is_floating_point else obj
259     if isinstance(obj, dict):
260         return {k: _to_fp32(v) for k, v in obj.items()}
261     if isinstance(obj, (list, tuple)):
262         return type(obj)(_to_fp32(v) for v in obj)

```

```

263     return obj
264
265
266 def run_experiment(cfg, seed, data):
267     print("\n" + "=" * 78)
268     print("[SEED %d] version=%s device=%s hidden=%d epochs(t/s)=%d/%d distill=%s"
269           % (seed, cfg.version, cfg.device, cfg.hidden,
270              cfg.epochs_teacher, cfg.epochs_student, getattr(cfg, "use_distill", True)))
271     print("=" * 78)
272     set_seed(seed)
273     tr, va, te = data
274     if cfg.cls_balanced:
275         cnt = np.bincount(np.asarray(tr["labels_cls"]).astype(np.int64), minlength=3).astype
276         (np.float64)
277         w = cnt.sum() / (3.0 * np.maximum(cnt, 1.0))
278         w = np.clip(w, 0.2, 10.0) # 防止某类缺失时权重爆炸
279         cfg.cls_weight = [float(x) for x in w]
280         print("[CLS] 类别计数=%s 权重=%s" % (cnt.astype(int).tolist(),
281                                             [round(x, 3) for x in cfg.cls_weight]))
282     dims = {m: tr["D"][m] for m in MODALITIES}
283     print("dims=%s" % dims)
284     train_loader = make_loader(tr, cfg, shuffle=True, augment=True)
285     valid_loader = make_loader(va, cfg, shuffle=False)
286     test_loader = make_loader(te, cfg, shuffle=False)
287
288     tag = "s%d" % seed
289     hist = []
290
291     # ----- 1) 教师: 全模态 -----
292     teacher = build_model(cfg, dims)
293     opt = torch.optim.AdamW([
294         {"params": [p for n, p in teacher.named_parameters() if "head_" not in n], "lr": cfg.
295         lr_backbone},
296         {"params": [p for n, p in teacher.named_parameters() if "head_" in n], "lr": cfg.
297         lr_head}],
298         weight_decay=cfg.weight_decay)
299     sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=max(1, cfg.
300     epochs_teacher))
301     best_mae, best_state, bad = 1e9, None, 0
302     for ep in range(1, cfg.epochs_teacher + 1):
303         t0 = time.time()
304         l, det = run_epoch(teacher, train_loader, cfg, cfg.device, optimizer=opt,
305         inject_cfg=None)
306         sched.step()
307         pv = predict(teacher, valid_loader, cfg, cfg.device, split=va)
308         mae = float(np.abs(pv["y_reg"] - pv["score"]).mean())

```

```

304     print("[T ep%02d] loss=%.4f mae=%.4f rec=%.4f (%.1fs)"
305           % (ep, l, mae, det.get("rec", 0.0), time.time() - t0))
306     hist.append(dict(phase="teacher", epoch=ep, loss=l, valid_mae=mae, **det))
307     if mae < best_mae - 1e-4:
308         best_mae, bad = mae, 0
309         best_state = {k: v.detach().cpu().clone() for k, v in teacher.state_dict().
items()}
310     else:
311         bad += 1
312         if bad >= cfg.patience:
313             print("[T] early stop @ep%d" % ep)
314             break
315     if best_state is not None:
316         teacher.load_state_dict(best_state)
317     print("[T] best valid MAE = %.4f" % best_mae)
318
319     # ----- 2) 学生: 缺失鲁棒 (课程学习 + 蒸馏) -----
320     student = build_model(cfg, dims)
321     opt = torch.optim.AdamW([
322         {"params": [p for n, p in student.named_parameters() if "head_" not in n], "lr": cfg.
lr_backbone},
323         {"params": [p for n, p in student.named_parameters() if "head_" in n], "lr": cfg.
lr_head}],
324         weight_decay=cfg.weight_decay)
325     sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=max(1, cfg.
epochs_student))
326     use_kd = getattr(cfg, "use_distill", True)
327     ema = None
328     if getattr(cfg, "ema", False):
329         ema = {k: v.detach().clone().float() for k, v in student.state_dict().items()}
330         print("[EMA] 启用权重滑动平均, decay=%.4f" % getattr(cfg, "ema_decay", 0.995))
331     best_mae, best_state, bad = 1e9, None, 0
332     for ep in range(1, cfg.epochs_student + 1):
333         t0 = time.time()
334         r0, r1, mode = curriculum(ep, cfg)
335         if getattr(cfg, "ablate_inject", False):          # A0: 干净数据训练, 不注入缺失
336             inject_cfg = None
337         else:
338             inject_cfg = (r0, r1, mode)
339         if use_kd and ep > cfg.kd_warmup:
340             lam_kd = cfg.lam_kd0 * float(np.exp(-(ep - 1 - cfg.kd_warmup) / cfg.
kd_decay_epochs))
341         else:
342             lam_kd = 0.0
343         l, det = run_epoch(student, train_loader, cfg, cfg.device, optimizer=opt,
344                           teacher=teacher, lam_kd=lam_kd, inject_cfg=inject_cfg,

```

```

345         rng=np.random.default_rng(seed * 1000 + ep),
346         ema=ema, ema_decay=float(getattr(cfg, "ema_decay", 0.995)),
347         epoch=ep)
348     sched.step()
349     if ema is not None:
350         _bk = {k: v.detach().clone() for k, v in student.state_dict().items()}
351         dt = next(student.parameters()).dtype
352         student.load_state_dict({k: v.to(dt) for k, v in ema.items()})
353         pv = predict(student, valid_loader, cfg, cfg.device, split=va)
354         student.load_state_dict(_bk)
355     else:
356         pv = predict(student, valid_loader, cfg, cfg.device, split=va)
357     mae = float(np.abs(pv["y_reg"] - pv["score"]).mean())
358     print("[S ep%02d] loss=%0.4f mae=%0.4f rec=%0.4f kd=%0.4f | rho~[%0.2f,%0.2f] %s (%0.1fs)"
359           % (ep, l, mae, det.get("rec", 0.0), det.get("kd", 0.0), r0, r1, mode, time.
360             time() - t0))
361     hist.append(dict(phase="student", epoch=ep, loss=l, valid_mae=mae,
362                    rho_min=r0, rho_max=r1, lam_kd=lam_kd, **det))
363     if mae < best_mae - 1e-4:
364         best_mae, bad = mae, 0
365         best_state = {k: v.detach().cpu().clone() for k, v in student.state_dict().
366                       items()}
367     else:
368         bad += 1
369         if bad >= cfg.patience:
370             print("[S] early stop @ep%d" % ep)
371             break
372     if best_state is not None:
373         student.load_state_dict(best_state)
374     print("[S] best valid MAE = %0.4f%s" % (best_mae, " (EMA 权重)" if ema is not None else ""))
375
376     # ----- 3) 阈值选择 + 评价 -----
377     pv = predict(student, valid_loader, cfg, cfg.device, split=va)
378     theta, f1v = tune_threshold(pv["score"], pv["y_cls"])
379     print("[TH] theta=%0.2f(valid MacroF1=%0.4f)" % (theta, f1v))
380     res = {}
381     res["valid_clean"] = report(pv, theta, tag="[VALID clean]")
382     pt = predict(student, test_loader, cfg, cfg.device, split=te)
383     res["test_clean"] = report(pt, theta, tag="[TEST clean]")
384     res["theta"] = theta
385
386     # ----- 4) 缺失因素扫描（可选，用于第4节规律分析） -----
387     if cfg.sweep:
388         rows = []
389         pv0 = pv
390         for mtype in ["text", "audio", "vision", "text+audio", "text+vision", "audio+vision"]:

```

```

389         for pos in ["head", "middle", "tail", "random"]:
390             for rho in [0.1, 0.2, 0.3, 0.4, 0.5]:
391                 pa = predict(student, valid_loader, cfg, cfg.device, split=va,
392                             scenario=(mtype, pos, rho), unaware=False)
393                 pu = predict(student, valid_loader, cfg, cfg.device, split=va,
394                             scenario=(mtype, pos, rho), unaware=True)
395                 ra = report(pa, theta)
396                 ru = report(pu, theta)
397                 rows.append(dict(missing_type=mtype, position=pos, rho=rho,
398                                aware_mae=ra["mae"], aware_f1=ra["f1"], aware_acc=ra["
acc"],
399                                aware_corr=ra["pearson"],
400                                unaware_mae=ru["mae"], unaware_f1=ru["f1"],
401                                unaware_acc=ru["acc"], unaware_corr=ru["pearson"])))
402
403     import pandas as pd
404     df = pd.DataFrame(rows)
405     df.to_csv(os.path.join(cfg.out_dir, "sweep_valid_%s.csv" % tag), index=False)
406     print("[SWEEP] 已写入 %s" % os.path.join(cfg.out_dir, "sweep_valid_%s.csv" % tag))
407
408     # 代表性场景在 test 上的"缺失泛化"验证 (附件2 test 有标签, 可量化)
409     trows = []
410     for mtype, pos, rho in [("text", "middle", 0.3), ("audio", "random", 0.3),
411                             ("vision", "random", 0.3), ("text+audio", "random", 0.3),
412                             ("audio+vision", "middle", 0.3), ("text+audio+vision", "random",
0.3)]:
413         pa = predict(student, test_loader, cfg, cfg.device, split=te,
414                     scenario=(mtype, pos, rho), unaware=False)
415         pu = predict(student, test_loader, cfg, cfg.device, split=te,
416                     scenario=(mtype, pos, rho), unaware=True)
417         ra, ru = report(pa, theta), report(pu, theta)
418         trows.append(dict(missing_type=mtype, position=pos, rho=rho,
419                          aware_mae=ra["mae"], aware_f1=ra["f1"], aware_acc=ra["acc"],
420                          unaware_mae=ru["mae"], unaware_f1=ru["f1"], unaware_acc=ru["
acc"])))
421
422     pd.DataFrame(trows).to_csv(os.path.join(cfg.out_dir, "sweep_test_%s.csv" % tag),
index=False)
423     print("[SWEEP] 已写入 %s" % os.path.join(cfg.out_dir, "sweep_test_%s.csv" % tag))
424     res["sweep"] = rows
425
426     # ----- 5) 保存 -----
427     ckpt = os.path.join(cfg.out_dir, "student_%s.pt" % tag)
428     state = student.state_dict()
429     if getattr(cfg, "save_half", False):
430         state = {k: (v.half() if v.is_floating_point() else v) for k, v in state.items()}
431     torch.save(dict(state=state, dims=dims, theta=theta,
stats=tr["stats"], cfg=cfg.__dict__, model_kind=getattr(cfg, "model_kind"

```

```

, "mrf"),
431         mag_levels=int(getattr(cfg, "mag_levels", 0) or 0),
432         ms_pool=bool(getattr(cfg, "ms_pool", False)),
433         ms_windows=list(getattr(cfg, "ms_windows", (5, 25, 50))),
434         use_pol=float(getattr(cfg, "lam_pol", 0.0) or 0.0) > 0,
435         version=cfg.version, seed=seed, half=bool(getattr(cfg, "save_half",
False))), ckpt)
436 print("[SAVE] %s (%.2fMB)" % (ckpt, os.path.getsize(ckpt) / 1e6))
437 import pandas as pd
438 pd.DataFrame(hist).to_csv(os.path.join(cfg.out_dir, "history_%s.csv" % tag), index=False)
439 with open(os.path.join(cfg.out_dir, "metrics_%s.json" % tag), "w", encoding="utf-8") as f:
440     json.dump({k: v for k, v in res.items() if k != "sweep"}, f,
441               ensure_ascii=False, indent=2)
442 return res
443
444
445 def main():
446     cfg = get_config()
447     torch.set_num_threads(max(1, os.cpu_count() or 4))
448     print("[CFG] %s" % json.dumps({k: v for k, v in cfg.__dict__.items()
449                                   if isinstance(v, (int, float, str, bool))},
450                                   ensure_ascii=False, indent=2))
451     print("[DATA] 读取 %s" % cfg.pkl_path)
452     tr, va, te = load_all(cfg)
453     for name, s in (("train", tr), ("valid", va), ("test", te)):
454         print(describe(s, name))
455
456     all_res = []
457     for seed in cfg.seeds:
458         all_res.append(run_experiment(cfg, seed, (tr, va, te)))
459
460     if len(all_res) > 1:
461         print("\n" + "=" * 78)
462         print("[SUMMARY] %d seeds: mean +/- std" % len(all_res))
463         keys = ["mae", "pearson", "ccc", "acc", "fl"]
464         for split in ("valid_clean", "test_clean"):
465             line = split + ":"
466             for k in keys:
467                 v = np.array([r[split][k] for r in all_res])
468                 line += "%s=%.4f+/-%.4f " % (k, v.mean(), v.std())
469             print(line)
470         with open(os.path.join(cfg.out_dir, "summary.json"), "w", encoding="utf-8") as f:
471             json.dump([{"k: v for k, v in r.items() if k != "sweep"} for r in all_res],
472                       f, ensure_ascii=False, indent=2)
473
474

```

```

475 if __name__ == "__main__":
476     main()

```

## C.4 问题三：骨干加载与批构造（三源解释的公共入口）

code/03\_xai/xai\_common.py（共 247 行）

```

1  # -*- coding: utf-8 -*-
2  r"""问题3 共享工具：骨干加载、split 构建、掩码/遮挡算子、目标标量、缓存读写。
3
4  设计要点（与 02_model 的推理链路严格一致，保证”解释的是同一个模型”）：
5  * 复用 ‘runtime.load_ensemble’ 与 ‘train.prep/stack_masks/unstack_masks’，
6    不重写前向路径，避免”解释的模型 != 预测的模型”；
7  * 屏蔽模式统一用 ‘miss=1’（模型在训练时见过的”整模式缺失”语义），
8    并同步把特征置零（与 ‘unstack_masks’ 的做法一致）；
9  * 目标标量支持 ‘mu’（强度，主口径）与 ‘p_pos/p_neg/pol/p_max’（极性，对照口径）。
10 """
11 import glob
12 import json
13 import os
14 import sys
15
16 import numpy as np
17 import torch
18
19 HERE = os.path.dirname(os.path.abspath(__file__))
20 CODE = os.path.dirname(HERE)
21 ROOT = os.path.dirname(CODE)
22 for p in (CODE, os.path.join(CODE, "02_model")):
23     if p not in sys.path:
24         sys.path.append(p)
25
26 from data_utils import load_any, MoseiDataset, collate # noqa: E402
27 from runtime import load_ensemble, predict_split # noqa: E402
28 from train import prep, stack_masks, unstack_masks # noqa: E402
29
30 MODALITIES = ("text", "audio", "vision")
31 LABELS = ("Negative", "Neutral", "Positive")
32 TARGETS = ("mu", "p_pos", "p_neg", "p_pol", "p_max")
33
34
35 # -----
36 # 骨干与数据
37 # -----
38 def load_backbone(ckpt_dir=None, device="cpu", verbose=True):
39     ckpt_dir = ckpt_dir or os.path.join(ROOT, "runs", "ens_top2")

```



```

40 ckpts = sorted(glob.glob(os.path.join(ckpt_dir, "student_*.pt")))
41 if not ckpts:
42     raise SystemExit("未找到权重: %s/student_*.pt" % ckpt_dir)
43 models = load_ensemble(ckpts, device=device, verbose=verbose)
44 for m, _ in models:                                # 只解释输入，不需要参数梯度
45     for p in m.parameters():
46         p.requires_grad_(False)
47     theta = float(np.mean([ck["theta"] for _, ck in models]))
48     return models, models[0][1]["stats"], theta
49
50
51 def load_split(pk1, stats, key=None):
52     """读任意 pk1（附件2/3/4）并用 ckpt 内统计量归一化。"""
53     splits = load_any(pk1, norm_stats=stats)
54     if key is None:
55         key = "test" if "test" in splits else list(splits.keys())[0]
56     return splits[key], key
57
58
59 def make_batch(split, idxs, device="cpu", mask=True):
60     """按 DataLoader 同路径构造 batch（含 pad 有效位兜底）。
61
62     mask=True 时额外执行 'apply_masks'（把 miss|pad 位置的特征置零），
63     与 'infer_att3.py' 的推理链路**完全一致**——这是“解释的模型=预测的模型”的前提：
64     若不置零，填充位的 BERT 垃圾值会经编码器内的局部卷积渗入最后一个有效位，
65     造成解释与提交预测的口径偏差（实测最大分数差 0.049、1/20 标签不同）。
66     """
67     ds = MoseiDataset(split, augment=False)
68     b = prep(collate([ds[int(i)] for i in idxs]), device)
69     return apply_masks(b) if mask else b
70
71
72 def apply_masks(b):
73     """与推理链路一致：把 miss|pad 位置的特征置零。"""
74     M, _ = stack_masks(b)
75     unstack_masks(b, M)
76     return b
77
78
79 def valid_len(b, m, j):
80     """第 j 个样本、模态 m 的有效长度（非填充位）。"""
81     return int((~b["pad"][m][j]).sum().item())
82
83
84 # -----
85 # 掩码/遮挡算子

```

```

86 # -----
87 def mask_modalities(b, mods, mode="miss"):
88     """屏蔽若干模态。mode='miss' 置缺失指示；mode='pad' 整模态不可见。"""
89     out = dict(x={}, miss={}, pad={})
90     for k in ("y_reg", "y_cls", "idx"):
91         if k in b:
92             out[k] = b[k]
93     for m in MODALITIES:
94         out["x"][m] = b["x"][m].clone()
95         out["miss"][m] = b["miss"][m].clone()
96         out["pad"][m] = b["pad"][m].clone()
97     for m in mods:
98         if mode == "miss":
99             out["miss"][m] = torch.ones_like(b["miss"][m])
100         else:
101             p = torch.ones_like(b["pad"][m])
102             p[:, 0] = False # 安全位：避免注意力全遮 -> NaN
103             out["pad"][m] = p
104     for m in MODALITIES:
105         bad = (out["miss"][m] | out["pad"][m]).float().unsqueeze(-1)
106         out["x"][m] = out["x"][m] * (1.0 - bad)
107     return out
108
109
110 def occlude_window(b, m, start, width):
111     """把模态 m 的 [start, start+width) 段用相邻段替换（保持序列长度与形态）。
112
113     首段向右取邻段、末段向左取邻段；缺失/填充位保持不可用。
114     """
115     out = dict(x={}, miss={}, pad={})
116     for k in ("y_reg", "y_cls", "idx"):
117         if k in b:
118             out[k] = b[k]
119     L = b["x"][m].size(1)
120     for mm in MODALITIES:
121         out["x"][mm] = b["x"][mm].clone()
122         out["miss"][mm] = b["miss"][mm].clone()
123         out["pad"][mm] = b["pad"][mm].clone()
124     s = max(0, int(start))
125     e = min(L, s + int(width))
126     if e <= s:
127         return out
128     off = e - s
129     if e + off <= L:
130         src = slice(e, e + off)
131         dst = slice(s, e)

```

```

132 elif s - off >= 0:
133     src = slice(s - off, s)
134     dst = slice(s, e)
135 else:                                     # 序列过短: 退回均值替换
136     src, dst = None, slice(s, e)
137 for j in range(out["x"][m].size(0)):
138     if src is None:
139         avail = ~(b["miss"][m][j] | b["pad"][m][j])
140         fill = b["x"][m][j][avail].mean(dim=0) if avail.any() else torch.zeros_like(
141             b["x"][m][j][0])
142         out["x"][m][j, dst] = fill
143     else:
144         out["x"][m][j, dst] = b["x"][m][j][src]
145         out["miss"][m][j, dst] = False      # 被替换段视为"有信息"
146 bad = (out["miss"][m] | out["pad"][m]).float().unsqueeze(-1)
147 out["x"][m] = out["x"][m] * (1.0 - bad)
148 return out
149
150
151 # -----
152 # 前向与目标标量
153 # -----
154 def predict_batch(models, b, target="mu", aux_keys=()):
155     """集成前向。返回 score/prob/target (均为可微张量) 与可选 aux。"""
156     mus, ps = [], []
157     aux_sum = {k: None for k in aux_keys}
158     for m, _ in models:
159         if aux_keys:
160             mu, _, lg, ax = m(b["x"], b["miss"], b["pad"], return_aux=True)
161             for k in aux_keys:
162                 v = ax.get(k)
163                 if torch.is_tensor(v) and v.size(0) == mu.size(0):
164                     v = v.detach()
165                     aux_sum[k] = v if aux_sum[k] is None else aux_sum[k] + v
166             else:
167                 mu, _, lg = m(b["x"], b["miss"], b["pad"])
168                 mus.append(mu)
169                 ps.append(torch.softmax(lg, dim=-1))
170     score = torch.stack(mus).mean(0)
171     prob = torch.stack(ps).mean(0)
172     if target == "mu":
173         t = score
174     elif target == "p_pos":
175         t = prob[:, 2]
176     elif target == "p_neg":
177         t = prob[:, 0]

```

```

178 elif target == "p_pol":
179     t = prob[:, 2] - prob[:, 0]
180 elif target == "p_max":
181     t = prob.max(dim=-1).values
182 else:
183     raise ValueError("未知 target=%s (可选 %s)" % (target, TARGETS))
184 out = dict(score=score, prob=prob, target=t)
185 for k in aux_keys:
186     if aux_sum[k] is not None:
187         out[k] = aux_sum[k] / len(models)
188 return out
189
190
191 # -----
192 # 重要性后处理与缓存
193 # -----
194 def norm_valid(w, n_valid):
195     """把重要性向量限制在有效位内并归一到 [0,1] (全零时返回全零)。"""
196     w = np.asarray(w, dtype=np.float64).copy()
197     n = int(max(1, min(len(w), n_valid)))
198     seg = w[:n]
199     w[n:] = 0.0
200     lo, hi = float(seg.min()), float(seg.max())
201     if hi - lo < 1e-12:
202         return np.zeros_like(w)
203     w[:n] = (seg - lo) / (hi - lo)
204     return w
205
206
207 def softmax1d(x, tau=1.0):
208     x = np.asarray(x, dtype=np.float64) / max(float(tau), 1e-9)
209     x = x - x.max()
210     e = np.exp(x)
211     return e / e.sum()
212
213
214 def save_npz(path, **arrays):
215     os.makedirs(os.path.dirname(path), exist_ok=True)
216     np.savez_compressed(path, **arrays)
217     return path
218
219
220 def load_npz(path):
221     with np.load(path) as z:
222         return {k: z[k] for k in z.files}
223

```

```

224
225 def save_json(path, obj):
226     os.makedirs(os.path.dirname(path), exist_ok=True)
227     with open(path, "w", encoding="utf-8") as f:
228         json.dump(obj, f, ensure_ascii=False, indent=1, default=_jsonable)
229     return path
230
231
232 def _jsonable(o):
233     if isinstance(o, (np.integer,)):
234         return int(o)
235     if isinstance(o, (np.floating,)):
236         return float(o)
237     if isinstance(o, np.ndarray):
238         return o.tolist()
239     return str(o)
240
241
242 def q3_meta(ckpt_dir, models, theta, extra=None):
243     """记录本次解释使用的骨干与阈值，供复现核对。"""
244     files = sorted(os.path.basename(p) for p in
245                    glob.glob(os.path.join(ckpt_dir, "student_*.pt")))
246     return dict(model_files=files, n_ckpt=len(models), ckpt_dir=str(ckpt_dir),
247                theta=round(float(theta), 4), extra=extra or {})

```

### C.5 问题三：精确 Shapley 值（八子集全枚举）

code/03\_xai/shapley.py（共 258 行）

```

1  # -*- coding: utf-8 -*-
2  r"""S1: 模态级精确 Shapley 值（3 模态 -> 8 个子集全枚举）。
3
4  数学定义（论文 section5.3 用）：
5      v(S) 为只用模态子集 S 时模型的目标标量；模态 m 的 Shapley 值
6       $\phi_m = \sum_{S \subset M \setminus \{m\}} \frac{|S|!(n-|S|-1)!}{n!} \cdot [v(S \cup \{m\}) - v(S)]$ ,  $n=3$ 
7      精确性检验： $\sum_m \phi_m = v(M) - v(\{\})$ 。
8
9  本脚本一次算完 4 个口径组合（保证可比、避免重复前向）：
10     target in {mu(强度,主), p_pol = p_pos - p_neg(极性,对照)}
11     semantic in {miss(置缺失指示,主), pad(整模态不可见,对照)}
12  并输出两套归一化（ReLU 归一 / softmax|phi|）供后续按验证集“模态忠实性”择优。
13
14  两类效用：
15     * 样本级：v(S) 取该样本自身的预测 -> 逐样本 phi（写入附件4 解释 CSV）；
16     * 数据集级：v(S) 取验证集上的 -MAE 与 Macro-F1 -> 全局模态重要性（论文结论）。
17

```

```

18 用法:
19 python shapley.py --pkl <pkl> --name att4 --out_dir <runs/q3>
20 python shapley.py --pkl <附件2 aligned_50.pkl> --split valid --name valid --with_global
21 """
22 import argparse
23 import itertools
24 import math
25 import os
26 import sys
27
28 import numpy as np
29 import torch
30
31 HERE = os.path.dirname(os.path.abspath(__file__))
32 if HERE not in sys.path:
33     sys.path.append(HERE)
34 from xai_common import (MODALITIES, load_backbone, load_split, make_batch,      # noqa: E402
35                          predict_batch, mask_modalities, norm_valid, softmax1d,
36                          save_npz, save_json, q3_meta, ROOT)
37
38 SUBSETS = [tuple(m for i, m in enumerate(MODALITIES) if (s >> i) & 1) for s in range(8)]
39 MASKS = list(range(8))
40 NAMES = {0: "none", 1: "text", 2: "audio", 4: "vision",
41          3: "text+audio", 5: "text+vision", 6: "audio+vision", 7: "all"}
42 WEIGHTS = {}
43
44
45 def mods_of(mask):
46     """位掩码->模态名列表(text=1, audio=2, vision=4)。"""
47     return [MODALITIES[i] for i in range(3) if (mask >> i) & 1]
48
49
50 def shapley_weights(n=3):
51     """精确 Shapley 的组合权重  $w(S) = |S|!(n-|S|-1)!/n!$  (按位掩码索引)。"""
52     if n not in WEIGHTS:
53         w = {}
54         for i in range(3):
55             others = [j for j in range(3) if j != i]
56             tab = {}
57             for r in range(len(others) + 1):
58                 for S in itertools.combinations(others, r):
59                     tab[sum(1 << j for j in S)] = (math.factorial(len(S)) *
60                                                     math.factorial(n - len(S) - 1) /
61                                                     math.factorial(n))
62             w[i] = tab
63         WEIGHTS[n] = w

```

```

64     return WEIGHTS[n]
65
66
67 def phi_from_values(v):
68     """由 8 个子集的 v 值（按 mask 索引）算精确 Shapley 向量 (3,)。"""
69     W = shapley_weights(3)
70     phi = np.zeros(3)
71     for i in range(3):
72         for T, wt in W[i].items():
73             phi[i] += wt * (v[T | (1 << i)] - v[T])
74     return phi
75
76
77 def normalizers(phi):
78     """两套归一化（对 (N,3) 或 (3,) 的 phi 逐样本处理）。"""
79     a = np.asarray(phi, dtype=np.float64)
80     single = a.ndim == 1
81     a = a[None] if single else a
82     relu = np.maximum(a, 0.0)
83     s = relu.sum(1, keepdims=True)
84     pi_relu = np.where(s > 1e-12, relu / np.maximum(s, 1e-12), np.nan)
85     pi_soft = np.exp(np.abs(a) / 1.0)
86     pi_soft = pi_soft / pi_soft.sum(1, keepdims=True)
87     return (pi_relu[0], pi_soft[0]) if single else (pi_relu, pi_soft)
88
89
90 def run_batch(models, b, target, semantic):
91     """对一个 batch 算 8 个子集的 v 值 -> V (B,8)，并返回全模态输出。"""
92     B = b["x"]["text"].size(0)
93     V = np.zeros((B, 8))
94     full = None
95     for mask in MASKS:
96         bb = b if mask == 7 else mask_modalities(
97             b, [m for m in MODALITIES if m not in mods_of(mask)], mode=semantic)
98         out = predict_batch(models, bb, target=target)
99         V[:, mask] = out["target"].detach().cpu().numpy()
100         if mask == 7:
101             full = dict(score=out["score"].detach().cpu().numpy(),
102                         prob=out["prob"].detach().cpu().numpy())
103     return V, full
104
105
106 def global_values(models, split, target, semantic, batch=64, device="cpu"):
107     """数据集级效用：v(S) = -MAE（回归）与 Macro-F1（分类头概率）。"""
108     from sklearn.metrics import f1_score
109     N = split["N"]

```

```

110 y = np.asarray(split["labels_reg"])
111 yc = np.asarray(split["labels_cls"])
112 v_mae, v_f1 = np.zeros(8), np.zeros(8)
113 for mask in MASKS:
114     preds, probs = [], []
115     for i0 in range(0, N, batch):
116         idxs = list(range(i0, min(i0 + batch, N)))
117         b = make_batch(split, idxs, device)
118         if mask != 7:
119             b = mask_modalities(
120                 b, [m for m in MODALITIES if m not in mods_of(mask)], mode=semantic)
121         out = predict_batch(models, b, target=target)
122         preds.append(out["score"].detach().cpu().numpy())
123         probs.append(out["prob"].detach().cpu().numpy())
124     sc, pb = np.concatenate(preds), np.concatenate(probs)
125     v_mae[mask] = -float(np.abs(sc - y).mean())
126     try:
127         v_f1[mask] = float(f1_score(yc, pb.argmax(1), average="macro"))
128     except Exception:
129         v_f1[mask] = np.nan
130     print(" [GLOBAL] S=%-12s MAE=%.4f MacroF1=%.4f"
131           % (NAMES[mask], -v_mae[mask], v_f1[mask]))
132 return v_mae, v_f1
133
134
135 def main():
136     ap = argparse.ArgumentParser()
137     ap.add_argument("--pkl", default=None, help="默认附件2 aligned (用 --split valid)")
138     ap.add_argument("--split", default=None, help="pkl 内的 split 名 (附件2 需指定 valid)")
139     ap.add_argument("--name", default="att4", help="输出文件名后缀")
140     ap.add_argument("--ckpt_dir", default=os.path.join(ROOT, "runs", "ens_top2"))
141     ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
142     ap.add_argument("--batch_size", type=int, default=32)
143     ap.add_argument("--limit", type=int, default=0, help=">0 时只算前 N 条")
144     ap.add_argument("--device", default="cpu")
145     ap.add_argument("--with_global", action="store_true", help="额外算数据集级 Shapley")
146     ap.add_argument("--global_target", default="mu", choices=["mu", "p_pol"])
147     a = ap.parse_args()
148
149     pkl = a.pkl or os.path.join(ROOT, "附件2", "aligned_50.pkl")
150     models, stats, theta = load_backbone(a.ckpt_dir, a.device)
151     split, key = load_split(pkl, stats, a.split)
152     if a.limit:
153         from data_utils import slice_split
154         split = slice_split(split, a.limit)
155     N = split["N"]

```



```

156 print("[DATA] %s/%s N=%d" % (os.path.basename(pk1), key, N))
157
158 combos = [("mu", "miss"), ("mu", "pad"), ("p_pol", "miss"), ("p_pol", "pad")]
159 store, v_store = {}, {}
160 score_all, prob_all = None, None
161 for target, semantic in combos:
162     tag = "%s_%s" % (target, semantic)
163     V = np.zeros((N, 8))
164     phis = np.zeros((N, 3))
165     sc, pb = [], []
166     for i0 in range(0, N, a.batch_size):
167         idxs = list(range(i0, min(i0 + a.batch_size, N)))
168         b = make_batch(split, idxs, a.device)
169         v, full = run_batch(models, b, target, semantic)
170         for r in range(len(idxs)):
171             phis[i0 + r] = phi_from_values(v[r])
172         V[i0:i0 + len(idxs)] = v
173         sc.append(full["score"])
174         pb.append(full["prob"])
175     store[tag], v_store[tag] = phis, V
176     resid = float(np.abs(phis.sum(1) - (V[:, 7] - V[:, 0])).max())
177     print(" [%-10s] phi 均值 文/语/视 = %+0.4f / %+0.4f / %+0.4f; 精确性残差 max=%0.2e"
178           % (tag, *phis.mean(0), resid))
179     assert resid < 1e-6, "Shapley 精确性检验失败"
180     if tag == "mu_miss":
181         score_all, prob_all = np.concatenate(sc), np.concatenate(pb)
182
183 # 主口径: mu + miss; 同时给出两套归一化
184 phi_main = store["mu_miss"]
185 pi_relu, pi_soft = normalizers(phi_main)
186 n_relu = int(np.isnan(pi_relu).any(1).sum())
187 print(" [NORM] ReLU 归一在全负样本上退化 %d/%d 条; softmax 口径无退化" % (n_relu, N))
188 miss_rate = np.zeros((N, 3))
189 for i in range(N):
190     for k, m in enumerate(MODALITIES):
191         span = float((~split["pad"][m][i]).sum())
192         if span <= 1.0: # 整条全零: 整模态缺失 (口径 D1)
193             miss_rate[i, k] = 1.0
194         else:
195             miss_rate[i, k] = float((split["miss"][m][i] & ~split["pad"][m][i]).sum())
196 / span
197
198 out = dict(ids=np.asarray(split["ids"]).astype(str),
199           score=score_all, prob=prob_all, phi_mu_miss=phi_main,
200           pi_relu=np.nan_to_num(pi_relu, nan=0.0), pi_soft=pi_soft,
201           miss_rate=miss_rate, subset_names=np.array([NAMES[m] for m in MASKS]))

```

```

201 for tag in store:
202     out["phi_" + tag] = store[tag]
203     out["v_" + tag] = v_store[tag]
204 path = save_npz(os.path.join(a.out_dir, "q3_shapley_%s.npz" % a.name), **out)
205
206 # CSV (逐样本)
207 import pandas as pd
208 df = pd.DataFrame({
209     "id": out["ids"],
210     "score": np.round(score_all, 4),
211     "prob_neg": np.round(prob_all[:, 0], 4), "prob_neu": np.round(prob_all[:, 1], 4),
212     "prob_pos": np.round(prob_all[:, 2], 4),
213     "main_modality": [MODALITIES[int(np.argmax(r))] for r in pi_soft],
214     "main_modality_relu": [MODALITIES[int(np.argmax(r))] if not np.isnan(r).any()
215                             else "" for r in pi_relu],
216     "pi_text": np.round(pi_soft[:, 0], 4), "pi_audio": np.round(pi_soft[:, 1], 4),
217     "pi_vision": np.round(pi_soft[:, 2], 4),
218     "phi_text": np.round(phi_main[:, 0], 4), "phi_audio": np.round(phi_main[:, 1], 4),
219     "phi_vision": np.round(phi_main[:, 2], 4),
220     "phi_text_padsem": np.round(store["mu_pad"][:, 0], 4),
221     "phi_audio_padsem": np.round(store["mu_pad"][:, 1], 4),
222     "phi_vision_padsem": np.round(store["mu_pad"][:, 2], 4),
223     "miss_text": np.round(miss_rate[:, 0], 4), "miss_audio": np.round(miss_rate[:, 1], 4),
224     "miss_vision": np.round(miss_rate[:, 2], 4),
225 })
226 csv = os.path.join(a.out_dir, "q3_shapley_%s.csv" % a.name)
227 os.makedirs(a.out_dir, exist_ok=True)
228 df.to_csv(csv, index=False, encoding="utf-8-sig")
229
230 report = dict(meta=q3_meta(a.ckpt_dir, models, theta,
231                             dict(pk1=os.path.relpath(pk1, ROOT), split=key, n=N)),
232               data=os.path.relpath(path, ROOT),
233               phi_mean={k: np.round(store[k].mean(0), 4).tolist() for k in store},
234               pi_soft_mean=np.round(pi_soft.mean(0), 4).tolist(),
235               pi_relu_mean=np.round(np.nan_to_num(pi_relu, nan=0.0).mean(0), 4).
236               tolist(),
237               pi_relu_degenerate=int(n_relu),
238               v_mean_mu_miss={NAMES[m]: round(float(v_store["mu_miss"][:, m].mean()),
239               4)
240                               for m in MASKS})
241
242 if a.with_global:
243     print("[GLOBAL] 数据集级效用 (%s / miss 语义) : " % a.global_target)
244     v_mae, v_f1 = global_values(models, split, a.global_target, "miss", a.batch_size,
245                                 a.device)
246     report["global"] = dict(target=a.global_target,
247                             v_negmae={NAMES[m]: round(float(v_mae[m]), 4) for m in

```

```

MASKS},
245         v_macrof1={NAMES[m]: round(float(v_f1[m]), 4) for m in
MASKS},
246         phi_negmae=np.round(phi_from_values(v_mae), 4).tolist(),
247         phi_macrof1=np.round(
248             phi_from_values(np.nan_to_num(v_f1)), 4).tolist())
249     print("   phi(-MAE) = %s" % report["global"]["phi_negmae"])
250     print("   phi(MacroF1) = %s" % report["global"]["phi_macrof1"])
251     save_json(os.path.join(a.out_dir, "q3_shapley_%s_report.json" % a.name), report)
252     print("[SAVE] %s\n[SAVE] %s" % (os.path.relpath(path, ROOT), os.path.relpath(csv, ROOT)
    ))
253     print("SHAPLEY_OK")
254     return 0
255
256
257 if __name__ == "__main__":
258     sys.exit(main())

```

## C.6 问题三：积分梯度与片段级重要性

code/03\_xai/ig\_gradcam.py（共 134 行）

```

1  #-*- coding: utf-8 -*-
2  r"""S2: 片段级时间重要性的梯度类来源（注意力 / 积分梯度 / 注意力 x 梯度）。
3
4  三个来源（论文 section5.3 的 (a)(b) 两族）：
5  (a) 注意力源      w^att_k = alpha_k          （模型内生，仅作参考）
6  (a') 注意力 x 梯度源  w^agrad_k = alpha_k . ||d t/d x_k||_2    （Grad-CAM 式加权）
7  (b) 积分梯度      w^ig_k = ||(x_k - x'_k) (*) (1/L)Sigma_l grad_{x_k} t(x' + 1/L(x-x'))||_2
8
9  基线 x': 'zero'（归一化空间中即训练集均值，主口径）/ 'mean'（本 batch 有效位均值，对照）。
10
11  用法：
12  python ig_gradcam.py --pk1 <pk1> --name att4 --steps 20 --target mu --out_dir <runs/q3>
13  """
14  import argparse
15  import os
16  import sys
17
18  import numpy as np
19  import torch
20
21  HERE = os.path.dirname(os.path.abspath(__file__))
22  if HERE not in sys.path:
23      sys.path.append(HERE)
24  from xai_common import (MODALITIES, load_backbone, load_split, make_batch,      # noqa:

```

```

25         predict_batch, save_npz, save_json, q3_meta, ROOT)
26
27
28 def ig_batch(models, b, steps=20, baseline="zero", target="mu", need_agrad=True):
29     """返回 (w_ig (B,3,L), w_att (B,3,L), w_agrad (B,3,L))。"""
30     x0 = {m: b["x"][m].detach().clone() for m in MODALITIES}
31     if baseline == "zero":
32         base = {m: torch.zeros_like(x0[m]) for m in MODALITIES}
33     else:
34         # mean: 该样本有效位均值
35         base = {}
36         for m in MODALITIES:
37             valid = (~(b["miss"][m] | b["pad"][m])).float().unsqueeze(-1)
38             mu = (x0[m] * valid).sum(1, keepdim=True) / valid.sum(1, keepdim=True).clamp(
39                 min=1)
40             base[m] = mu.expand_as(x0[m]).contiguous()
41         gsum = {m: torch.zeros_like(x0[m]) for m in MODALITIES}
42         for l in range(1, steps + 1):
43             a = 1 / float(steps)
44             xs = {m: (base[m] + a * (x0[m] - base[m])).requires_grad_(True) for m in
45                 MODALITIES}
46             out = predict_batch(models, dict(x=xs, miss=b["miss"], pad=b["pad"]), target=target
47                 )
48             g = torch.autograd.grad(out["target"].sum(), [xs[m] for m in MODALITIES],
49                 retain_graph=False)
50             for i, m in enumerate(MODALITIES):
51                 gsum[m] = gsum[m] + g[i].detach()
52             w_ig = torch.stack([(x0[m] - base[m]) * (gsum[m] / steps)).norm(dim=-1)
53                 for m in MODALITIES], dim=1) # (B,3,L)
54
55             # 注意力源 + 注意力 x 梯度源 (一次前向 + 一次反向)
56             xg = {m: x0[m].clone().requires_grad_(True) for m in MODALITIES}
57             out = predict_batch(models, dict(x=xg, miss=b["miss"], pad=b["pad"]),
58                 target=target, aux_keys=("alpha",))
59             grads = torch.autograd.grad(out["target"].sum(), [xg[m] for m in MODALITIES],
60                 retain_graph=False)
61             gnorm = torch.stack([grads[i].detach().norm(dim=-1) for i in range(3)], dim=1) # (B
62                 ,3,L)
63             B, _, L = gnorm.shape
64             alpha = out.get("alpha")
65             if alpha is not None and alpha.size(0) == B and alpha.size(1) == 3 * L:
66                 w_att = alpha.detach().reshape(B, 3, L)
67             else:
68                 # 兜底: 无法取注意力时退化为梯度幅值
69                 w_att = gnorm.clone()
70             w_agrad = w_att * gnorm
71             return (w_ig.detach().cpu().numpy(), w_att.cpu().numpy(), w_agrad.detach().cpu()).

```

```

        numpy())
66
67
68 def main():
69     ap = argparse.ArgumentParser()
70     ap.add_argument("--pkl", default=None)
71     ap.add_argument("--split", default=None)
72     ap.add_argument("--name", default="att4")
73     ap.add_argument("--ckpt_dir", default=os.path.join(ROOT, "runs", "ens_top2"))
74     ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
75     ap.add_argument("--steps", type=int, default=20)
76     ap.add_argument("--baseline", default="zero", choices=["zero", "mean"])
77     ap.add_argument("--target", default="mu", choices=["mu", "p_pol"])
78     ap.add_argument("--batch_size", type=int, default=8)
79     ap.add_argument("--limit", type=int, default=0)
80     ap.add_argument("--device", default="cpu")
81     a = ap.parse_args()
82
83     pkl = a.pkl or os.path.join(ROOT, "附件2", "aligned_50.pkl")
84     models, stats, theta = load_backbone(a.ckpt_dir, a.device)
85     split, key = load_split(pkl, stats, a.split)
86     if a.limit:
87         from data_utils import slice_split
88         split = slice_split(split, a.limit)
89     N = split["N"]
90     L = split["L"]["text"]
91     print("[DATA] %s/%s N=%d L=%d | IG steps=%d baseline=%s target=%s"
92           % (os.path.basename(pkl), key, N, L, a.steps, a.baseline, a.target))
93
94     W_IG = np.zeros((N, 3, L), np.float64)
95     W_AT = np.zeros((N, 3, L), np.float64)
96     W_AG = np.zeros((N, 3, L), np.float64)
97     for i0 in range(0, N, a.batch_size):
98         idxs = list(range(i0, min(i0 + a.batch_size, N)))
99         b = make_batch(split, idxs, a.device)
100         ig, att, ag = ig_batch(models, b, a.steps, a.baseline, a.target)
101         W_IG[i0:i0 + len(idxs)] = ig
102         W_AT[i0:i0 + len(idxs)] = att
103         W_AG[i0:i0 + len(idxs)] = ag
104         if (i0 // a.batch_size) % 20 == 0:
105             print(" %d/%d" % (min(i0 + a.batch_size, N), N))
106
107     # 有效位掩码（重要性只在有效位内有意义）
108     valid = np.zeros((N, 3, L), bool)
109     for i in range(N):
110         for k, m in enumerate(MODALITIES):

```

```

111         valid[i, k, :int((~split["pad"])[m][i]).sum())] = True
112     for W in (W_IG, W_AT, W_AG):
113         W[~valid] = 0.0
114     path = save_npz(os.path.join(a.out_dir, "q3_temporal_ig_%s.npz" % a.name),
115                     ids=np.asarray(split["ids"]).astype(str),
116                     w_ig=W_IG, w_att=W_AT, w_agrad=W_AG, valid=valid,
117                     meta=np.array([a.steps, a.baseline, a.target], dtype=object))
118     rep = dict(meta=q3_meta(a.ckpt_dir, models, theta,
119                             dict(pk1=os.path.relpath(pk1, ROOT), split=key, n=N,
120                                 steps=a.steps, baseline=a.baseline, target=a.target)),
121               w_ig_mean=[round(float(W_IG[:, k].sum(1).mean()), 4) for k in range(3)],
122               w_att_mean=[round(float(W_AT[:, k].sum(1).mean()), 4) for k in range(3)],
123               sparsity_ig_top5=[round(float(np.sort(W_IG[i, k, valid[i, k]]][::-1][:5].
124               sum() /
125               max(W_IG[i, k, valid[i, k]].sum(), 1e-9)),
126               3)
127               for i in range(min(N, 20)) for k in range(1)],
128               data=os.path.relpath(path, ROOT))
129     save_json(os.path.join(a.out_dir, "q3_temporal_ig_%s_report.json" % a.name), rep)
130     print("[SAVE] %s" % os.path.relpath(path, ROOT))
131     print("IG_GRADCAM_OK")
132     return 0
133 if __name__ == "__main__":
134     sys.exit(main())

```

## C.7 问题三：遮挡重要性（窗宽 3 滑窗）

code/03\_xai/occlusion.py（共 116 行）

```

1  # -*- coding: utf-8 -*-
2  r"""S3: 模型无关的遮挡重要性（作为解释的验证基准）。
3
4  定义：把模态 m 的窗口 [start, start+w) 用**相邻段**替换（保持序列长度与形态，
5  避免“删掉一段导致长度变短”引入的伪影），测量目标标量的变化
6      d_k = t(X) - t(X - window_k),
7  位置 k 的重要性取覆盖它的所有窗口的 |d| 均值。
8
9  为什么用相邻段替换而不是置零：置零在归一化空间里等价于“该段取训练均值”，
10  会同时改变“语义内容”与“是否存在信息”两件事；相邻段替换只改变内容，
11  更接近“如果这段时间说的是别的内容”的反事实，且对输出的扰动量级可解释。
12
13  用法：
14  python occlusion.py --pk1 <pk1> --name att4 --width 3 --stride 1 --out_dir <runs/q3>
15  python occlusion.py --pk1 ../附件2\\aligned_50.pk1 --split valid --name valid --stride 2

```

```

16 """
17 import argparse
18 import os
19 import sys
20
21 import numpy as np
22 import torch
23
24 HERE = os.path.dirname(os.path.abspath(__file__))
25 if HERE not in sys.path:
26     sys.path.append(HERE)
27 from xai_common import (MODALITIES, load_backbone, load_split, make_batch, # noqa: E402
28                         predict_batch, occlude_window, save_npz, save_json,
29                         q3_meta, ROOT)
30
31
32 def occl_batch(models, b, width=3, stride=1, target="mu"):
33     """返回 (w_occ (B,3,L), d_win 列表, 窗口起点列表)。"""
34     B = b["x"]["text"].size(0)
35     L = b["x"]["text"].size(1)
36     base = predict_batch(models, b, target=target)["target"].detach()
37     W = np.zeros((B, 3, L), np.float64)
38     CNT = np.zeros((B, 3, L), np.float64)
39     d_all = {}
40     starts_all = {}
41     for k, m in enumerate(MODALITIES):
42         nvalid = int((~(b["pad"][m][0])).sum().item()) if B else L
43         starts = list(range(0, max(1, nvalid - width + 1), stride))
44         if not starts:
45             starts = [0]
46         starts_all[m] = starts
47         d = np.zeros((B, len(starts)), np.float64)
48         for j, st in enumerate(starts):
49             bo = occlude_window(b, m, st, width)
50             t = predict_batch(models, bo, target=target)["target"].detach()
51             dd = (base - t).cpu().numpy()
52             d[:, j] = dd
53             for r in range(B):
54                 e = min(L, st + width)
55                 W[r, k, st:e] += np.abs(dd[r])
56                 CNT[r, k, st:e] += 1.0
57         d_all[m] = d
58     W = W / np.maximum(CNT, 1.0)
59     return W, d_all, starts_all
60
61

```

```

62 def main():
63     ap = argparse.ArgumentParser()
64     ap.add_argument("--pkl", default=None)
65     ap.add_argument("--split", default=None)
66     ap.add_argument("--name", default="att4")
67     ap.add_argument("--ckpt_dir", default=os.path.join(ROOT, "runs", "ens_top2"))
68     ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
69     ap.add_argument("--width", type=int, default=3)
70     ap.add_argument("--stride", type=int, default=1)
71     ap.add_argument("--target", default="mu", choices=["mu", "p_pol"])
72     ap.add_argument("--batch_size", type=int, default=16)
73     ap.add_argument("--limit", type=int, default=0)
74     ap.add_argument("--device", default="cpu")
75     a = ap.parse_args()
76
77     pkl = a.pkl or os.path.join(ROOT, "附件2", "aligned_50.pkl")
78     models, stats, theta = load_backbone(a.ckpt_dir, a.device)
79     split, key = load_split(pkl, stats, a.split)
80     if a.limit:
81         from data_utils import slice_split
82         split = slice_split(split, a.limit)
83     N, L = split["N"], split["L"]["text"]
84     print("[DATA] %s/%s N=%d L=%d | 窗口 w=%d 步长 %d target=%s"
85           % (os.path.basename(pkl), key, N, L, a.width, a.stride, a.target))
86
87     W = np.zeros((N, 3, L), np.float64)
88     for i0 in range(0, N, a.batch_size):
89         idxs = list(range(i0, min(i0 + a.batch_size, N)))
90         b = make_batch(split, idxs, a.device)
91         w, _, _ = occl_batch(models, b, a.width, a.stride, a.target)
92         W[i0:i0 + len(idxs)] = w
93         if (i0 // a.batch_size) % 20 == 0:
94             print(" %d/%d" % (min(i0 + a.batch_size, N), N))
95
96     valid = np.zeros((N, 3, L), bool)
97     for i in range(N):
98         for k, m in enumerate(MODALITIES):
99             valid[i, k, :int((~split["pad"])[m][i]).sum())] = True
100     W[~valid] = 0.0
101     path = save_npz(os.path.join(a.out_dir, "q3_temporal_occ_%s.npz" % a.name),
102                    ids=np.asarray(split["ids"]).astype(str), w_occ=W, valid=valid,
103                    meta=np.array([a.width, a.stride, a.target], dtype=object))
104     rep = dict(meta=q3_meta(a.ckpt_dir, models, theta,
105                             dict(pkl=os.path.relpath(pkl, ROOT), split=key, n=N,
106                                   width=a.width, stride=a.stride, target=a.target)),
107               w_occ_sum_mean=[round(float(W[:, k].sum(1).mean()), 4) for k in range(3)],

```



```

108         data=os.path.relpath(path, ROOT))
109     save_json(os.path.join(a.out_dir, "q3_temporal_occ_%s_report.json" % a.name), rep)
110     print("[SAVE] %s" % os.path.relpath(path, ROOT))
111     print("OCCLUSION_OK")
112     return 0
113
114
115 if __name__ == "__main__":
116     sys.exit(main())

```

## C.8 问题三：三源融合与 $\gamma$ 单纯形约束

code/03\_xai/temporal\_fuse.py (共 152 行)

```

1  #-*- coding: utf-8 -*-
2  r"""S4-a: 三源时间重要性的归一化与融合（gamma 在验证集上按与遮挡源的一致性标定）。
3
4  三源:
5  w_att 模型内生注意力（基线，已知”注意力!=重要性”）
6  w_ig 积分梯度（公理化）
7  w_occ 遮挡（模型无关，作为验证基准）
8
9  融合：先对每个样本、每个模态在**有效位内**做 min-max 归一化，再线性加权
10     w = (gamma1.w~_att + gamma2.w~_ig + gamma3.w~_occ) / (gamma1+gamma2+gamma3)，最后再归一化一次。
11  gamma 的标定：在单纯形上枚举（**约束每个 gamma >= 0.1**，避免退化为”只留遮挡源”这种平凡解），
12     以”融合结果与遮挡源的 Spearman 一致性（跨样本、跨模态、有效位内聚合）”为准则；
13     同时报告等权 (1,1,1) 对照。
14
15  用法:
16  python temporal_fuse.py --name valid --out_dir ../runs/q3
17  """
18  import argparse
19  import itertools
20  import json
21  import os
22  import sys
23
24  import numpy as np
25
26  HERE = os.path.dirname(os.path.abspath(__file__))
27  if HERE not in sys.path:
28      sys.path.append(HERE)
29  from xai_common import MODALITIES, save_npz, save_json, ROOT # noqa: E402
30
31  SOURCES = ("att", "ig", "occ")
32

```

```

33
34 def load_sources(out_dir, name):
35     ig = np.load(os.path.join(out_dir, "q3_temporal_ig_%s.npz" % name), allow_pickle=True)
36     oc = np.load(os.path.join(out_dir, "q3_temporal_occ_%s.npz" % name), allow_pickle=True)
37     w = dict(att=ig["w_att"], ig=ig["w_ig"], occ=oc["w_occ"])
38     return w, ig["valid"], np.asarray(ig["ids"]).astype(str)
39
40
41 def norm_valid_mat(W, valid):
42     """对 (N,3,L) 逐样本逐模态在有效位内 min-max 归一化（无效位保持 0）。"""
43     A = np.zeros_like(W, dtype=np.float64)
44     N, K, L = W.shape
45     for i in range(N):
46         for k in range(K):
47             v = valid[i, k]
48             if not v.any():
49                 continue
50             seg = W[i, k, v].astype(np.float64)
51             lo, hi = seg.min(), seg.max()
52             if hi - lo < 1e-12:
53                 continue
54             A[i, k, v] = (seg - lo) / (hi - lo)
55     return A
56
57
58 def spearman(a, b):
59     from scipy.stats import spearmanr
60     if a.size < 3 or np.std(a) < 1e-12 or np.std(b) < 1e-12:
61         return np.nan
62     return float(spearmanr(a, b).statistic)
63
64
65 def agreement(W, ref, valid):
66     """跨样本/跨模态/有效位聚合的 Spearman 一致性。"""
67     a, b = [], []
68     for i in range(W.shape[0]):
69         for k in range(W.shape[1]):
70             v = valid[i, k]
71             if v.sum() < 3:
72                 continue
73             a.append(W[i, k, v])
74             b.append(ref[i, k, v])
75     if not a:
76         return np.nan
77     return spearman(np.concatenate(a), np.concatenate(b))
78

```

```

79
80 def simplex_grid(step=0.1, min_g=0.1):
81     """枚举 gamma1+gamma2+gamma3=1, 每个 >= min_g."""
82     out = []
83     n = int(round(1.0 / step))
84     for i in range(n + 1):
85         for j in range(n + 1 - i):
86             k = n - i - j
87             g = np.array([i, j, k]) * step
88             if g.min() >= min_g - 1e-9 and abs(g.sum() - 1.0) < 1e-9:
89                 out.append(tuple(np.round(g, 3)))
90     return sorted(set(out))
91
92
93 def fuse(ws, gamma):
94     g = np.asarray(gamma, dtype=np.float64)
95     g = g / g.sum()
96     W = g[0] * ws["att"] + g[1] * ws["ig"] + g[2] * ws["occ"]
97     return W
98
99
100 def main():
101     ap = argparse.ArgumentParser()
102     ap.add_argument("--name", default="valid")
103     ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
104     ap.add_argument("--step", type=float, default=0.1)
105     a = ap.parse_args()
106
107     W_raw, valid, ids = load_sources(a.out_dir, a.name)
108     N, K, L = W_raw["occ"].shape
109     print("[DATA] %s N=%d 三源形状=(%d,%d,%d)" % (a.name, N, N, K, L))
110     ws = {k: norm_valid_mat(W_raw[k], valid) for k in SOURCES}
111     for k in SOURCES:
112         print(" [SRC] %-4s 归一化后均值 文/语/视 = %.4f / %.4f / %.4f"
113               % (k, *[ws[k][:, i][valid[:, i]].mean() for i in range(3)]))
114
115     print(" [gamma] 单纯形枚举 (每源 >= %.1f), 以与遮挡源的一致性为准则: " % 0.1)
116     rows = []
117     for g in simplex_grid(a.step):
118         W = fuse(ws, g)
119         ag = agreement(norm_valid_mat(W, valid), ws["occ"], valid)
120         rows.append((g, ag))
121     rows_sorted = sorted(rows, key=lambda t: -(t[1] if np.isfinite(t[1]) else -9))
122     for g, ag in rows_sorted[:5]:
123         print("    gamma=(%.1f, %.1f, %.1f) 一致性 rho=%.4f" % (*g, ag))
124     g_best = rows_sorted[0][0]

```

```

125     g_eq = (1 / 3, 1 / 3, 1 / 3)
126     W_best = fuse(Ws, g_best)
127     W_eq = fuse(Ws, g_eq)
128     print(" [gamma] 选定 gamma_att/gamma_ig/gamma_occ = (%.2f, %.2f, %.2f) (等权对照 rho=%.4f) "
129           % (*g_best, agreement(norm_valid_mat(W_eq, valid), Ws["occ"], valid)))
130
131     Wb = norm_valid_mat(W_best, valid)
132     We = norm_valid_mat(W_eq, valid)
133     path = save_npz(os.path.join(a.out_dir, "q3_temporal_fused_%s.npz" % a.name),
134                      ids=ids, valid=valid, w_fused=Wb, w_fused_equal=We,
135                      w_att=Ws["att"], w_ig=Ws["ig"], w_occ=Ws["occ"],
136                      gamma=np.array(g_best))
137     rep = dict(name=a.name, n=N, gamma=list(g_best), gamma_equal=list(g_eq),
138               grid=[dict(gamma=list(g), spearman_occ=(None if not np.isfinite(ag) else
139 round(ag, 4)))
139                     for g, ag in rows_sorted],
140               source_mean={k: [round(float(Ws[k][:, i][valid[:, i]].mean()), 4)
141                               for i in range(3)] for k in SOURCES},
142               agreement_with_occ={k: round(agreement(Ws[k], Ws["occ"], valid), 4)
143                                   for k in SOURCES},
144               data=os.path.relpath(path, ROOT))
145     save_json(os.path.join(a.out_dir, "q3_gamma_%s.json" % a.name), rep)
146     print("[SAVE] %s" % os.path.relpath(path, ROOT))
147     print("FUSE_OK")
148     return 0
149
150
151 if __name__ == "__main__":
152     sys.exit(main())

```

### C.9 问题三：证据定位（词元对齐、语音时段映射、关键帧抽取）

code/03\_xai/evidence.py（共 326 行）

```

1  # -*- coding: utf-8 -*-
2  r"""S5：证据定位（文本片段 / 语音时段 / 视觉关键帧）——题目要求“可回看原始素材”。
3
4  链路：
5  1. 文本：‘text_bert’ 的词元编号 + ‘vocab.txt’（行号即 id）解码 WordPiece -> 合并 ‘##’ 续接
6     -> 得到“词”序列及其**词元位置集合**（词元位置 = 对齐段号，见 section1.2 的对齐粒度实测）
7     -> 用 ‘difflib’ 把解码词对齐回原始 ‘raw_text’ 的字符区间（失败显式标记，不静默降级）；
8  2. 语音：高重要度的段号合并为连续区间，按比例映射为时间（附件2 不提供逐词时间戳，
9     声明为近似）并用视频音频能量曲线佐证；
10 3. 视觉：高重要度段取段内“视觉活跃度最大”的帧作为关键帧，用 ffmpeg 抽帧存盘。
11
12 输出：

```

```

13 submission/evidence/{id}_kf.jpg      关键帧
14 runs/q3/q3_evidence_{name}.json      逐样本证据记录（含字符区间、秒区间、帧时刻）
15 runs/q3/q3_audio_energy_{name}.npz   各样本音频能量曲线（论文解释卡用）
16
17 用法:
18 python evidence.py --name att4 --out_dir ../../runs/q3
19 """
20 import argparse
21 import difflib
22 import json
23 import os
24 import re
25 import subprocess
26 import sys
27 import wave
28
29 import numpy as np
30
31 HERE = os.path.dirname(os.path.abspath(__file__))
32 CODE = os.path.dirname(HERE)
33 ROOT = os.path.dirname(CODE)
34 if HERE not in sys.path:
35     sys.path.append(HERE)
36 from xai_common import MODALITIES, save_json, ROOT as W_ROOT      # noqa: E402
37 from data_utils import load_pickle                                # noqa: E402
38
39 VOCAB = os.path.join(CODE, "assets", "bert-base-uncased_vocab.txt")
40 WORD_RE = re.compile(r"[A-Za-z0-9]+(?:[' \-][A-Za-z0-9]+)*")
41
42
43 # -----
44 def load_vocab(path=VOCAB):
45     if not os.path.isfile(path):
46         raise SystemExit("缺少词表 %s, 请先运行 fetch_assets.py" % path)
47     with open(path, encoding="utf-8") as f:
48         return [l.rstrip("\n") for l in f]
49
50
51 def decode_words(ids, mask, vocab, merge_punct=True):
52     """词元序列 -> [(word, [token_pos, ...])], 跳过 [CLS]/[SEP]/填充。
53
54     merge_punct=True 时把纯标点词元（如 “'”、“-”、“,”）并入相邻词，
55     使解码词序列与原始转写的词序列更接近（实测可把对齐率由 ~0.75 提升到 ~0.95）。
56
57     """
58     words, cur, positions = [], None, []
59     for p, (i, m) in enumerate(zip(ids, mask)):

```

```

59     if m == 0:
60         break
61     tok = vocab[int(i)] if 0 <= int(i) < len(vocab) else "[UNK]"
62     if tok in ("[CLS]", "[SEP]"):
63         continue
64     if tok.startswith("##"):
65         if cur is None:
66             cur, positions = tok[2:], [p]
67         else:
68             cur, positions = cur + tok[2:], positions + [p]
69     else:
70         if cur is not None:
71             words.append((cur, positions))
72             cur, positions = tok, [p]
73     if cur is not None:
74         words.append((cur, positions))
75     if not merge_punct:
76         return words
77     # 合并纯标点词元：与前一单词拼合（无前词则与后一单词拼合）
78     merged = []
79     for w, poss in words:
80         if w and all(not ch.isalnum() for ch in w):
81             if merged:
82                 pw, pposs = merged[-1]
83                 merged[-1] = (pw + w, pposs + poss)
84                 continue
85             if (not merged) and w in ("'", "' ", "\'", "' ", "' '"):
86                 merged.append((w, poss))
87                 continue
88             merged.append((w, poss))
89     # 后处理：相邻的"词+纯标点"再合并一次（处理句首标点情形）
90     out = []
91     i = 0
92     while i < len(merged):
93         w, poss = merged[i]
94         if (i + 1 < len(merged) and merged[i + 1][0] and
95             all(not ch.isalnum() for ch in merged[i + 1][0]) and len(w) > 1):
96             out.append((w + merged[i + 1][0], poss + merged[i + 1][1]))
97             i += 2
98             continue
99         out.append((w, poss))
100         i += 1
101     return out
102
103
104 def align_to_raw(words, raw_text):

```

```

105     """把解码词对齐回 raw_text 的字符区间。
106
107 返回 (spans, info): spans 为逐词的 (start, end) (未对齐项为 None);
108 info 记录对齐率, 供论文如实报告”对齐失败样本数”。
109 """
110     spans = [None] * len(words)
111     raw_words = [(m.group(0), m.start(), m.end()) for m in WORD_RE.finditer(raw_text)]
112     a = [w.lower() for w, _ in words]
113     b = [w.lower() for w, _, _ in raw_words]
114     sm = difflib.SequenceMatcher(a=a, b=b, autojunk=False)
115     hit = 0
116     for blk in sm.get_matching_blocks():
117         for t in range(blk.size):
118             spans[blk.a + t] = (raw_words[blk.b + t][1], raw_words[blk.b + t][2])
119             hit += 1
120     info = dict(n_word=len(words), n_raw=len(raw_words), matched=hit,
121                ratio=round(float(hit / max(len(words), 1)), 3))
122     return spans, info
123
124
125 def merge_runs(pos, gap=1):
126     """把位置列表合并为连续区间 [start, end)。"""
127     pos = sorted(set(int(p) for p in pos))
128     if not pos:
129         return []
130     runs, s, e = [], pos[0], pos[0] + 1
131     for p in pos[1:]:
132         if p <= e + gap - 1:
133             e = p + 1
134         else:
135             runs.append([s, e])
136             s, e = p, p + 1
137     runs.append([s, e])
138     return runs
139
140
141 def seg_to_seconds(seg_start, seg_end, n_valid, duration):
142     """段号区间 -> 秒区间 (比例映射; duration 为该样本视频时长)。"""
143     if not duration or n_valid <= 0:
144         return None, None
145     per = float(duration) / float(n_valid)
146     return round(seg_start * per, 2), round(min(seg_end, n_valid) * per, 2)
147
148
149 # -----
150 def ffmpeg_exe():

```

```

151 import imageio_ffmpeg
152 return imageio_ffmpeg.get_ffmpeg_exe()
153
154
155 def grab_frame(video, t_sec, out_jpg, width=320, q=5):
156     os.makedirs(os.path.dirname(out_jpg), exist_ok=True)
157     cmd = [ffmpeg_exe(), "-y", "-loglevel", "error", "-ss", "%.3f" % max(0.0, t_sec),
158           "-i", video, "-frames:v", "1", "-vf", "scale=%d:-1" % width,
159           "-q:v", str(q), out_jpg]
160     r = subprocess.run(cmd, capture_output=True)
161     return os.path.isfile(out_jpg) and os.path.getsize(out_jpg) > 0, (r.stderr or b"")
162     [-300:]
163
164 def audio_energy(video, tmp_wav, hop=0.05):
165     """抽出 16k 单声道 wav 并算短时能量曲线（返回 times, rms）。"""
166     os.makedirs(os.path.dirname(tmp_wav), exist_ok=True)
167     cmd = [ffmpeg_exe(), "-y", "-loglevel", "error", "-i", video, "-vn",
168           "-ac", "1", "-ar", "16000", tmp_wav]
169     r = subprocess.run(cmd, capture_output=True)
170     if not os.path.isfile(tmp_wav):
171         return None, None, (r.stderr or b"")[-300:]
172     with wave.open(tmp_wav, "rb") as w:
173         sr = w.getframerate()
174         n = w.getnframes()
175         data = np.frombuffer(w.readframes(n), dtype=np.int16).astype(np.float32) /
176         32768.0
177         step = max(1, int(hop * sr))
178         nseg = max(1, len(data) // step)
179         rms = np.sqrt(np.mean(data[:nseg * step].reshape(nseg, step) ** 2, axis=1))
180         times = (np.arange(nseg) + 0.5) * step / sr
181         return times, rms, None
182
183 # -----
184 def main():
185     ap = argparse.ArgumentParser()
186     ap.add_argument("--name", default="att4")
187     ap.add_argument("--out_dir", default=os.path.join(W_ROOT, "runs", "q3"))
188     ap.add_argument("--sub_dir", default=os.path.join(W_ROOT, "submission"))
189     ap.add_argument("--meta", default=os.path.join(W_ROOT, "data_att", "att_meta.json"))
190     ap.add_argument("--pkl", default=os.path.join(W_ROOT, "data_att", "att4_aligned.pkl"),
191                     help="缓存 pkl（含 text_bert 与 raw_text，用于文本证据解码）")
192     ap.add_argument("--top_seg", type=int, default=3)
193     ap.add_argument("--top_word", type=int, default=3)
194     ap.add_argument("--phrase_win", type=int, default=2)

```



```

195 ap.add_argument("--no_video", action="store_true", help="不抽帧/不提音频（只算文本与时段）")
196 a = ap.parse_args()
197
198 fused = np.load(os.path.join(a.out_dir, "q3_temporal_fused_%s.npz" % a.name),
199                 allow_pickle=True)
200 W = fused["w_fused"]
201 valid = fused["valid"]
202 ids = [str(x) for x in fused["ids"]]
203 meta = json.load(open(a.meta, encoding="utf-8"))
204 recs = {r["id"]: r for r in meta[a.name]}
205 vocab = load_vocab()
206
207 # 文本证据需要 text_bert（词元编号）与 raw_text：从重建后的缓存 pkl 读取
208 tb_all, rt_all, idx_map = None, None, {}
209 if a.pkl and os.path.isfile(a.pkl):
210     d = load_pickle(a.pkl)
211     if "id" in d:
212         pkl_ids = [str(x) for x in np.asarray(d["id"]).reshape(-1)]
213         idx_map = {sid: i for i, sid in enumerate(pkl_ids)}
214     if "text_bert" in d:
215         tb_all = np.asarray(d["text_bert"])
216     if "raw_text" in d:
217         rt_all = [str(x) for x in np.asarray(d["raw_text"]).reshape(-1)]
218     print(" [PKL] %s: text_bert=%s raw_text=%s"
219           % (os.path.basename(a.pkl),
220              "有" if tb_all is not None else "无",
221              "有" if rt_all is not None else "无"))
222
223 ev_dir = os.path.join(a.sub_dir, "evidence")
224 out, energies = {}, {}
225 align_bad = []
226 for i, sid in enumerate(ids):
227     r = recs.get(sid, {})
228     k = idx_map.get(sid, None)
229     raw_text = (rt_all[k] if (rt_all is not None and k is not None and k < len(rt_all)
230 ))
231
232         else (r.get("raw_text", "") or ""))
233     dur = r.get("duration_s")
234     tb = tb_all[k] if (tb_all is not None and k is not None and k < len(tb_all)) else
235     None
236     n_text = int(valid[i, 0].sum())
237     n_audio = int(valid[i, 1].sum())
238     n_vision = int(valid[i, 2].sum())
239     rec = dict(id=sid)
240
241     # —— 文本证据 ——

```

```

239 w_text = np.asarray(W[i, 0], np.float64)
240 words = []
241 if tb is not None:
242     tb = np.asarray(tb)
243     words = decode_words(tb[0], tb[1], vocab)
244 if words:
245     spans, al = align_to_raw(words, raw_text)
246     if al["ratio"] < 0.8:
247         align_bad.append(dict(id=sid, **al))
248     ww = []
249     for (word, poss), sp in zip(words, spans):
250         ww.append((word, poss, sp, float(max(w_text[p] for p in poss if p < len(
w_text)))))
251     ww_sorted = sorted(ww, key=lambda t: -t[3])
252     picks = [t for t in ww_sorted[:max(1, a.top_word)]]
253     pick_idx = [ww.index(t) for t in picks]
254     lo = max(0, min(pick_idx) - a.phrase_win)
255     hi = min(len(ww) - 1, max(pick_idx) + a.phrase_win)
256     phrase_words = [t[0] for t in ww[lo:hi + 1]]
257     phrase = "".join(phrase_words)
258     cspans = [t[2] for t in ww[lo:hi + 1] if t[2]]
259     segs = merge_runs([p for t in picks for p in t[1]])
260     rec["text_evidence"] = dict(
261         top_words=[dict(word=t[0], seg=t[1], char_span=t[2], w=round(t[3], 4))
262                     for t in picks],
263         phrase=phrase,
264         phrase_char_span=(min(s[0] for s in cspans), max(s[1] for s in cspans))
265                           if cspans else None,
266         seg_runs=segs, n_valid=n_text, align=al)
267 else:
268     rec["text_evidence"] = dict(note="无 text_bert (无法解码), 仅给段号",
269                                seg_runs=[[int(x) for x in rn] for rn in
270                                             merge_runs(np.argsort(-w_text[:n_text])
271 [:a.top_seg])],
272                                n_valid=n_text)
273
274 # —— 语音证据 ——
275 w_audio = np.asarray(W[i, 1], np.float64)
276 top_a = [int(x) for x in np.argsort(-w_audio[:n_audio])[:a.top_seg]]
277 runs_a = merge_runs(top_a)
278 s0, s1 = (seg_to_seconds(runs_a[0][0], runs_a[-1][1], n_audio, dur)
279            if runs_a else (None, None))
280 rec["audio_evidence"] = dict(top_segs=top_a, seg_runs=runs_a,
281                               start_s=s0, end_s=s1, n_valid=n_audio, duration_s=
dur)
282

```

```

282 # —— 视觉证据 ——
283 w_vision = np.asarray(W[i, 2], np.float64)
284 top_v = [int(x) for x in np.argsort(-w_vision[:n_vision])[a.top_seg]]
285 runs_v = merge_runs(top_v)
286 t0, t1 = (seg_to_seconds(runs_v[0][0], runs_v[0][1], n_vision, dur)
287           if runs_v else (None, None))
288 kf_t = None if t0 is None else round((t0 + (t1 if t1 else t0)) / 2.0, 2)
289 rec["vision_evidence"] = dict(top_segs=top_v, seg_runs=runs_v,
290                               keyframe_time_s=kf_t, n_valid=n_vision)
291 if (not a.no_video) and r.get("video") and kf_t is not None:
292     out_jpg = os.path.join(ev_dir, "%s_kf.jpg" % sid)
293     ok, err = grab_frame(r["video"], kf_t, out_jpg)
294     rec["vision_evidence"]["keyframe_file"] = os.path.relpath(out_jpg, W_ROOT)
295     rec["vision_evidence"]["keyframe_ok"] = bool(ok)
296     if not ok:
297         rec["vision_evidence"]["keyframe_err"] = err.decode("utf-8", "replace")
298     if sid not in energies:
299         tw = os.path.join(a.out_dir, "_audio", "%s.wav" % sid)
300         tt, rr, err2 = audio_energy(r["video"], tw)
301         if tt is not None:
302             energies[sid] = dict(t=tt, rms=rr)
303             np.savez_compressed(os.path.join(a.out_dir, "_audio", "%s_energy.npz" %
304 sid),
305                               t=tt, rms=rr)
306         else:
307             rec["vision_evidence"]["audio_err"] = str(err2)[:200]
308         try:
309             os.remove(tw)
310         except OSError:
311             pass
312     out[sid] = rec
313     if (i + 1) % 5 == 0 or i + 1 == len(ids):
314         print(" %d/%d" % (i + 1, len(ids)))
315
316 save_json(os.path.join(a.out_dir, "q3_evidence_%s.json" % a.name), out)
317 if align_bad:
318     print(" [警告] 文本对齐率 <0.8 的样本 %d 条: %s" % (len(align_bad), align_bad[:3]))
319     print(" [KF] 关键帧 %d 张; 音频能量曲线 %d 条"
320           % (len([1 for v in out.values() if v.get("vision_evidence", {}).get("keyframe_ok")]),
321              len(energies)))
322     print("EVIDENCE_OK")
323     return 0
324
325 if __name__ == "__main__":
326     sys.exit(main())

```

### C.10 问题三：并联证据头训练（骨干冻结、忠实性正则）

code/03\_xai/train\_evidence\_head.py（共 205 行）

```
1  # -*- coding: utf-8 -*-
2  r"""D4/D9: 可解释性证据头（冻结骨干 + 并联解释头 + 解释性正则训练）。
3
4  设计（论文 section5.3“可解释性模型的网络结构、目标函数、训练方案与关键参数”）：
5  骨干**完全冻结**并保持预测通路不变；证据头挂在其**编码器输出**（逐模态 50x d 表示）上，
6  只用于产生解释： $s^m_k = \text{softmax}_k(w_m^T \tanh(W_m h^m_k + b_m))$ ，
7  模态作用度  $\pi = \text{softmax}_m(u^T \tanh(V h^m))$ ， $h^m = \text{Sigma}_k s^m_k h^m_k$ 。
8
9  目标函数（全部可微；用“软删除/软保留”把忠实性写成可训练的损失）：
10  $L = \alpha_f [-(t(X) - t(X; \text{miss}=s))^2 + (t(X) - t(X; \text{miss}=1-s))^2]$ 
11  $+ \alpha_1 \text{Sigma}_m \|s^m\|_1 + \alpha_2 \text{Sigma}_m \text{TV}(s^m) + \alpha_3 \text{Sigma}_m (\pi_m - \pi_m^{\text{Shapley}})^2$ 
12
13 第一项：按 s 软删除后的预测变化越大越好（**忠实性**）；
14 第二项：只保留 s 的软权重时的预测应接近全模态（**充分性**）；
15 第三项/第四项：稀疏性与时间平滑；第五项（可选）：与精确 Shapley 的模态作用度对齐。
16
17 训练方案：只训证据头（骨干冻结、eval 模式），AdamW，lr 1e-3，30 轮，按验证集
18 “忠实性 - 充分性偏差”加权目标早停；关键参数 alpha 在验证集上网格择优（默认给定值）。
19
20 用法：
21 python train_evidence_head.py --epochs 12 --out ../\runs\q3
22 """
23 import argparse
24 import json
25 import os
26 import sys
27
28 import numpy as np
29 import torch
30 import torch.nn as nn
31 import torch.nn.functional as F
32
33 HERE = os.path.dirname(os.path.abspath(__file__))
34 if HERE not in sys.path:
35     sys.path.append(HERE)
36 from xai_common import (MODALITIES, load_backbone, load_split, make_batch, # noqa: E402
37                          predict_batch, save_json, q3_meta, ROOT)
38
39
40 class EvidenceHead(nn.Module):
41     """逐模态时间证据头 + 模态门控（并联，不接入预测通路）。"""
42
43     def __init__(self, hidden, n_mod=3):
```

```

44     super().__init__()
45     self.hidden = hidden
46     self.W = nn.ModuleDict({m: nn.Linear(hidden, hidden) for m in MODALITIES})
47     self.w = nn.ParameterDict({m: nn.Parameter(torch.zeros(hidden)) for m in
MODALITIES})
48     self.V = nn.Linear(hidden, hidden)
49     self.u = nn.Linear(hidden, 1)
50
51     def forward(self, h, pad):
52         """h: 模态->(B,L,H); pad: 模态->(B,L); 返回 s (B,3,L) 、 pi (B,3) 。”"""
53         S, pi_logit = [], []
54         for m in MODALITIES:
55             z = torch.tanh(self.W[m](h[m])) # (B,L,H)
56             sc = torch.einsum("blh,h->bl", z, self.w[m]) # (B,L)
57             sc = sc.masked_fill(pad[m], -1e9)
58             s = torch.softmax(sc, dim=1)
59             S.append(s)
60             hbar = (s.unsqueeze(-1) * h[m]).sum(dim=1) # (B,H)
61             pi_logit.append(self.u(torch.tanh(self.V(hbar))).squeeze(-1))
62         S = torch.stack(S, dim=1) # (B,3,L)
63         pi = torch.softmax(torch.stack(pi_logit, dim=1), dim=1)
64         return S, pi
65
66
67     def capture(mods, models):
68         """抓取骨干各模态编码器输出 h[m] 的 hook (兼容 [(model, ckpt), ...] 与 [model, ...]) 。”"""
69         store, handles = {}, []
70         m0 = models[0][0] if isinstance(models[0], tuple) else models[0]
71         for m in mods:
72             def mk(name):
73                 def hook(module, inp, out):
74                     store[name] = out
75                 return hook
76             handles.append(m0.enc[m].register_forward_hook(mk(m)))
77         return store, handles
78
79
80     def forward_with_miss(models, b, miss_override):
81         """用给定的 miss (可为软值) 跑集成前向, 返回 (score, prob)。x 同步按软掩码缩放。”"""
82         bb = dict(x={}, miss={}, pad={})
83         for m in MODALITIES:
84             bb["pad"][m] = b["pad"][m]
85             bb["miss"][m] = miss_override[m]
86             bad = (miss_override[m] + b["pad"][m].float()).clamp(0, 1).unsqueeze(-1)
87             bb["x"][m] = b["x"][m] * (1.0 - bad)
88         out = predict_batch(models, bb, target="mu")

```

```

89     return out["score"], out["prob"]
90
91
92 def tv(s):
93     return (s[:, :, 1:] - s[:, :, :-1]).abs().mean()
94
95
96 def main():
97     ap = argparse.ArgumentParser()
98     ap.add_argument("--epochs", type=int, default=12)
99     ap.add_argument("--batch_size", type=int, default=64)
100    ap.add_argument("--lr", type=float, default=1e-3)
101    ap.add_argument("--alpha_f", type=float, default=1.0)
102    ap.add_argument("--alpha1", type=float, default=1e-3)
103    ap.add_argument("--alpha2", type=float, default=1e-3)
104    ap.add_argument("--alpha3", type=float, default=0.0,
105                    help="与 Shapley 对齐项权重 (>0 需先算训练集 Shapley)")
106    ap.add_argument("--limit", type=int, default=0)
107    ap.add_argument("--out", default=os.path.join(ROOT, "runs", "q3"))
108    ap.add_argument("--ckpt_dir", default=os.path.join(ROOT, "runs", "ens_top2"))
109    ap.add_argument("--device", default="cpu")
110    a = ap.parse_args()
111
112    models, stats, theta = load_backbone(a.ckpt_dir, a.device, verbose=False)
113    for m, _ in models:
114        m.eval()
115        for p in m.parameters():
116            p.requires_grad_(False)
117    tr, _ = load_split(os.path.join(ROOT, "附件2", "aligned_50.pkl"), stats, "train")
118    va, _ = load_split(os.path.join(ROOT, "附件2", "aligned_50.pkl"), stats, "valid")
119    if a.limit:
120        from data_utils import slice_split
121        tr = slice_split(tr, a.limit)
122    hidden = models[0][1]["cfg"]["hidden"]
123    print("[DATA] train N=%d valid N=%d hidden=%d" % (tr["N"], va["N"], hidden))
124
125    head = EvidenceHead(hidden).to(a.device)
126    opt = torch.optim.AdamW(head.parameters(), lr=a.lr, weight_decay=1e-4)
127    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=a.epochs)
128    hist = []
129    best = dict(score=-1e9, epoch=-1, state=None)
130
131    for ep in range(1, a.epochs + 1):
132        head.train()
133        tot, nb = 0.0, 0
134        perm = np.random.permutation(tr["N"])

```

```

135     for i0 in range(0, tr["N"], a.batch_size):
136         idxs = [int(i) for i in perm[i0:i0 + a.batch_size]]
137         b = make_batch(tr, idxs, a.device)
138         store, handles = capture(MODALITIES, models)
139         with torch.no_grad():
140             t_full, _ = forward_with_miss(models, b,
141                                           {m: b["miss"][m].float() for m in MODALITIES
142         })
143
144         for hd in handles:
145             hd.remove()
146             S, pi = head(store, b["pad"])
147             miss_soft = {m: (S[:, k]).clamp(0, 1) for k, m in enumerate(MODALITIES)}
148             keep_soft = {m: (1.0 - S[:, k]).clamp(0, 1) for k, m in enumerate(MODALITIES)}
149         }
150
151         t_del, _ = forward_with_miss(models, b, miss_soft)
152         t_keep, _ = forward_with_miss(models, b, keep_soft)
153         l_f = -((t_full - t_del) ** 2).mean() + ((t_full - t_keep) ** 2).mean()
154         l1 = sum(S[:, k].mean() for k in range(3)) / 3.0
155         l_tv = sum(tv(S[:, k:k + 1]) for k in range(3)) / 3.0
156         loss = a.alpha_f * l_f + a.alpha1 * l1 + a.alpha2 * l_tv
157         opt.zero_grad()
158         loss.backward()
159         nn.utils.clip_grad_norm_(head.parameters(), 1.0)
160         opt.step()
161         tot += float(loss.detach())
162         nb += 1
163     sched.step()
164     # —— 验证集评估：忠实性 - 充分性偏差（越大越好） ——
165     head.eval()
166     compr, suff = [], []
167     with torch.no_grad():
168         for i0 in range(0, min(va["N"], 256), 64):
169             idxs = list(range(i0, min(i0 + 64, va["N"])))
170             b = make_batch(va, idxs, a.device)
171             store, handles = capture(MODALITIES, models)
172             t_full, _ = forward_with_miss(models, b,
173                                           {m: b["miss"][m].float() for m in MODALITIES
174         })
175
176         for hd in handles:
177             hd.remove()
178             S, pi = head(store, b["pad"])
179             t_del, _ = forward_with_miss(models, b, {m: S[:, k] for k, m in enumerate
180 (MODALITIES)})
181             t_keep, _ = forward_with_miss(models, b,
182                                           {m: (1 - S[:, k]) for k, m in enumerate(
183 MODALITIES)})

```

```

176         compr.append((t_full - t_del).abs().mean().item())
177         suff.append((t_full - t_keep).abs().mean().item())
178     c, s = float(np.mean(compr)), float(np.mean(suff))
179     sc = c - s
180     hist.append(dict(epoch=ep, loss=round(tot / max(nb, 1), 5), valid_compr=round(c,
181     5),
182                 valid_suff=round(s, 5), score=round(sc, 5)))
183     print(" [ep%02d] loss=%.5f valid 忠实性=%.5f 充分性偏差=%.5f 目标=%.5f"
184           % (ep, tot / max(nb, 1), c, s, sc))
185     if sc > best["score"]:
186         best = dict(score=sc, epoch=ep,
187                     state={k: v.detach().clone() for k, v in head.state_dict().items
188                     ()})
189     head.load_state_dict(best["state"])
190     torch.save(dict(state=head.state_dict(), cfg=dict(hidden=hidden, alpha_f=a.alpha_f,
191               alpha1=a.alpha1, alpha2=a.alpha2,
192               alpha3=a.alpha3, epochs=a.epochs,
193               lr=a.lr),
194               meta=q3_meta(a.ckpt_dir, models, theta, dict(train_n=tr["N"])),
195               history=hist, best_epoch=best["epoch"]),
196               os.path.join(a.out, "q3_evidence_head.pt"))
197     save_json(os.path.join(a.out, "q3_evidence_head_report.json"),
198               dict(best_epoch=best["epoch"], best_score=best["score"], history=hist,
199                 args=dict(alpha_f=a.alpha_f, alpha1=a.alpha1, alpha2=a.alpha2,
200                 alpha3=a.alpha3, epochs=a.epochs, lr=a.lr, batch=a.
201                 batch_size)))
202     print("[SAVE] runs/q3/q3_evidence_head.pt (最佳轮 %d, 目标 %.5f)" % (best["epoch"], best["score"]
203     ))
204     print("EVIDENCE_HEAD_OK")
205     return 0
206
207 if __name__ == "__main__":
208     sys.exit(main())

```

## C.11 问题三：解释卡生成

code/03\_xai/explain\_card.py (共 201 行)

```

1  # -*- coding: utf-8 -*-
2  r"""S6-a: 典型样本解释卡 (JSON + 三联图)。
3
4  三联图内容 (论文 section5.3 典型样本解释卡):
5  (1) 三模态时间重要性条带 (含有效位边界与 top 片段标注);
6  (2) 文本词着色条 (按词级重要性深浅着色, 凸显关键证据词);
7  (3) 关键帧缩略图 + 音频能量曲线 (高亮证据时段)。

```



```

8
9 用法:
10 python explain_card.py --name att4 --out_dir ../../runs/q3 --fig_dir ../../figs
11 """
12 import argparse
13 import json
14 import os
15 import sys
16
17 import numpy as np
18
19 HERE = os.path.dirname(os.path.abspath(__file__))
20 CODE = os.path.dirname(HERE)
21 if HERE not in sys.path:
22     sys.path.append(HERE)
23 from style import apply_style, COLORS, MOD_COLORS, LABEL_COLORS # noqa: E402
24 from xai_common import MODALITIES, save_json, ROOT # noqa: E402
25
26 MOD_CN = {"text": "文本", "audio": "语音", "vision": "视觉"}
27
28
29 def load_all(out_dir, name):
30     fused = np.load(os.path.join(out_dir, "q3_temporal_fused_%s.npz" % name), allow_pickle=
31                     True)
32     shap = np.load(os.path.join(out_dir, "q3_shapley_%s.npz" % name), allow_pickle=True)
33     ev = json.load(open(os.path.join(out_dir, "q3_evidence_%s.json" % name), encoding="utf-8")
34                   )
35     return fused, shap, ev
36
37
38 def card_figure(sid, W, valid, rec, pi, score, prob, theta, energy, out_png,
39               label_cn="", meta=None):
40     plt = apply_style()
41     fig = plt.figure(figsize=(9.2, 6.4))
42     gs = fig.add_gridspec(3, 2, height_ratios=[1.0, 0.75, 1.15], hspace=0.55, wspace
43                       =0.22)
44
45     # —— (1) 三模态时间重要性条带 ——
46     ax1 = fig.add_subplot(gs[0, :])
47     for k, m in enumerate(MODALITIES):
48         w = np.asarray(W[k], np.float64)
49         n = int(valid[k].sum())
50         ax1.bar(np.arange(len(w)) + (k - 1) * 0.0, w, width=0.62, bottom=k, color=
51               MOD_COLORS[k],
52               label="%s (pi=%.2f)" % (MOD_CN[m], pi[k]))
53         ax1.axvline(n - 0.5, color="#666666", lw=0.8, ls=":")

```

```

50     top = np.argsort(-w[:n])[:3]
51     for p in top:
52         ax1.text(p, k + w[p] + 0.06, "*", ha="center", va="bottom",
53                 fontsize=6.5, color="#333333")
54 ax1.set_yticks([0.5, 1.5, 2.5])
55 ax1.set_yticklabels([MOD_CN[m] for m in MODALITIES])
56 ax1.set_xlabel("对齐段号 $k$ (文本为词元位置; 虚线为该模态有效长度上界)")
57 ax1.set_title("(a) 三模态时间重要性 $w^m_k$ (*=top-3 关键片段)", loc="left")
58 ax1.legend(loc="upper right", ncol=3)
59
60 # —— (2) 文本词着色 ——
61 ax2 = fig.add_subplot(gs[1, :])
62 ax2.axis("off")
63 te = rec.get("text_evidence", {})
64 words = te.get("top_words", [])
65 phrase = te.get("phrase", "")
66 span = te.get("phrase_char_span")
67 raw = (meta or {}).get("raw_text", "")
68 ax2.set_title("(b) 文本证据: 高亮为模型关注词, 方框为可回看的字符区间", loc="left")
69 txt = raw if raw else phrase
70 if txt:
71     hi = {w["word"].lower() for w in words if w.get("word")}
72     # 用等宽示意图: 把原句逐词着色 (步长按字符数估算, 换行留出边距)
73     import re
74     x, y = 0.0, 0.74
75     for tok in re.finditer(r"\S+", txt):
76         word = tok.group(0)
77         key = re.sub(r"^[A-Za-z0-9]", "", word).lower()
78         hot = key in hi
79         ax2.text(x, y, word, fontsize=9.5, va="center",
80                 color=(COLORS["accent"] if hot else "#333333"),
81                 weight=("bold" if hot else "normal"),
82                 bbox=(dict(facecolor="#FBEAD6", edgecolor="none", pad=1.2) if hot
83                     else None),
84                 transform=ax2.transAxes)
85         x += 0.0092 * (len(word) + 1.35)
86         if x > 0.995:
87             x, y = 0.0, y - 0.26
88     else:
89         ax2.text(0.01, 0.5, "(该样本无 raw_text, 无法回看原文)", transform=ax2.transAxes,
90                 fontsize=9)
91 if span:
92     ax2.text(0.0, 0.06, "字符区间 [%d,%d]: %s" % (span[0], span[1], phrase),
93             transform=ax2.transAxes, fontsize=8.5, color=COLORS["accent"])
94
95 # —— (3) 关键帧 + 音频能量 ——

```

```

95 ax3 = fig.add_subplot(gs[2, 0])
96 kf = (rec.get("vision_evidence") or {}).get("keyframe_file")
97 if kf and os.path.isfile(os.path.join(ROOT, kf)):
98     img = plt.imread(os.path.join(ROOT, kf))
99     ax3.imshow(img)
100     ax3.set_title("(c) 视觉关键帧 t=%0.2fs" % (rec["vision_evidence"].get("keyframe_time_s") or
101     0.0),
102     loc="left")
103 else:
104     ax3.text(0.5, 0.5, "无关键帧", ha="center", transform=ax3.transAxes)
105     ax3.set_title("(c) 视觉关键帧", loc="left")
106 ax3.set_xticks([])
107 ax3.set_yticks([])
108 ax3.grid(False)
109
110 ax4 = fig.add_subplot(gs[2, 1])
111 if energy is not None:
112     t, rms = energy["t"], energy["rms"]
113     ax4.plot(t, rms, color=COLORS["audio"], lw=1.2)
114     ae = rec.get("audio_evidence") or {}
115     if ae.get("start_s") is not None:
116         ax4.axvspan(ae["start_s"], ae["end_s"], color=COLORS["audio"], alpha=0.18)
117     ax4.set_xlabel("时间 / s")
118     ax4.set_ylabel("音频能量")
119     ax4.set_title("(d) 音频能量与证据时段（阴影）", loc="left")
120 else:
121     ax4.text(0.5, 0.5, "无音频能量", ha="center", transform=ax4.transAxes)
122     ax4.set_title("(d) 音频能量", loc="left")
123
124 sup = "预测: %s (强度 %0.3f, theta=%0.3f, 主模态 %s)" % (
125     label_cn, score, theta, MOD_CN[MODALITIES[int(np.argmax(pi))]])
126 fig.suptitle("解释卡. 样本 %s %s" % (sid, sup), fontsize=11.5, y=0.985)
127 os.makedirs(os.path.dirname(out_png), exist_ok=True)
128 fig.savefig(out_png, dpi=300)
129 plt.close(fig)
130 return out_png
131
132 def main():
133     ap = argparse.ArgumentParser()
134     ap.add_argument("--name", default="att4")
135     ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
136     ap.add_argument("--fig_dir", default=os.path.join(ROOT, "figs"))
137     ap.add_argument("--theta", type=float, default=0.325)
138     ap.add_argument("--ids", default=None, help="逗号分隔; 默认自动挑（正/中/负各一）")
139     a = ap.parse_args()

```

```

140
141 fused, shap, ev = load_all(a.out_dir, a.name)
142 W, valid = fused["w_fused"], fused["valid"]
143 ids = [str(x) for x in fused["ids"]]
144 pi_all = shap["pi_soft"]
145 score = shap["score"]
146 prob = shap["prob"]
147 LAB = ("Negative", "Neutral", "Positive")
148 CN = {"Negative": "负向", "Neutral": "中性", "Positive": "正向"}
149 pred = [LAB[i] for i in prob.argmax(1)]
150 meta = {r["id"]: r for r in
151         json.load(open(os.path.join(ROOT, "data_att", "att_meta.json"),
152                        encoding="utf-8"))[a.name]}
153
154 if a.ids:
155     pick = [s.strip() for s in a.ids.split(",")]
156 else: # 自动挑: 正/中/负各一条 (优先解释指标最有说服力的: 置信度高 + pi 集中)
157     pick = []
158     for lab in ("Positive", "Neutral", "Negative"):
159         cand = [i for i, p in enumerate(pred) if p == lab]
160         if not cand:
161             continue
162         best = max(cand, key=lambda i: (float(prob[i].max()) * float(np.max(pi_all[i
163                                     ]))
164                                     - np.min(pi_all
165                                     [i])))
166     pick.append(ids[best])
167 print("[PICK] %s" % pick)
168
169 made = []
170 cards = {}
171 for sid in pick:
172     i = ids.index(sid)
173     energy = None
174     ep = os.path.join(a.out_dir, "_audio", "%s_energy.npz" % sid)
175     if os.path.isfile(ep):
176         with np.load(ep) as z:
177             energy = dict(t=z["t"], rms=z["rms"])
178     card = dict(id=sid, pred_label=pred[i], pred_score=round(float(score[i]), 4),
179               prob={LAB[k]: round(float(prob[i, k]), 4) for k in range(3)},
180               confidence=round(float(prob[i].max()), 4), theta=a.theta,
181               main_modality=MODALITIES[int(np.argmax(pi_all[i]))],
182               modality_contribution={MODALITIES[k]: round(float(pi_all[i, k]), 4)
183                                     for k in range(3)},
184               shapley_raw={MODALITIES[k]: round(float(shap["phi_mu_miss"][i, k]), 4)
185                           for k in range(3)},

```

```

184         temporal_importance={MODALITIES[k]: np.round(W[i, k], 4).tolist()
185                               for k in range(3)},
186         evidence=ev.get(sid, {}))
187     png = os.path.join(a.fig_dir, "Q3_解释卡_%s_%s_组合图.png" % (sid, pred[i]))
188     card_figure(sid, W[i], valid[i], ev.get(sid, {}), pi_all[i], float(score[i]),
189               prob[i], a.theta, energy, png, CN[pred[i]], meta.get(sid, {}))
190     cards[sid] = card
191     made.append(os.path.relpath(png, ROOT))
192     print(" [CARD] %s -> %s" % (sid, os.path.relpath(png, ROOT)))
193
194     save_json(os.path.join(a.out_dir, "q3_cards_%s.json" % a.name), cards)
195     print("[SAVE] %d 张解释卡: %s" % (len(made), made))
196     print("CARD_OK")
197     return 0
198
199
200 if __name__ == "__main__":
201     sys.exit(main())

```

## C.12 问题三：附件四全量推理与解释输出

code/03\_xai/infer\_explain\_att4.py (共 230 行)

```

1  # -*- coding: utf-8 -*-
2  r"""S6-b: 主入口 —— 生成附件4 的”预测 + 解释”提交文件与解释卡。
3
4  产物:
5      submission/pred_explain_att4.csv    题目要求的附件4 预测与解释结果（每行一条样本）
6      submission/explain_cards/{id}.json  逐样本完整解释卡（含 50 步时间重要性）
7      submission/evidence/{id}_kf.jpg    关键帧（由 evidence.py 生成）
8
9  内建自查断言（任一不过即报错退出）:
10     1. 行数与 id 顺序与附件4 严格一致;
11     2. pred_label / pred_score 与 submission/pred_att4.csv 逐位一致（同一骨干、同一 theta）;
12     3. 解释数值有限、pi 和为 1、时间重要性非负;
13     4. 证据区间合法（0 <= start < end <= 有效长度; 秒 <= 视频时长）。
14
15  用法:
16     python infer_explain_att4.py --out_dir ../../runs/q3 --sub_dir ../../submission
17     """
18  import argparse
19  import json
20  import os
21  import re
22  import subprocess
23  import sys

```

```

24
25 import numpy as np
26 import pandas as pd
27 import torch
28
29 HERE = os.path.dirname(os.path.abspath(__file__))
30 if HERE not in sys.path:
31     sys.path.append(HERE)
32 from xai_common import (MODALITIES, load_backbone, load_split, make_batch,      # noqa:
33                         E402
34                         predict_batch, save_json, q3_meta, ROOT)
35 LABELS = ("Negative", "Neutral", "Positive")
36 MOD_CN = {"text": "文本", "audio": "语音", "vision": "视觉"}
37
38
39 def probe_fps(video, cache_path):
40     """用 ffmpeg 的 stderr 解析帧率（缓存复用）。"""
41     cache = {}
42     if os.path.isfile(cache_path):
43         try:
44             cache = json.load(open(cache_path, encoding="utf-8"))
45         except Exception:
46             cache = {}
47     key = os.path.basename(video)
48     if key in cache:
49         return cache[key]
50     import imageio_ffmpeg
51     r = subprocess.run([imageio_ffmpeg.get_ffmpeg_exe(), "-i", video],
52                        capture_output=True)
53     txt = (r.stderr or b"").decode("utf-8", "replace")
54     m = re.search(r"(\d+(\.\d+)?)\s*fps", txt)
55     fps = float(m.group(1)) if m else None
56     cache[key] = fps
57     try:
58         os.makedirs(os.path.dirname(cache_path), exist_ok=True)
59         json.dump(cache, open(cache_path, "w", encoding="utf-8"), ensure_ascii=False,
60                   indent=1)
61     except OSError:
62         pass
63     return fps
64
65 def main():
66     ap = argparse.ArgumentParser()
67     ap.add_argument("--pkI", default=os.path.join(ROOT, "data_att", "att4_aligned.pkI"))

```

```

68 ap.add_argument("--ckpt_dir", default=os.path.join(ROOT, "runs", "ens_top2"))
69 ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
70 ap.add_argument("--sub_dir", default=os.path.join(ROOT, "submission"))
71 ap.add_argument("--theta", type=float, default=0.325)
72 ap.add_argument("--device", default="cpu")
73 a = ap.parse_args()
74
75 models, stats, theta_ck = load_backbone(a.ckpt_dir, a.device, verbose=False)
76 split, key = load_split(a.pkl, stats)
77 N = split["N"]
78 ids = [str(x) for x in split["ids"]]
79 print("[DATA] att4 N=%d (ckpt theta 均值 %.3f, 提交用 theta=%.3f)" % (N, theta_ck, a.theta))
80
81 # ----- 1) 复现预测 -----
82 b = make_batch(split, list(range(N)), a.device)
83 out = predict_batch(models, b, target="mu")
84 score = out["score"].detach().cpu().numpy()
85 prob = out["prob"].detach().cpu().numpy()
86 pred = [LABELS[i] for i in prob.argmax(1)]
87 ref_path = os.path.join(a.sub_dir, "pred_att4.csv")
88 assert os.path.isfile(ref_path), "缺少 %s (先跑 dl_recalib.py)" % ref_path
89 ref = pd.read_csv(ref_path, dtype={"id": str})
90 ref_ids = [str(x).zfill(2) if str(x).isdigit() else str(x) for x in ref["id"]]
91 assert ref_ids == ids, "附件4 id 顺序不一致: %s" % ref_ids[:4]
92 lab_eq = float((np.array(pred) == ref["pred_label"].values).mean())
93 sc_diff = float(np.abs(score - ref["pred_score"].values).max())
94 print("[CHECK] 与 pred_att4.csv: 标签一致率 %.3f; 分数最大差 %.1e" % (lab_eq, sc_diff))
95 assert lab_eq == 1.0, "标签与 pred_att4.csv 不一致, 禁止输出"
96 assert sc_diff < 1e-3, "分数与 pred_att4.csv 不一致 (差 %.4f)" % sc_diff
97
98 # ----- 2) 读解释与证据 -----
99 fused = np.load(os.path.join(a.out_dir, "q3_temporal_fused_att4.npz"), allow_pickle=True)
100 shap = np.load(os.path.join(a.out_dir, "q3_shapley_att4.npz"), allow_pickle=True)
101 met = np.load(os.path.join(a.out_dir, "q3_temporal_occ_att4.npz"), allow_pickle=True)
102 ev = json.load(open(os.path.join(a.out_dir, "q3_evidence_att4.json"), encoding="utf-8"))
103 faith_p = os.path.join(a.out_dir, "q3_faithfulness_att4.csv")
104 faith = pd.read_csv(faith_p, dtype={"id": str}) if os.path.isfile(faith_p) else None
105 meta = {r["id"]: r for r in json.load(
106     open(os.path.join(ROOT, "data_att", "att_meta.json"), encoding="utf-8"))["att4"]}
107
108 W = fused["w_fused"]
109 W_att, W_ig, W_occ = fused["w_att"], fused["w_ig"], fused["w_occ"]
110 valid = fused["valid"]
111 pi = shap["pi_soft"]
112 pi_relu = shap["pi_relu"]
113 phi = shap["phi_mu_miss"]

```

```

114 occ = met["w_occ"]
115 fps_cache = os.path.join(a.out_dir, "video_fps.json")
116
117 rows, cards = [], {}
118 for i, sid in enumerate(ids):
119     n_text = int(valid[i, 0].sum())
120     n_audio = int(valid[i, 1].sum())
121     n_vision = int(valid[i, 2].sum())
122     e = ev.get(sid, {})
123     m = meta.get(sid, {})
124     # 断言: pi 归一化与时间重要性合法性
125     assert abs(float(pi[i].sum()) - 1.0) < 1e-6, "pi 未归一化: %s" % sid
126     assert np.isfinite(W[i]).all() and (W[i] >= -1e-9).all(), "时间重要性非法: %s" %
sid
127
128     te = e.get("text_evidence", {}) or {}
129     tr = te.get("seg_runs") or [[None, None]]
130     ae = e.get("audio_evidence", {}) or {}
131     ar = ae.get("seg_runs") or [[None, None]]
132     ve = e.get("vision_evidence", {}) or {}
133     vr = ve.get("seg_runs") or [[None, None]]
134     kf_t = ve.get("keyframe_time_s")
135     fps = probe_fps(m["video"], fps_cache) if m.get("video") else None
136     frame_idx = int(round(kf_t * fps)) if (kf_t is not None and fps) else None
137
138     # 证据合法性断言
139     if tr and tr[0][0] is not None:
140         assert 0 <= tr[0][0] < max(1, n_text), "文本证据段号非法: %s" % sid
141     if ae.get("end_s") is not None and m.get("duration_s"):
142         assert ae["end_s"] <= m["duration_s"] + 0.05, "语音证据超视频时长: %s" % sid
143
144     # 忠实性/充分性/稳定性/稀疏性 (来自 metrics_xai.py, 取融合解释那一行)
145     compr = suff = stab = spar = None
146     if faith is not None:
147         sub = faith[(faith["id"] == sid) & (faith["source"] == "fused")]
148         if len(sub):
149             compr = float(sub["compr_mean"].iloc[0])
150             suff = float(sub["suff_absmean"].iloc[0])
151             spar = float(sub["sparsity_mean"].iloc[0])
152     rep = json.load(open(os.path.join(a.out_dir, "q3_faithfulness_att4_report.json"),
153                          encoding="utf-8")) if os.path.isfile(
154         os.path.join(a.out_dir, "q3_faithfulness_att4_report.json")) else {}
155     stab = (rep.get("stability", {}).get(sid, {}) or {}).get("stability_fused")
156
157     phrase = te.get("phrase", "")
158     row = dict(

```



```

159     id=sid, pred_label=pred[i], pred_score=round(float(score[i]), 4),
160     prob_neg=round(float(prob[i, 0]), 4), prob_neu=round(float(prob[i, 1]), 4),
161     prob_pos=round(float(prob[i, 2]), 4),
162     confidence=round(float(prob[i].max()), 4), theta=a.theta,
163     main_modality=MODALITIES[int(np.argmax(pi[i]))],
164     w_text=round(float(pi[i, 0]), 4), w_audio=round(float(pi[i, 1]), 4),
165     w_vision=round(float(pi[i, 2]), 4),
166     w_text_relu=round(float(pi_relu[i, 0]), 4),
167     w_audio_relu=round(float(pi_relu[i, 1]), 4),
168     w_vision_relu=round(float(pi_relu[i, 2]), 4),
169     phi_text=round(float(phi[i, 0]), 4), phi_audio=round(float(phi[i, 1]), 4),
170     phi_vision=round(float(phi[i, 2]), 4),
171     text_evidence=phrase or te.get("note", ""),
172     text_seg_start=tr[0][0], text_seg_end=tr[0][1],
173     text_char_start=(te.get("phrase_char_span") or [None, None])[0],
174     text_char_end=(te.get("phrase_char_span") or [None, None])[1],
175     audio_start_s=ae.get("start_s"), audio_end_s=ae.get("end_s"),
176     audio_seg_start=ar[0][0], audio_seg_end=ar[0][1],
177     vision_seg_start=vr[0][0], vision_seg_end=vr[0][1],
178     vision_frame_idx=frame_idx, vision_time_s=kf_t,
179     vision_keyframe=ve.get("keyframe_file", ""),
180     compr=None if compr is None else round(compr, 4),
181     suff=None if suff is None else round(suff, 4),
182     stability=stab, sparsity=None if spar is None else round(spar, 4),
183 )
184 rows.append(row)
185 cards[sid] = dict(
186     id=sid, pred_label=pred[i], pred_score=round(float(score[i]), 4),
187     prob={LABELS[k]: round(float(prob[i, k]), 4) for k in range(3)},
188     confidence=round(float(prob[i].max()), 4), theta=a.theta,
189     main_modality=row["main_modality"],
190     modality_contribution={MODALITIES[k]: round(float(pi[i, k]), 4) for k in
range(3)},
191     modality_contribution_relu={MODALITIES[k]: round(float(pi_relu[i, k]), 4)
for k in range(3)},
192     shapley_raw={MODALITIES[k]: round(float(phi[i, k]), 4) for k in range(3)},
193     temporal_importance_fused={MODALITIES[k]: np.round(W[i, k], 4).tolist()
for k in range(3)},
194     temporal_importance_att={MODALITIES[k]: np.round(W_att[i, k], 4).tolist()
for k in range(3)},
195     temporal_importance_ig={MODALITIES[k]: np.round(W_ig[i, k], 4).tolist()
for k in range(3)},
196     temporal_importance_occ={MODALITIES[k]: np.round(W_occ[i, k], 4).tolist()
for k in range(3)},
197     gamma=fused["gamma"].tolist(),
198     evidence=dict(text=te, audio=ae, vision=ve),
199

```

```

204         raw_text=m.get("raw_text", ""), video=os.path.basename(m.get("video", "") or ""),
205         duration_s=m.get("duration_s"),
206         faithfulness=dict(comprehensiveness=row["compr"], sufficiency=row["suff"],
207                           stability=stab, sparsity=row["sparsity"]))
208
209     df = pd.DataFrame(rows)
210     out_csv = os.path.join(a.sub_dir, "pred_explain_att4.csv")
211     df.to_csv(out_csv, index=False, encoding="utf-8-sig")
212     os.makedirs(os.path.join(a.sub_dir, "explain_cards"), exist_ok=True)
213     for sid, c in cards.items():
214         with open(os.path.join(a.sub_dir, "explain_cards", "%s.json" % sid), "w",
215                  encoding="utf-8") as f:
216             json.dump(c, f, ensure_ascii=False, indent=1)
217     save_json(os.path.join(a.out_dir, "q3_explain_meta.json"),
218              dict(meta=q3_meta(a.ckpt_dir, models, a.theta, dict(pk1=os.path.relpath(a.
219                                pk1, ROOT))),
220                  csv=os.path.relpath(out_csv, ROOT), n=N,
221                  columns=list(df.columns)))
222     print("[SAVE] %s (%d 行 x %d 列)" % (os.path.relpath(out_csv, ROOT), len(df), df.shape
223                                         [1]))
224     print("[SAVE] submission/explain_cards/*.json (%d 个)" % len(cards))
225     cnt = df["main_modality"].value_counts().to_dict()
226     print("[STAT] 主参考模态分布: %s" % cnt)
227     print("INFER_EXPLAIN_OK")
228     return 0
229
230 if __name__ == "__main__":
231     sys.exit(main())

```

### C.13 问题三：交付自查（17 项一致性校验）

code/03\_xai/verify\_q3.py（共 150 行）

```

1  #-*- coding: utf-8 -*-
2  r"""S9: 问题3 交付自查（任一断言不过即失败）。
3
4  检查项：
5  1. pred_explain_att4.csv 行数 = 20、id 顺序与附件4 文件名一致、列齐全；
6  2. 其 pred_label / pred_score 与 submission/pred_att4.csv 逐位一致；
7  3. 每行模态作用度之和为 1；时间重要性数值合法（在卡片 JSON 中抽检）；
8  4. 证据区间合法（段号、秒区间不超过视频时长）；
9  5. 关键帧文件存在且非空；
10 6. 解释卡 JSON 每个 id 齐全；
11 7. submission/ 总体积 <= 50 MB；
12 8. 匿名性：CSV/JSON 中不含单位/队号/学号/手机号/邮箱等身份信息。

```

```

13
14 用法:
15 python verify_q3.py --sub_dir ../\submission --out_dir ../\runs\q3
16 """
17 import argparse
18 import json
19 import os
20 import re
21 import sys
22
23 import numpy as np
24 import pandas as pd
25
26 HERE = os.path.dirname(os.path.abspath(__file__))
27 if HERE not in sys.path:
28     sys.path.append(HERE)
29 from xai_common import save_json, ROOT          # noqa: E402
30
31 BANNED = [r"参赛队", r"队号", r"学号", r"指导教[师]?", r"大[学]?[\u4e00-\u9fff]{0,6}学院",
32           r"1[3-9]\d{9}", r"[\w.\-]+@[ \w.\-]+\.\w+", r"[\u4e00-\u9fff]{2,4}(同学|老师)"]
33
34
35 def main():
36     ap = argparse.ArgumentParser()
37     ap.add_argument("--sub_dir", default=os.path.join(ROOT, "submission"))
38     ap.add_argument("--out_dir", default=os.path.join(ROOT, "runs", "q3"))
39     ap.add_argument("--max_mb", type=float, default=50.0)
40     a = ap.parse_args()
41     checks, fails = [], []
42
43     def ck(name, ok, detail=""):
44         checks.append(dict(check=name, ok=bool(ok), detail=str(detail)))
45         if not ok:
46             fails.append("%s: %s" % (name, detail))
47         print(" [%s] %-34s %s" % ("OK" if ok else "FAIL", name, detail))
48
49     # 1) 行数 / id / 列
50     path = os.path.join(a.sub_dir, "pred_explain_att4.csv")
51     ck("文件存在", os.path.isfile(path), os.path.relpath(path, ROOT))
52     df = pd.read_csv(path, dtype={"id": str})
53     df["id"] = [str(x).zfill(2) if str(x).isdigit() else str(x) for x in df["id"]]
54     meta4 = json.load(open(os.path.join(ROOT, "data_att", "att_meta.json"),
55                           encoding="utf-8"))
56     ids = [r["id"] for r in meta4["att4"]]
57     ck("行数=20", len(df) == 20, "实为 %d" % len(df))
58     ck("id 顺序与附件4 一致", list(df["id"]) == ids, "%s" % list(df["id"])[ :4])

```

```

59 need = ["id", "pred_label", "pred_score", "prob_neg", "prob_neu", "prob_pos",
60         "main_modality", "w_text", "w_audio", "w_vision", "text_evidence",
61         "text_seg_start", "text_seg_end", "audio_start_s", "audio_end_s",
62         "vision_frame_idx", "vision_time_s", "compr", "suff", "stability", "sparsity"]
63 miss = [c for c in need if c not in df.columns]
64 ck("必需列齐全", not miss, "缺%s" % miss)
65
66 # 2) 与 pred_att4.csv 一致
67 ref = pd.read_csv(os.path.join(a.sub_dir, "pred_att4.csv"), dtype={"id": str})
68 ck("标签与 pred_att4.csv 一致",
69     float((df["pred_label"].values == ref["pred_label"].values).mean()) == 1.0)
70 ck("分数与 pred_att4.csv 一致",
71     float(np.abs(df["pred_score"].values - ref["pred_score"].values).max()) < 1e-3,
72     "最大差 %2e" % float(np.abs(df["pred_score"].values - ref["pred_score"].values).max()))
73
74 # 3) pi 归一化
75 s = df[["w_text", "w_audio", "w_vision"]].values.sum(1)
76 ck("模态作用度之和=1", float(np.abs(s - 1).max()) < 2e-3, "最大偏差 %2e"
77     % float(np.abs(s - 1).max()))
78 ck("主模态取值合法",
79     set(df["main_modality"]) <= {"text", "audio", "vision"},
80     "%s" % sorted(set(df["main_modality"])))
81
82 # 4) 证据合法性
83 bad_seg = df[(df["text_seg_start"].notna()) & (df["text_seg_end"].notna()) &
84              (~(df["text_seg_start"] >= 0) & (df["text_seg_end"] <= 50) &
85               (df["text_seg_start"] < df["text_seg_end"])))])
86 ck("文本段号合法", len(bad_seg) == 0, "越界 %d 行" % len(bad_seg))
87 dur = {r["id"]: r.get("duration_s") for r in meta4["att4"]}
88 bad_t = [i for i, r in df.iterrows()
89          if pd.notna(r["audio_end_s"]) and dur.get(r["id"]) and
90             r["audio_end_s"] > dur[r["id"]] + 0.05]
91 ck("语音时段不超视频时长", len(bad_t) == 0, "越界 %d 行" % len(bad_t))
92 ck("含文本证据的样本数>0", int(df["text_evidence"].notna().sum()) > 0,
93     "%d/20" % int(df["text_evidence"].notna().sum()))
94
95 # 5) 关键帧
96 kf = [os.path.join(ROOT, p) for p in df["vision_keyframe"].dropna().astype(str) if p]
97 ok_kf = [p for p in kf if os.path.isfile(p) and os.path.getsize(p) > 500]
98 ck("关键帧文件有效", len(ok_kf) >= max(1, int(0.8 * len(kf))),
99     "%d/%d 张可用" % (len(ok_kf), len(kf)))
100
101 # 6) 解释卡
102 cdir = os.path.join(a.sub_dir, "explain_cards")
103 have = sorted(os.path.splitext(f)[0] for f in os.listdir(cdir) if os.path.isdir(cdir
104 ) else [])

```

```

104 ck("解释卡 JSON 齐全", set(have) == set(ids), "%d/20" % len(have))
105 if set(have) >= set(ids):
106     c = json.load(open(os.path.join(cdir, ids[0] + ".json"), encoding="utf-8"))
107     ti = c["temporal_importance_fused"]["text"]
108     ck("时间重要性长度=50", len(ti) == 50, "len=%d" % len(ti))
109     ck("时间重要性非负且有界", bool(np.min(ti) >= -1e-6 and np.max(ti) <= 1 + 1e-6))
110
111 # 7) 体积
112 tot = 0
113 for r, _, fs in os.walk(a.sub_dir):
114     if "_backup" in r:
115         continue
116     for f in fs:
117         tot += os.path.getsize(os.path.join(r, f))
118 mb = tot / 1024 / 1024
119 ck("submission 体积 <= %.0fMB" % a.max_mb, mb <= a.max_mb, "%.2fMB" % mb)
120
121 # 8) 匿名性
122 hits = []
123 for r, _, fs in os.walk(a.sub_dir):
124     if "_backup" in r:
125         continue
126     for f in fs:
127         if not f.lower().endswith((".csv", ".json", ".md", ".txt")):
128             continue
129         p = os.path.join(r, f)
130         try:
131             txt = open(p, encoding="utf-8", errors="replace").read()
132         except OSError:
133             continue
134         for pat in BANNED:
135             for m in re.finditer(pat, txt):
136                 hits.append("%s :: %s" % (os.path.relpath(p, ROOT), m.group(0)))
137 ck("无身份信息", not hits, "%d 处命中" % len(hits))
138 if hits:
139     print("    %s" % hits[:5])
140
141 rep = dict(n_check=len(checks), n_fail=len(fails), checks=checks, fails=fails,
142           submission_mb=round(mb, 2))
143 save_json(os.path.join(a.out_dir, "verify_q3.json"), rep)
144 print("\n%s (%d 项检查, %d 项失败)" % ("VERIFY_PASS" if not fails else "VERIFY_FAIL",
145                                     len(checks), len(fails)))
146 return 0 if not fails else 1
147
148
149 if __name__ == "__main__":

```

```
150 sys.exit(main())
```